

# Portable Backgammon Match Clock

## Current Firmware Code Walkthrough

ESP32-S3 CrowPanel 5-inch / 800x480 / GT911

|                |                                       |
|----------------|---------------------------------------|
| Source         | <code>BackgammonMatchClock.ino</code> |
| Source size    | 4,357 lines at publication            |
| Firmware label | <code>lcd-refresh-2026-06-07</code>   |
| Manual edition | September 2026 update                 |

Hardware, wiring, state model, timing rules, UI, persistence, diagnostics, and complete source

# Contents

|                                          |          |
|------------------------------------------|----------|
| <b>1 Purpose and Scope</b>               | <b>3</b> |
| <b>2 System Architecture</b>             | <b>3</b> |
| 2.1 Main runtime cycle . . . . .         | 3        |
| <b>3 Hardware Contract</b>               | <b>4</b> |
| 3.1 Target board . . . . .               | 4        |
| 3.2 Shared I2C bus . . . . .             | 4        |
| 3.3 Physical buttons . . . . .           | 4        |
| 3.4 RGB display pins . . . . .           | 4        |
| <b>4 State Model</b>                     | <b>5</b> |
| 4.1 Device modes . . . . .               | 5        |
| 4.2 Core match state . . . . .           | 5        |
| <b>5 Clock Engine</b>                    | <b>5</b> |
| 5.1 Simple delay . . . . .               | 5        |
| 5.2 Bronstein delay . . . . .            | 5        |
| 5.3 Fischer increment . . . . .          | 6        |
| 5.4 Game and match timer scope . . . . . | 6        |
| <b>6 Scoring and Match Lifecycle</b>     | <b>6</b> |
| 6.1 Game controls . . . . .              | 6        |
| 6.2 Crawford and post-Crawford . . . . . | 6        |
| <b>7 User Interface</b>                  | <b>6</b> |
| 7.1 Main-screen geometry . . . . .       | 6        |
| 7.2 Appearance . . . . .                 | 7        |
| 7.3 Fonts . . . . .                      | 7        |
| <b>8 Touch and Button Input</b>          | <b>7</b> |
| <b>9 Rendering Pipeline</b>              | <b>7</b> |
| <b>10 Battery, Charging, and RTC</b>     | <b>8</b> |
| 10.1 MAX17048 . . . . .                  | 8        |
| 10.2 DS3231 . . . . .                    | 8        |
| <b>11 Persistence and Match Log</b>      | <b>8</b> |
| <b>12 Startup and Diagnostics</b>        | <b>8</b> |
| <b>13 Build and Upload</b>               | <b>8</b> |

|                                           |           |
|-------------------------------------------|-----------|
| <b>14 Maintenance Guide</b>               | <b>9</b>  |
| 14.1 Adding a new setting . . . . .       | 9         |
| 14.2 Adding a new screen . . . . .        | 9         |
| 14.3 Changing fonts . . . . .             | 9         |
| 14.4 Changing panel timing . . . . .      | 9         |
| <b>15 Known Limitations</b>               | <b>9</b>  |
| <b>16 Top-Level Function Inventory</b>    | <b>10</b> |
| <b>A Complete Current Firmware Source</b> | <b>11</b> |

# 1 Purpose and Scope

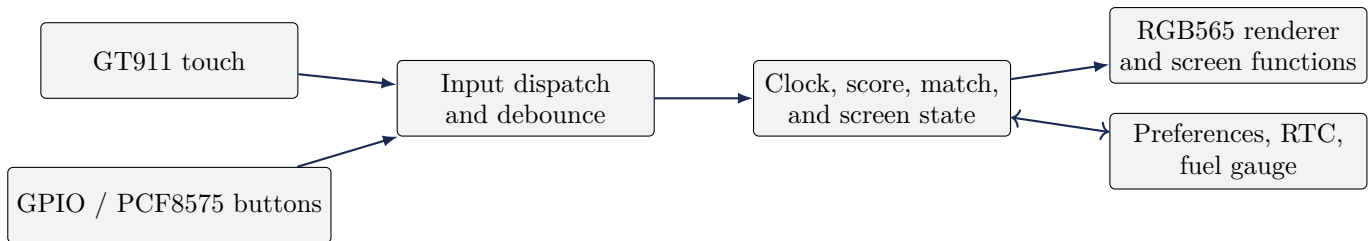
This document explains the current portable backgammon match clock firmware as an engineering and maintenance reference. It supersedes the earlier V2.0 PDF, whose source snapshot was 4,104 lines and predated several user-interface and hardware changes. The present sketch is a single Arduino translation unit with custom display, touch, game, timing, persistence, and peripheral code.

The clock provides:

- two player clocks, scores, editable names, and checker colors;
- touch control plus Player A, Player B, and Play/Pause physical buttons;
- Simple delay, Bronstein delay, and Fischer increment;
- timer-by-game and timer-by-match scopes;
- Crawford and post-Crawford match state;
- day/night themes and Score/Points-to-Win views;
- a MAX17048 fuel gauge, DS3231 RTC, and eight-entry match log;
- a compiled RGB565 splash image and optimized numeric fonts; and
- direct ESP-IDF RGB-panel rendering without LVGL.

# 2 System Architecture

The firmware uses an event-and-state architecture. Input handlers mutate the model, model transitions mark the display dirty, and rendering rebuilds the appropriate screen. Time-sensitive timer regions can be refreshed separately from the rest of the interface.



## 2.1 Main runtime cycle

`setup()` initializes serial diagnostics, button objects, I2C peripherals, the RGB panel, splash screen, preferences, match state, and RTC cache. The order is deliberate: the onboard PCA9557 and touch-reset path must be ready before the GT911 and display are used.

`loop()` performs five recurring jobs:

1. poll touch and physical buttons;
2. update the active clock when in match mode;
3. commit delayed preference writes;
4. evaluate inactivity sleep; and
5. refresh either the whole screen or only the timer panels.

The one-second timer refresh and dirty-screen redraw split reduces visible flashing compared with repainting the entire 800x480 framebuffer continuously.

## 3 Hardware Contract

### 3.1 Target board

The target is the Elecrow CrowPanel ESP32-S3 5-inch HMI with an 800x480 RGB-parallel panel, GT911 capacitive touch controller, onboard PCA9557, and 8 MB OPI PSRAM. PSRAM is mandatory because a full RGB565 frame requires approximately 768,000 bytes before additional buffers and application state.

### 3.2 Shared I2C bus

| Device or signal | Pin/address   | Purpose                                          |
|------------------|---------------|--------------------------------------------------|
| SDA              | GPIO 19       | Shared data line; use 3.3 V pull-ups only.       |
| SCL              | GPIO 20       | Shared clock line.                               |
| PCA9557          | 0x18-0x1F     | CrowPanel reset and board initialization.        |
| PCF8575          | 0x20-0x27     | Three active-low physical buttons.               |
| MAX17048         | 0x36          | Single-cell battery voltage and state of charge. |
| GT911            | normally 0x5D | Capacitive touch coordinates.                    |
| DS3231           | 0x68          | Date and time for the status bar and match log.  |

All devices share SDA, SCL, 3.3 V, and GND. A passive I2C splitter duplicates connectors; it does not create separate buses or resolve address conflicts.

### 3.3 Physical buttons

The PCF8575 is initialized with all port bits high. Each normally-open switch connects its assigned port to ground when pressed.

| Function   | PCF8575 port | Runtime action                    |
|------------|--------------|-----------------------------------|
| Play/Pause | P00          | Start a game, pause, or resume.   |
| Player A   | P01          | End A's turn and start B's clock. |
| Player B   | P07          | End B's turn and start A's clock. |

GPIO 38 remains as compatibility support for the earlier single-button wiring. The direct Player B and score/menu pins use 255, which is the sketch's disabled-pin sentinel.

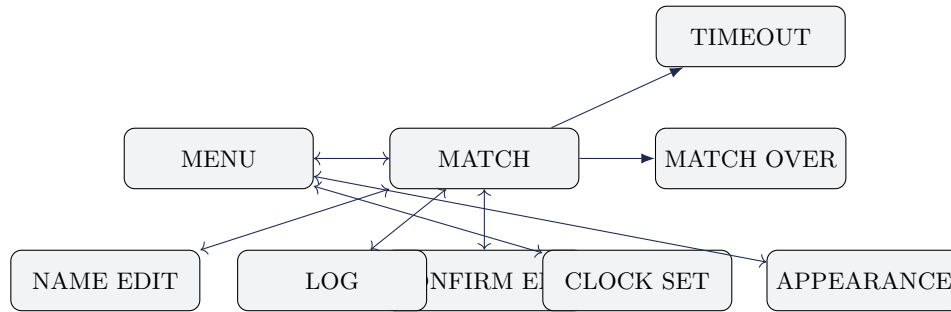
### 3.4 RGB display pins

| Role                         | GPIO assignment                                        |
|------------------------------|--------------------------------------------------------|
| D0-D15                       | 8, 3, 46, 9, 1, 5, 6, 7, 15, 16, 4, 45, 48, 47, 21, 14 |
| DE / VSYNC / HSYNC<br>/ PCLK | 40 / 41 / 39 / 0                                       |
| Backlight                    | 2                                                      |

These pins belong to the panel and must not be reused by external controls. The firmware requests a 20 MHz pixel clock and ten-line bounce buffer.

## 4 State Model

### 4.1 Device modes



`DeviceMode` prevents a touch or physical-button event intended for one screen from mutating another. Modal views such as name editing, RTC setup, and end-match confirmation therefore have their own drawing and tap handlers.

### 4.2 Core match state

Scores, reserve times, active player, running/paused state, game number, Crawford flags, winner, and timeout player are stored independently. This is important: `game_number` advances only in `startNewGame()`, not when a score is manually corrected.

The valid match targets are all odd values from 1 to 21. Scores clamp to the range zero through the target, and match completion occurs at `score >= match_target`; there is no off-by-one requirement.

## 5 Clock Engine

Each player's stored reserve is milliseconds. `turnStartMs` identifies the beginning of the active turn. A pause records `pausedTurnElapsedMs`; a resume reconstructs the original turn start, so the displayed and ultimately charged time continue exactly from the pause point.

Let  $R$  be reserve time,  $E$  elapsed turn time, and  $D$  the configured delay or increment.

### 5.1 Simple delay

The display holds steady during the delay. At turn end:

$$R_{new} = R - \max(0, E - D)$$

Unused delay is discarded.

### 5.2 Bronstein delay

The active display counts down immediately. At turn end, up to  $D$  is refunded, which is equivalent to:

$$R_{new} = R - \max(0, E - D)$$

The distinction from Simple delay is visual during the turn, not the final reserve deduction.

## 5.3 Fischer increment

Elapsed time is deducted and a fixed increment is added:

$$R_{new} = R - E + D$$

A fast move can therefore increase the player's reserve.

## 5.4 Game and match timer scope

In Game scope, a new game resets both reserves to the configured time per game. In Match scope, the initial reserve is:

$$\text{starting seconds} = \text{time per game} \times \text{match target}$$

and carries across games. Reaching zero stops the clock, records the timed-out player, ends the current game, and displays the timeout state. The score penalty is intentionally entered manually.

# 6 Scoring and Match Lifecycle

Score changes are accepted only while the clock is paused or in timeout state. A positive change during a running game first calls `endCurrentGame()`. Negative changes are treated as corrections and do not advance game or Crawford state.

## 6.1 Game controls

**Play Game** Starts a new game or resumes a paused turn.

**Pause Game** Preserves current reserves and elapsed turn state.

**End Game** Stops the current game without resetting match clocks or scores.

**End Match** Opens a confirmation dialog. Confirmation records the current leader and score.

**Reset** Replaces End Match after completion and clears the active match state.

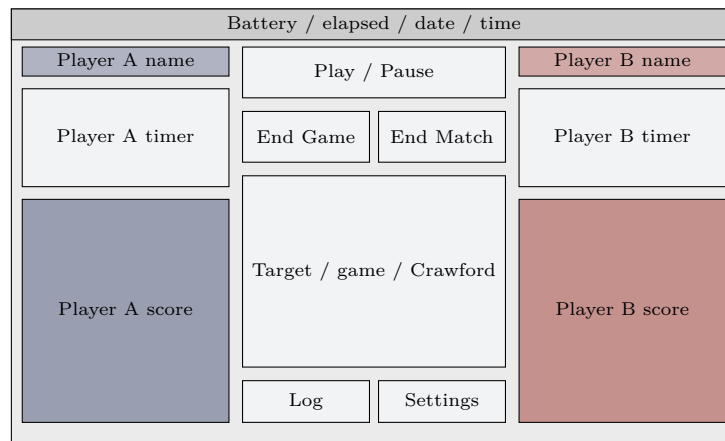
## 6.2 Crawford and post-Crawford

When a player first reaches one-away, the next game is marked Crawford. During that game the doubling cube is disabled. Completion of the Crawford game sets `crawford_used`, clears the active flag, enables the cube, and enters post-Crawford state. The implementation also recognizes the practical case in which the opponent's score changes during the Crawford game.

# 7 User Interface

## 7.1 Main-screen geometry

The screen is partitioned into explicit `Rect` values. Player columns are 228 pixels wide, the center controls are 290 pixels wide, and the 34-pixel status bar spans the top. Large touch hit rectangles extend beyond visible button edges to improve usability.



## 7.2 Appearance

Night mode uses the original black background. Day mode uses a medium-gray overall background and very-light-gray neutral panels while leaving the status bar dark. Player name bars always use the selected player color with white text.

The score view can show either actual Score or Points to Win. Points to Win is display-only: stored scores, Crawford detection, winner detection, and logs all continue to use actual points.

The active timer uses a darker emphasis treatment. Any reserve at or below 90 seconds receives a light-red timer background. Audible warnings begin at 30 seconds and become more frequent at 10 seconds.

The match-target panel also communicates Crawford state. Crawford uses a light-pink panel with black target/game text and a dark-yellow **CRAWFORD** label. Post-Crawford uses a light-blue panel with black target/game text and a dark-orange **POST CRAWFORD** label.

## 7.3 Fonts

The sketch includes optimized numeric-only GFX fonts:

- `7segment48pt7b_numbers.h` for timer digits and colon;
- `collegeb96pt7b_numbers.h` for player scores;
- `FreeSansBold48pt7b_numbers.h` for the match target; and
- `FreeSans12pt7b.h` for general interface text.

Fallback branches allow compilation when an optional font header is absent, but the intended visual result requires the project-local headers.

## 8 Touch and Button Input

The GT911 status register is polled and acknowledged after each data-ready report. Touch coordinates are transformed into the 800x480 orientation and clamped. The handler tracks start, last, minimum, and maximum positions plus a release grace period. This supports reliable taps and vertical score swipes.

The button code debounces both direct GPIO and PCF8575 inputs. During a running turn, only the active player's own clock button can end the turn. Pressing the inactive player's side is ignored, preventing an accidental bump from restarting the delay window.

## 9 Rendering Pipeline

`CrowPanelRGBDisplay` provides the subset of familiar GFX operations used by the application. It wraps the ESP-IDF RGB panel, an RGB565 framebuffer, and a line buffer. Drawing includes fills, outlines, text, scaled GFX glyphs, and image copying.

The renderer has two update paths:

- `drawDisplay()` paints the complete current screen after state or mode changes.
- `drawTimerPanels()` repaints only the two timer regions on the one-second clock cadence.

This division is central to controlling flicker on the RGB panel. New features should call `markDisplayDirty()` only when a complete redraw is needed.

## 10 Battery, Charging, and RTC

### 10.1 MAX17048

The firmware detects the gauge at `0x36`, reads its version, issues a quick-start command, and reads cell voltage and state of charge. The displayed percentage is clamped to zero through 100. The MAX17048 measures the cell; it is not a charger.

If no dedicated active-low charger status pin is configured, the lightning bolt is inferred from a sustained upward voltage or state-of-charge trend. This is an estimate, so a connected charger can be missed near full charge and transient battery recovery can occasionally resemble charging.

### 10.2 DS3231

The RTC supplies date/time for the status bar and log. Clock Settings edits year, month, day, hour, and minute, clamps the day for the selected month, and writes BCD registers. Invalid or unset dates appear as `DATE UNSET`.

## 11 Persistence and Match Log

ESP32 **Preferences** uses namespace `bgclock`. Persisted values include target, time, delay, increment, brightness, timing mode, timer scope, display theme, score view, names, player colors, and up to eight log entries.

General settings use a 1.5-second delayed write to reduce flash wear while a control is tapped repeatedly. Names and colors are saved when editing is confirmed. Each completed match log entry contains date, names, and score.

## 12 Startup and Diagnostics

The expected healthy serial sequence includes:

- PSRAM found: `yes`;
- PCA9557 initialization;
- optional PCF8575 at `0x20-0x27`;
- optional MAX17048 at `0x36`;
- optional DS3231 at `0x68`;
- GT911, usually at `0x5D`; and
- zero return values from RGB panel creation, reset, and initialization.

The current startup goes directly to the compiled `splash4.png` image via `splash_image.h`; the former arcade-style diagnostic loading screen has been removed.

## 13 Build and Upload

Arduino IDE settings:

| Setting        | Value              |
|----------------|--------------------|
| Board          | ESP32S3 Dev Module |
| PSRAM          | OPI PSRAM          |
| Upload speed   | 115200             |
| Serial monitor | 115200 baud        |

The 800x480 framebuffer cannot be allocated if PSRAM is disabled; the usual symptoms are **no mem for frame buffer** and a white screen. Uploads at 921600 have also stopped responding on this unit, so 115200 is the reliable setting.

Repository helper commands:

```
sh scripts/compile-arduino.sh
sh scripts/upload-arduino.sh /dev/cu.usbserial-XXXXX
```

## 14 Maintenance Guide

### 14.1 Adding a new setting

Add model state and a default, render its control, handle its tap, validate it in `loadSettings()`, write it in `saveSettingsIfNeeded()`, and call `markSettingsDirty()` after changes. If it affects the active match, decide explicitly whether it is allowed mid-game.

### 14.2 Adding a new screen

Add a `DeviceMode`, drawing function, hit rectangles, and mode-specific tap branch. Ensure physical buttons either have a meaningful action or return without mutating match state. Set the destination mode and mark the display dirty on every entry and exit.

### 14.3 Changing fonts

Keep the exported symbol name synchronized with the include and drawing call. Numeric-only trimming saves substantial flash, but the glyph range and lookup indices must remain valid. Compile immediately after replacing a generated header and verify timer, score, and target independently.

### 14.4 Changing panel timing

Treat pixel-clock and sync changes as hardware changes. Test splash and static screens as well as timer updates; a setting that seems stable on a simple fill can still show tearing with image data. Keep OPI PSRAM enabled.

## 15 Known Limitations

- Charging status is inferred unless a charger STAT signal is wired.
- Match logs are capped at eight entries and use Preferences rather than a filesystem.
- Scores and timeout penalties remain intentionally manual.
- The UI uses fixed 800x480 coordinates and is not resolution-independent.
- The single-file sketch is approachable but increasingly large; future maintenance would benefit from separating hardware, match logic, and screens.

## 16 Top-Level Function Inventory

| Function group                         | Responsibility                                     |
|----------------------------------------|----------------------------------------------------|
| setup, loop                            | Initialization and cooperative runtime scheduling. |
| handleButtons, handleTouch             | Debounce, gesture recognition, and input dispatch. |
| pauseClock, resumeClock, switchClockTo | Turn and pause lifecycle.                          |
| applyTurnElapsed, visibleReserveMs     | Timing-mode accounting and live display value.     |
| adjustScore, checkMatchState           | Legal score updates and winner detection.          |
| startNewGame, endCurrentGame           | Explicit game counter and per-game reset boundary. |
| markCrawfordUsed                       | Crawford to post-Crawford transition.              |
| forceEndMatch, resetMatch              | Manual match completion and clean restart.         |
| drawDisplay, drawTimerPanels           | Full and partial rendering entry points.           |
| drawNameEditor, drawMatchLog           | Player customization and history views.            |
| drawClockSettings                      | RTC editor.                                        |
| drawAppearanceSettings                 | Theme and score-view editor.                       |
| initCrowPanelExpander, initTouch       | Board and GT911 setup.                             |
| initButtonExpander                     | PCF8575 discovery and initialization.              |
| initFuelGauge, initRtcClock            | Optional battery and time peripherals.             |
| loadSettings, saveSettingsIfNeeded     | Preferences validation and delayed commits.        |
| recordMatchLog, clearMatchLog          | Persistent result history.                         |
| maybeSleep, setBrightness              | Power-facing display behavior.                     |

## A Complete Current Firmware Source

This appendix is included directly from the project sketch at LaTeX build time. Its line numbers therefore correspond to the exact source used to generate this edition.

```

1  /*
2   Portable Backgammon Match Clock and Scoreboard
3   Target: Elecrow CrowPanel ESP32 HMI 5.0-inch display (DIS07050H)
4
5   Display library:
6       This sketch uses the ESP32-S3 RGB LCD peripheral for the built-in 800x480
7       RGB-parallel panel.
8
9   Clock model:
10      - At the start of each game, no clock is running.
11      - Pressing Player A's large button ends A's turn and starts B's clock.
12      - Pressing Player B's large button ends B's turn and starts A's clock.
13      - Optional delay/increment is supported. Delay is Bronstein/simple delay:
14        main time does not tick down until the active turn exceeds delaySeconds.
15      - Increment, if enabled, is added to the player whose turn was ended.
16
17   Crawford model:
18      - When either player reaches match_target - 1 and Crawford has not been used,
19        the following game is marked Crawford and the cube is disabled.
20      - After that game ends, all later games are post-Crawford and cube is enabled.
21  */
22
23  #include <Arduino.h>
24  #include <Wire.h>
25  #include <stdarg.h>
26  #include <Preferences.h>
27  #include <esp_sleep.h>
28  #include <esp_heap_caps.h>
29  #include <esp_lcd_panel_ops.h>
30  #include <esp_lcd_panel_rgb.h>
31  #include "splash_image.h"
32
33  #if __has_include(<gfxfont.h>)
34  #include <gfxfont.h>
35  #else
36  typedef struct {
37      uint16_t bitmapOffset;
38      uint8_t width;
39      uint8_t height;
40      uint8_t xAdvance;
41      int8_t xOffset;
42      int8_t yOffset;
43  } GFXglyph;
44
45  typedef struct {
46      uint8_t *bitmap;
47      GFXglyph *glyph;
48      uint16_t first;
49      uint16_t last;
50      uint16_t yAdvance;
51  } GFXfont;
52  #endif
53
54  #if __has_include("7segment48pt7b_numbers.h")
55  #include "7segment48pt7b_numbers.h"
56  #define BGMC_HAS_CLOCK_FONT 1
57  #define BGMC_CLOCK_FONT SevenSegment48pt7bNumbers
58  #elif __has_include("Seven_Segment48pt7b_numbers.h")
59  #include "Seven_Segment48pt7b_numbers.h"
60  #define BGMC_HAS_CLOCK_FONT 1
61  #define BGMC_CLOCK_FONT Seven_Segment48pt7bNumbers
62  #elif __has_include("Seven_Segment48pt7b.h")
63  #include "Seven_Segment48pt7b.h"
64  #define BGMC_HAS_CLOCK_FONT 1
65  #define BGMC_CLOCK_FONT Seven_Segment48pt7b
66  #elif __has_include("Seven_Segment24pt7b.h")
67  #include "Seven_Segment24pt7b.h"
68  #define BGMC_HAS_CLOCK_FONT 1
69  #define BGMC_CLOCK_FONT Seven_Segment24pt7b
70  #elif __has_include("FreeMonoBold24pt7b.h")
71  #include "FreeMonoBold24pt7b.h"
72  #define BGMC_HAS_CLOCK_FONT 1
73  #define BGMC_CLOCK_FONT FreeMonoBold24pt7b
74  #elif __has_include(<Fonts/FreeMonoBold24pt7b.h>)
75  #include <Fonts/FreeMonoBold24pt7b.h>
76  #define BGMC_HAS_CLOCK_FONT 1
77  #define BGMC_CLOCK_FONT FreeMonoBold24pt7b
78  #else
79  #define BGMC_HAS_CLOCK_FONT 0

```

```

80 #endif
81
82 #if __has_include("FreeSans18pt7b.h")
83 #include "FreeSans18pt7b.h"
84 #define BGMC_HAS_UI_FONT 1
85 #elif __has_include(<Fonts/FreeSans18pt7b.h>)
86 #include <Fonts/FreeSans18pt7b.h>
87 #define BGMC_HAS_UI_FONT 1
88 #else
89 #define BGMC_HAS_UI_FONT 0
90 #endif
91
92 #if __has_include("FreeSans12pt7b.h")
93 #include "FreeSans12pt7b.h"
94 #define BGMC_HAS_SMALL_UI_FONT 1
95 #elif __has_include(<Fonts/FreeSans12pt7b.h>)
96 #include <Fonts/FreeSans12pt7b.h>
97 #define BGMC_HAS_SMALL_UI_FONT 1
98 #else
99 #define BGMC_HAS_SMALL_UI_FONT 0
100 #endif
101
102 #if __has_include("FreeSansBold96pt7b.h")
103 #include "FreeSansBold96pt7b.h"
104 #define BGMC_HAS_SCORE_96_FONT 1
105 #elif __has_include(<Fonts/FreeSansBold96pt7b.h>)
106 #include <Fonts/FreeSansBold96pt7b.h>
107 #define BGMC_HAS_SCORE_96_FONT 1
108 #else
109 #define BGMC_HAS_SCORE_96_FONT 0
110 #endif
111
112 #if __has_include("collegeb96pt7b_numbers.h")
113 #include "collegeb96pt7b_numbers.h"
114 #define BGMC_HAS_COLLEGE_SCORE_FONT 1
115 #define BGMC_COLLEGE_SCORE_FONT collegeb96pt7bNumbers
116 #elif __has_include("college96pt7b_numbers.h")
117 #include "college96pt7b_numbers.h"
118 #define BGMC_HAS_COLLEGE_SCORE_FONT 1
119 #define BGMC_COLLEGE_SCORE_FONT college96pt7bNumbers
120 #elif __has_include("collegeb96pt7b.h")
121 #include "collegeb96pt7b.h"
122 #define BGMC_HAS_COLLEGE_SCORE_FONT 1
123 #define BGMC_COLLEGE_SCORE_FONT collegeb96pt7b
124 #else
125 #define BGMC_HAS_COLLEGE_SCORE_FONT 0
126 #endif
127
128 #if __has_include("FreeSansBold48pt7b_numbers.h")
129 #include "FreeSansBold48pt7b_numbers.h"
130 #define BGMC_HAS_MATCH_TARGET_48_FONT 1
131 #define BGMC_MATCH_TARGET_48_FONT FreeSansBold48pt7bNumbers
132 #elif __has_include("FreeSansBold48pt7b.h")
133 #include "FreeSansBold48pt7b.h"
134 #define BGMC_HAS_MATCH_TARGET_48_FONT 1
135 #define BGMC_MATCH_TARGET_48_FONT FreeSansBold48pt7b
136 #elif __has_include(<Fonts/FreeSansBold48pt7b.h>)
137 #include <Fonts/FreeSansBold48pt7b.h>
138 #define BGMC_HAS_MATCH_TARGET_48_FONT 1
139 #define BGMC_MATCH_TARGET_48_FONT FreeSansBold48pt7b
140 #else
141 #define BGMC_HAS_MATCH_TARGET_48_FONT 0
142 #endif
143
144 #if __has_include("FreeSansBold24pt7b.h")
145 #include "FreeSansBold24pt7b.h"
146 #define BGMC_HAS_SCORE_FONT 1
147 #elif __has_include(<Fonts/FreeSansBold24pt7b.h>)
148 #include <Fonts/FreeSansBold24pt7b.h>
149 #define BGMC_HAS_SCORE_FONT 1
150 #else
151 #define BGMC_HAS_SCORE_FONT 0
152 #endif
153
154 // -----
155 // ISO7050H display-first pin map. External buttons can be added later on
156 // available expansion pins; avoid the LCD RGB bus pins below.
157 // -----
158 static const uint8_t PIN_PLAYER_A_BUTTON = 38; // GPIO_D IO38: one-button clock toggle to GND.
159 static const uint8_t PIN_PLAYER_B_BUTTON = 255;
160 static const uint8_t PIN_SCORE_A_PLUS = 255;
161 static const uint8_t PIN_SCORE_A_MINUS = 255;
162 static const uint8_t PIN_SCORE_B_PLUS = 255;
163 static const uint8_t PIN_SCORE_B_MINUS = 255;
164 static const uint8_t PIN_MENU_SELECT = 255;
165 static const uint8_t PIN_MENU_UP = 255;

```

```

166 static const uint8_t PIN_MENU_DOWN = 255;
167 static const uint8_t PIN_BUZZER = 255;
168 static const uint8_t PIN_TFT_BACKLIGHT = 2;
169 static const uint8_t PIN_BATTERY_ADC = 255;
170 static const uint8_t PIN_CHARGE_DETECT = 255; // Optional charger STAT pin, active-low when wired.
171
172 static const uint16_t PANEL_WIDTH = 800;
173 static const uint16_t PANEL_HEIGHT = 480;
174 static const uint8_t PIN_LCD_D0 = 8;
175 static const uint8_t PIN_LCD_D1 = 3;
176 static const uint8_t PIN_LCD_D2 = 46;
177 static const uint8_t PIN_LCD_D3 = 9;
178 static const uint8_t PIN_LCD_D4 = 1;
179 static const uint8_t PIN_LCD_D5 = 5;
180 static const uint8_t PIN_LCD_D6 = 6;
181 static const uint8_t PIN_LCD_D7 = 7;
182 static const uint8_t PIN_LCD_D8 = 15;
183 static const uint8_t PIN_LCD_D9 = 16;
184 static const uint8_t PIN_LCD_D10 = 4;
185 static const uint8_t PIN_LCD_D11 = 45;
186 static const uint8_t PIN_LCD_D12 = 48;
187 static const uint8_t PIN_LCD_D13 = 47;
188 static const uint8_t PIN_LCD_D14 = 21;
189 static const uint8_t PIN_LCD_D15 = 14;
190 static const uint8_t PIN_LCD_DE = 40;
191 static const uint8_t PIN_LCD_VSYNC = 41;
192 static const uint8_t PIN_LCD_HSYNC = 39;
193 static const uint8_t PIN_LCD_PCLK = 0;
194 static const uint8_t PIN_I2C_SDA = 19;
195 static const uint8_t PIN_I2C_SCL = 20;
196 static const uint8_t PCF8575_PLAY_PAUSE_PIN = 0; // PCF8575 P00, button to GND.
197 static const uint8_t PCF8575_PLAYER_A_PIN = 1; // PCF8575 P01, button to GND.
198 static const uint8_t PCF8575_PLAYER_B_PIN = 7; // PCF8575 P07, button to GND.
199 static const uint8_t MAX17048_I2C_ADDRESS = 0x36;
200 static const uint8_t DS3231_I2C_ADDRESS = 0x68;
201
202 // Rotary encoder placeholders. This sketch uses up/down buttons by default.
203 static const uint8_t PIN_ENCODER_A = 255;
204 static const uint8_t PIN_ENCODER_B = 255;
205
206 // Battery divider calibration placeholders.
207 static const float BATTERY_ADC_REF_VOLTS = 3.30f;
208 static const float BATTERY_DIVIDER_RATIO = 2.00f; // Vbat = Vadc * ratio.
209 static const float BATTERY_EMPTY_VOLTS = 3.30f;
210 static const float BATTERY_FULL_VOLTS = 4.20f;
211
212 // Display/backlight defaults.
213 static const uint8_t DEFAULT_BRIGHTNESS = 210; // 0..255 PWM duty.
214 static const bool TFT_BACKLIGHT_ACTIVE_HIGH = true;
215 static const uint16_t ILI9341_BLACK = 0x0000;
216 static const uint16_t ILI9341_WHITE = 0xFFFF;
217 static const uint16_t ILI9341_RED = 0xF800;
218 static const uint16_t ILI9341_GREEN = 0x07E0;
219 static const uint16_t ILI9341_BLUE = 0x001F;
220 static const uint16_t ILI9341_CYAN = 0x07FF;
221 static const uint16_t ILI9341_YELLOW = 0xFFE0;
222 static const uint16_t TFT_BACKGROUND = ILI9341_BLACK;
223 static const uint16_t TFT_PANEL = 0x2104;
224 static const uint16_t TFT_PANEL_ALT = 0x3186;
225 static const uint16_t TFT_BORDER = 0x7BEF;
226 static const uint16_t TFT_BUTTON_BORDER = 0xD6BA;
227 static const uint16_t TFT_TEXT = ILI9341_WHITE;
228 static const uint16_t TFT_TEXT_DARK = ILI9341_BLACK;
229 static const uint16_t TFT_ACTIVE_TEXT = 0x4208;
230 static const uint16_t TFT_MUTED = 0xBDF7;
231 static const uint16_t TFT_ACCENT_A = ILI9341_CYAN;
232 static const uint16_t TFT_ACCENT_B = 0xFD20;
233 static const uint16_t TFT_MAGENTA = 0xF81F;
234 static const uint16_t TFT_WARN = ILI9341_YELLOW;
235 static const uint16_t TFT_DANGER = ILI9341_RED;
236 static const uint16_t TFT_OK = ILI9341_GREEN;
237 static const uint16_t TFT_TIMER_WARNING_BG = 0xFBCF; // Light red for the final 90 seconds.
238 static const uint16_t TFT_CRAWFORD_BG = 0xFEB5; // Light pink (#F8C8DC).
239 static const uint16_t TFT_CRAWFORD_LABEL = 0xBC21; // Dark yellow (#B8860B).
240 static const uint16_t TFT_POST_CRAWFORD_BG = 0xAEDC; // Light blue (#ADD8E6).
241 static const uint16_t TFT_POST_CRAWFORD_TEXT = 0xC2E0; // Dark orange (#C65D00).
242 static const uint16_t PLAYER_COLOR_RED = 0x7800; // #7C0A02 approximated in RGB565.
243 static const uint16_t PLAYER_COLOR_BLUE = 0x194A; // #1C2951 approximated in RGB565.
244 static const uint16_t PLAYER_COLOR_GREEN = 0x19A5; // #1C352D approximated in RGB565.
245 static const uint16_t PLAYER_COLOR_YELLOW = 0xFD20; // #FFA700 approximated in RGB565.
246 static const uint16_t PLAYER_COLOR_GRAY = 0x4208;
247 static const uint16_t PLAYER_COLOR_WHITE = ILI9341_WHITE;
248
249 // Time behavior.
250 static const uint32_t DEFAULT_INITIAL_TIME_SECONDS = 15UL * 60UL;
251 static const uint8_t DEFAULT_DELAY_SECONDS = 12;

```

```

252 static const uint8_t DEFAULT_INCREMENT_SECONDS = 0;
253 static const uint32_t LOW_TIME_WARNING_SECONDS = 30;
254 static const uint32_t CRITICAL_TIME_WARNING_SECONDS = 10;
255 static const uint32_t SLEEP_AFTER_IDLE_MS = 10UL * 60UL * 1000UL;
256 static const uint32_t LCD_PIXEL_CLOCK_HZ = 20000000;
257 static const uint16_t LCD_BOUNCE_BUFFER_LINES = 10;
258 static const uint8_t PLAYER_NAME_MAX_LEN = 10;
259
260 // Debounce intervals. Large buttons stay snappy; score/menu buttons are slower.
261 static const uint16_t PLAYER_BUTTON_DEBOUNCE_MS = 25;
262 static const uint16_t SMALL_BUTTON_DEBOUNCE_MS = 90;
263 static const uint16_t DISPLAY_REFRESH_MS = 1000;
264 static const uint16_t SETTINGS_SAVE_DELAY_MS = 1500;
265 static const char *FIRMWARE_VERSION = "lcd-refresh-2026-06-07";
266 static const uint8_t MATCH_LOG_MAX_ENTRIES = 8;
267
268 void logLine(const char *message);
269 void logPrintf(const char *format, ...);
270
271 // -----
272 // Types
273 // -----
274 enum Player : uint8_t {
275     PLAYER_A = 0,
276     PLAYER_B = 1,
277     PLAYER_NONE = 255
278 };
279
280 enum TimingMode : uint8_t {
281     SIMPLE_DELAY,
282     BRONSTEIN_DELAY,
283     FISCHER_INCREMENT
284 };
285
286 enum TimerScope : uint8_t {
287     TIMER_BY_GAME,
288     TIMER_BY_MATCH
289 };
290
291 enum DisplayTheme : uint8_t {
292     THEME_NIGHT,
293     THEME_DAY
294 };
295
296 enum ScoreDisplayMode : uint8_t {
297     SCORE_STANDARD,
298     SCORE_POINTS_TO_WIN
299 };
300
301 enum DeviceMode : uint8_t {
302     MODE_MENU,
303     MODE_MATCH,
304     MODE_MATCH_OVER,
305     MODE_TIMEOUT,
306     MODE_NAME_EDIT,
307     MODE_LOG,
308     MODE_CLOCK_SET,
309     MODE_APPEARANCE,
310     MODE_CONFIRM_END_MATCH
311 };
312
313 enum MenuItem : uint8_t {
314     MENU_MATCH_TARGET,
315     MENU_INITIAL_TIME,
316     MENU_DELAY,
317     MENU_INCREMENT,
318     MENU_BRIGHTNESS,
319     MENU_START_MATCH,
320     MENU_RESET_MATCH,
321     MENU_COUNT
322 };
323
324 struct DebouncedButton {
325     uint8_t pin;
326     bool activeLow;
327     bool stablePressed;
328     bool lastRawPressed;
329     bool pressedEvent;
330     uint32_t lastChangeMs;
331     uint16_t debounceMs;
332
333     void begin(uint8_t buttonPin, uint16_t debounce, bool useActiveLow = true) {
334         pin = buttonPin;
335         activeLow = useActiveLow;
336         stablePressed = false;
337         lastRawPressed = false;

```

```

338     pressedEvent = false;
339     lastChangeMs = 0;
340     debounceMs = debounce;
341
342     if (pin != 255) {
343         pinMode(pin, activeLow ? INPUT_PULLUP : INPUT);
344     }
345 }
346
347 void update(uint32_t nowMs) {
348     pressedEvent = false;
349     if (pin == 255) {
350         return;
351     }
352
353     bool rawPressed = activeLow ? (digitalRead(pin) == LOW) : (digitalRead(pin) == HIGH);
354     if (rawPressed != lastRawPressed) {
355         lastRawPressed = rawPressed;
356         lastChangeMs = nowMs;
357     }
358
359     if ((nowMs - lastChangeMs) >= debounceMs && rawPressed != stablePressed) {
360         stablePressed = rawPressed;
361         if (stablePressed) {
362             pressedEvent = true;
363         }
364     }
365 }
366
367 bool wasPressed() const {
368     return pressedEvent;
369 }
370 };
371
372 struct ExpanderButton {
373     uint8_t bit;
374     bool stablePressed;
375     bool lastRawPressed;
376     bool pressedEvent;
377     uint32_t lastChangeMs;
378     uint16_t debounceMs;
379
380     void begin(uint8_t expanderBit, uint16_t debounce) {
381         bit = expanderBit;
382         stablePressed = false;
383         lastRawPressed = false;
384         pressedEvent = false;
385         lastChangeMs = 0;
386         debounceMs = debounce;
387     }
388
389     void update(uint32_t nowMs, uint16_t portState) {
390         pressedEvent = false;
391         if (bit >= 16) {
392             return;
393         }
394
395         bool rawPressed = (portState & (1U << bit)) == 0;
396         if (rawPressed != lastRawPressed) {
397             lastRawPressed = rawPressed;
398             lastChangeMs = nowMs;
399         }
400
401         if ((nowMs - lastChangeMs) >= debounceMs && rawPressed != stablePressed) {
402             stablePressed = rawPressed;
403             if (stablePressed) {
404                 pressedEvent = true;
405             }
406         }
407     }
408
409     bool wasPressed() const {
410         return pressedEvent;
411     }
412 };
413
414 struct Rect {
415     int16_t x;
416     int16_t y;
417     int16_t w;
418     int16_t h;
419
420     bool contains(int16_t px, int16_t py) const {
421         return px >= x && py >= y && px < x + w && py < y + h;
422     }
423 }

```

```

424 bool containsPadded(int16_t px, int16_t py, int16_t padX, int16_t padY) const {
425     return px >= x - padX && py >= y - padY && px < x + w + padX && py < y + h + padY;
426 }
427 };
428
429 struct TouchPoint {
430     bool touched;
431     uint16_t x;
432     uint16_t y;
433 };
434
435 // -----
436 // Globals
437 // -----
438 class CrowPanelRGBDisplay {
439 public:
440     CrowPanelRGBDisplay(uint8_t csPin, uint8_t dcPin, uint8_t rstPin)
441         : cs(csPin), dc(dcPin), rst(rstPin) {}
442
443     void begin() {
444         (void)cs;
445         (void)dc;
446         (void)rst;
447
448         esp_lcd_rgb_panel_config_t panelConfig = {};
449         panelConfig.clk_src = LCD_CLK_SRC_DEFAULT;
450         panelConfig.timings.pclk_hz = LCD_PIXEL_CLOCK_HZ;
451         panelConfig.timings.h_res = PANEL_WIDTH;
452         panelConfig.timings.v_res = PANEL_HEIGHT;
453         panelConfig.timings.hsync_pulse_width = 4;
454         panelConfig.timings.hsync_back_porch = 43;
455         panelConfig.timings.hsync_front_porch = 8;
456         panelConfig.timings.vsync_pulse_width = 4;
457         panelConfig.timings.vsync_back_porch = 12;
458         panelConfig.timings.vsync_front_porch = 8;
459         panelConfig.timings.flags.hsync_idle_low = 0;
460         panelConfig.timings.flags.vsync_idle_low = 0;
461         panelConfig.timings.flags.de_idle_high = 0;
462         panelConfig.timings.flags.pclk_active_neg = 1;
463         panelConfig.timings.flags.pclk_idle_high = 0;
464         panelConfig.data_width = 16;
465         panelConfig.bits_per_pixel = 16;
466         panelConfig.num_fbs = 1;
467         panelConfig.bounce_buffer_size_px = PANEL_WIDTH * LCD_BOUNCE_BUFFER_LINES;
468         panelConfig.dma_burst_size = 32;
469         panelConfig.hsync_gpio_num = PIN_LCD_HSYNC;
470         panelConfig.vsync_gpio_num = PIN_LCD_VSYNC;
471         panelConfig.de_gpio_num = PIN_LCD_DE;
472         panelConfig.pclk_gpio_num = PIN_LCD_PCLK;
473         panelConfig.disp_gpio_num = -1;
474         panelConfig.data_gpio_nums[0] = PIN_LCD_D0;
475         panelConfig.data_gpio_nums[1] = PIN_LCD_D1;
476         panelConfig.data_gpio_nums[2] = PIN_LCD_D2;
477         panelConfig.data_gpio_nums[3] = PIN_LCD_D3;
478         panelConfig.data_gpio_nums[4] = PIN_LCD_D4;
479         panelConfig.data_gpio_nums[5] = PIN_LCD_D5;
480         panelConfig.data_gpio_nums[6] = PIN_LCD_D6;
481         panelConfig.data_gpio_nums[7] = PIN_LCD_D7;
482         panelConfig.data_gpio_nums[8] = PIN_LCD_D8;
483         panelConfig.data_gpio_nums[9] = PIN_LCD_D9;
484         panelConfig.data_gpio_nums[10] = PIN_LCD_D10;
485         panelConfig.data_gpio_nums[11] = PIN_LCD_D11;
486         panelConfig.data_gpio_nums[12] = PIN_LCD_D12;
487         panelConfig.data_gpio_nums[13] = PIN_LCD_D13;
488         panelConfig.data_gpio_nums[14] = PIN_LCD_D14;
489         panelConfig.data_gpio_nums[15] = PIN_LCD_D15;
490         panelConfig.flags.fb_in_psram = 1;
491
492         esp_err_t err = esp_lcd_new_rgb_panel(&panelConfig, &panelHandle);
493         logPrintf("esp_lcd_new_rgb_panel: %d\n", err);
494         if (err != ESP_OK) {
495             panelHandle = nullptr;
496             return;
497         }
498
499         err = esp_lcd_panel_reset(panelHandle);
500         logPrintf("esp_lcd_panel_reset: %d\n", err);
501         err = esp_lcd_panel_init(panelHandle);
502         logPrintf("esp_lcd_panel_init: %d\n", err);
503
504         lineBuffer = (uint16_t *)heap_caps_malloc(PANEL_WIDTH * sizeof(uint16_t), MALLOC_CAP_INTERNAL | MALLOC_CAP_8BIT);
505         if (lineBuffer == nullptr) {
506             lineBuffer = (uint16_t *)heap_caps_malloc(PANEL_WIDTH * sizeof(uint16_t), MALLOC_CAP_8BIT);
507         }
508         logPrintf("LCD line buffer: %s\n", lineBuffer == nullptr ? "FAILED" : "OK");
509     }

```

```

510
511 void setRotation(uint8_t rotation) {
512     (void)rotation;
513     screenWidth = PANEL_WIDTH;
514     screenHeight = PANEL_HEIGHT;
515 }
516
517 void fillScreen(uint16_t color) {
518     fillRect(0, 0, screenWidth, screenHeight, color);
519 }
520
521 void setTextSize(uint8_t size) {
522     textSize = (size == 0) ? 1 : size;
523 }
524
525 void setTextColor(uint16_t color, uint16_t background) {
526     textColor = color;
527     textBackground = background;
528 }
529
530 void setFont(const GFXfont *font = nullptr) {
531     gfxFont = font;
532 }
533
534 void setCursor(int16_t x, int16_t y) {
535     cursorX = x;
536     cursorY = y;
537 }
538
539 void print(const String &text) {
540     print(text.c_str());
541 }
542
543 void print(const char *text) {
544     if (text == nullptr) {
545         return;
546     }
547     while (*text) {
548         drawChar(*text++);
549     }
550 }
551
552 void print(int value) {
553     char buffer[16];
554     snprintf(buffer, sizeof(buffer), "%d", value);
555     print(buffer);
556 }
557
558 void print(unsigned int value) {
559     char buffer[16];
560     snprintf(buffer, sizeof(buffer), "%u", value);
561     print(buffer);
562 }
563
564 void print(unsigned long value) {
565     char buffer[20];
566     snprintf(buffer, sizeof(buffer), "%lu", value);
567     print(buffer);
568 }
569
570 void printf(const char *format, ...) {
571     char buffer[80];
572     va_list args;
573     va_start(args, format);
574     vsnprintf(buffer, sizeof(buffer), format, args);
575     va_end(args);
576     print(buffer);
577 }
578
579 int16_t width() const { return screenWidth; }
580 int16_t height() const { return screenHeight; }
581
582 void fillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color) {
583     if (w <= 0 || h <= 0 || x >= screenWidth || y >= screenHeight) {
584         return;
585     }
586     if (x < 0) {
587         w += x;
588         x = 0;
589     }
590     if (y < 0) {
591         h += y;
592         y = 0;
593     }
594     if (x + w > screenWidth) {
595         w = screenWidth - x;

```

```

596     }
597     if (y + h > screenHeight) {
598         h = screenHeight - y;
599     }
600     if (w <= 0 || h <= 0) {
601         return;
602     }
603
604     if (panelHandle == nullptr || lineBuffer == nullptr) {
605         return;
606     }
607
608     for (int16_t col = 0; col < w; col++) {
609         lineBuffer[col] = color;
610     }
611
612     for (int16_t row = 0; row < h; row++) {
613         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + w, y + row + 1, lineBuffer);
614     }
615 }
616
617 void drawRGB565Image(int16_t x, int16_t y, uint16_t w, uint16_t h, const uint16_t *pixels) {
618     if (panelHandle == nullptr || lineBuffer == nullptr || pixels == nullptr) {
619         return;
620     }
621
622     uint16_t drawWidth = w;
623     uint16_t drawHeight = h;
624     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
625         return;
626     }
627     if (x + drawWidth > screenWidth) {
628         drawWidth = screenWidth - x;
629     }
630     if (y + drawHeight > screenHeight) {
631         drawHeight = screenHeight - y;
632     }
633
634     for (uint16_t row = 0; row < drawHeight; row++) {
635         const uint32_t sourceOffset = (uint32_t)row * (uint32_t)w;
636         for (uint16_t col = 0; col < drawWidth; col++) {
637             lineBuffer[col] = pgm_read_word(&pixels[sourceOffset + col]);
638         }
639         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + drawWidth, y + row + 1, lineBuffer);
640     }
641 }
642
643 void drawRGB565ImageScaled2x(int16_t x, int16_t y, uint16_t w, uint16_t h, const uint16_t *pixels) {
644     if (panelHandle == nullptr || lineBuffer == nullptr || pixels == nullptr) {
645         return;
646     }
647
648     uint16_t drawWidth = w * 2;
649     uint16_t drawHeight = h * 2;
650     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
651         return;
652     }
653     if (x + drawWidth > screenWidth) {
654         drawWidth = screenWidth - x;
655     }
656     if (y + drawHeight > screenHeight) {
657         drawHeight = screenHeight - y;
658     }
659
660     for (uint16_t row = 0; row < drawHeight; row++) {
661         const uint32_t sourceOffset = (uint32_t)(row / 2) * (uint32_t)w;
662         for (uint16_t col = 0; col < drawWidth; col++) {
663             lineBuffer[col] = pgm_read_word(&pixels[sourceOffset + (col / 2)]);
664         }
665         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + drawWidth, y + row + 1, lineBuffer);
666     }
667 }
668
669 void drawRoundRect(int16_t x, int16_t y, int16_t w, int16_t h, int16_t radius, uint16_t color) {
670     (void)radius;
671     fillRect(x, y, w, 1, color);
672     fillRect(x, y + h - 1, w, 1, color);
673     fillRect(x, y, 1, h, color);
674     fillRect(x + w - 1, y, 1, h, color);
675 }
676
677 void getTextBounds(const String &text, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
678     if (gfxFont != nullptr) {
679         getGfxTextBounds(text, x, y, x1, y1, w, h);
680         return;
681     }

```

```

682     *x1 = 0;
683     *y1 = 0;
684     *w = text.length() * 6 * textSize;
685     *h = 8 * textSize;
686 }
687
688 private:
689 void writeCommand(uint8_t command) {
690     (void)command;
691 }
692
693 void writeData(uint8_t data) {
694     (void)data;
695 }
696
697 void writeData16(uint16_t data) {
698     (void)data;
699 }
700
701 void setAddressWindow(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1) {
702     writeCommand(0x2A);
703     writeData16(x0);
704     writeData16(x1);
705     writeCommand(0x2B);
706     writeData16(y0);
707     writeData16(y1);
708     writeCommand(0x2C);
709 }
710
711 void drawPixel(int16_t x, int16_t y, uint16_t color) {
712     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
713         return;
714     }
715     fillRect(x, y, 1, 1, color);
716 }
717
718 void getGfxTextBounds(const String &text, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
719     int16_t minX = 32767;
720     int16_t minY = 32767;
721     int16_t maxX = -32768;
722     int16_t maxY = -32768;
723     int16_t cursor = x;
724     uint16_t first = pgm_read_word(&gfxFont->first);
725     uint16_t last = pgm_read_word(&gfxFont->last);
726     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&gfxFont->glyph);
727
728     for (uint16_t i = 0; i < text.length(); i++) {
729         char c = text.charAt(i);
730         if (c < first || c > last) {
731             continue;
732         }
733         GFXglyph glyph;
734         memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
735         if (glyph.width > 0 && glyph.height > 0) {
736             int16_t gx1 = cursor + (glyph.xOffset * textSize);
737             int16_t gy1 = y + (glyph.yOffset * textSize);
738             int16_t gx2 = gx1 + ((int16_t)glyph.width * textSize) - 1;
739             int16_t gy2 = gy1 + ((int16_t)glyph.height * textSize) - 1;
740             if (gx1 < minX) minX = gx1;
741             if (gy1 < minY) minY = gy1;
742             if (gx2 > maxX) maxX = gx2;
743             if (gy2 > maxY) maxY = gy2;
744         }
745         cursor += (int16_t)glyph.xAdvance * textSize;
746     }
747
748     if (maxX < minX || maxY < minY) {
749         *x1 = x;
750         *y1 = y;
751         *w = 0;
752         *h = 0;
753         return;
754     }
755
756     *x1 = minX;
757     *y1 = minY;
758     *w = maxX - minX + 1;
759     *h = maxY - minY + 1;
760 }
761
762 const char *glyphRow(char c, uint8_t row) const {
763     if (c >= 'a' && c <= 'z') {
764         c -= 32;
765     }
766     static const char blank[7][6] = {"00000", "00000", "00000", "00000", "00000", "00000", "00000"};
767     static const char unknown[7][6] = {"11111", "10001", "00010", "00100", "00100", "00000", "00100"};

```

```

768 #define GLYPH(ch, a,b,c,d,e,f,g) case ch: { static const char p[7][6] = {a,b,c,d,e,f,g}; return p[row]; }
769 switch (c) {
770     GLYPH(' ', "00000","00000","00000","00000","00000","00000","00000")
771     GLYPH('0', "01110","10001","10011","10101","11001","10001","01110")
772     GLYPH('1', "00100","01100","00100","00100","00100","00100","01110")
773     GLYPH('2', "01110","10001","00001","00010","00100","01000","11111")
774     GLYPH('3', "11110","00001","00001","01110","00001","00001","11110")
775     GLYPH('4', "00010","00110","01010","10010","11111","00010","00010")
776     GLYPH('5', "11111","10000","10000","11110","00001","00001","11110")
777     GLYPH('6', "01110","10000","10000","11110","10001","10001","01110")
778     GLYPH('7', "11111","00001","00010","00100","01000","01000","01000")
779     GLYPH('8', "01110","10001","10001","01110","10001","10001","01110")
780     GLYPH('9', "01110","10001","10001","01111","00001","00001","01110")
781     GLYPH('A', "01110","10001","10001","11111","10001","10001","10001")
782     GLYPH('B', "11110","10001","10001","11110","10001","10001","11110")
783     GLYPH('C', "01111","10000","10000","10000","10000","10000","01111")
784     GLYPH('D', "11110","10001","10001","10001","10001","10001","11110")
785     GLYPH('E', "11111","10000","10000","11110","10000","10000","11111")
786     GLYPH('F', "11111","10000","10000","11110","10000","10000","10000")
787     GLYPH('G', "01111","10000","10000","10011","10001","10001","01111")
788     GLYPH('H', "10001","10001","10001","11111","10001","10001","10001")
789     GLYPH('I', "01110","00100","00100","00100","00100","00100","01110")
790     GLYPH('J', "00111","00010","00010","00010","00010","10010","01100")
791     GLYPH('K', "10001","10010","10100","11000","10100","10010","10001")
792     GLYPH('L', "10000","10000","10000","10000","10000","10000","11111")
793     GLYPH('M', "10001","11011","10101","10101","10001","10001","10001")
794     GLYPH('N', "10001","11001","10101","10011","10001","10001","10001")
795     GLYPH('O', "01110","10001","10001","10001","10001","10001","01110")
796     GLYPH('P', "11110","10001","10001","11110","10000","10000","10000")
797     GLYPH('Q', "01110","10001","10001","10001","10101","10010","01101")
798     GLYPH('R', "11110","10001","10001","11110","10100","10010","10001")
799     GLYPH('S', "01111","10000","10000","01110","00001","00001","11110")
800     GLYPH('T', "11111","00100","00100","00100","00100","00100","00100")
801     GLYPH('U', "10001","10001","10001","10001","10001","10001","01110")
802     GLYPH('V', "10001","10001","10001","10001","10001","01010","00100")
803     GLYPH('W', "10001","10001","10001","10101","10101","10101","01010")
804     GLYPH('X', "10001","10001","01010","00100","01010","10001","10001")
805     GLYPH('Y', "10001","10001","01010","00100","00100","00100","00100")
806     GLYPH('Z', "11111","00001","00010","00100","01000","10000","11111")
807     GLYPH(':', "00000","00100","00100","00000","00100","00100","00000")
808     GLYPH('.', "00000","00000","00000","00000","00000","01100","01100")
809     GLYPH('-', "00000","00000","00000","11111","00000","00000","00000")
810     GLYPH('/', "00001","00010","00010","00100","01000","01000","10000")
811     GLYPH('+', "00000","00100","00100","11111","00100","00100","00000")
812     GLYPH('%', "11001","11010","00010","00100","01000","01011","10011")
813     default:
814         return (c == '\0') ? blank[row] : unknown[row];
815 }
816 #undef GLYPH
817 }
818
819 void drawChar(char c) {
820     if (c == '\n') {
821         cursorX = 0;
822         cursorY += gfxFont == nullptr ? (8 * textSize) : (pgm_read_word(&gfxFont->yAdvance) * textSize);
823         return;
824     }
825
826     if (gfxFont != nullptr) {
827         drawGfxChar(c);
828         return;
829     }
830
831     for (uint8_t row = 0; row < 7; row++) {
832         const char *bits = glyphRow(c, row);
833         for (uint8_t col = 0; col < 5; col++) {
834             uint16_t color = (bits[col] == '1') ? textColor : textBackground;
835             fillRect(cursorX + col * textSize, cursorY + row * textSize, textSize, textSize, color);
836         }
837     }
838     fillRect(cursorX + 5 * textSize, cursorY, textSize, 7 * textSize, textBackground);
839     cursorX += 6 * textSize;
840 }
841
842 void drawGfxChar(char c) {
843     uint16_t first = pgm_read_word(&gfxFont->first);
844     uint16_t last = pgm_read_word(&gfxFont->last);
845     if (c < first || c > last) {
846         return;
847     }
848
849     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&gfxFont->glyph);
850     const uint8_t *bitmap = (const uint8_t *)pgm_read_ptr(&gfxFont->bitmap);
851     GFXglyph glyph;
852     memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
853

```

```

854     uint16_t bitIndex = 0;
855     uint16_t bitmapOffset = glyph.bitmapOffset;
856     uint8_t bits = 0;
857     for (uint8_t yy = 0; yy < glyph.height; yy++) {
858         for (uint8_t xx = 0; xx < glyph.width; xx++) {
859             if ((bitIndex & 7) == 0) {
860                 bits = pgm_read_byte(&bitmap[bitmapOffset++]);
861             }
862             if (bits & 0x80) {
863                 fillRect(cursorX + (glyph.xOffset + xx) * textSize, cursorY + (glyph.yOffset + yy) * textSize,
864                     textSize, textSize, textColor);
865             }
866             bits <<= 1;
867             bitIndex++;
868         }
869     }
870     cursorX += (int16_t)glyph.xAdvance * textSize;
871 }
872
873 uint8_t cs;
874 uint8_t dc;
875 uint8_t rst;
876 int16_t screenWidth = PANEL_WIDTH;
877 int16_t screenHeight = PANEL_HEIGHT;
878 int16_t cursorX = 0;
879 int16_t cursorY = 0;
880 uint16_t textColor = ILI9341_WHITE;
881 uint16_t textBackground = ILI9341_BLACK;
882 uint8_t textSize = 1;
883 const GFXfont *gfxFont = nullptr;
884 esp_lcd_panel_handle_t panelHandle = nullptr;
885 uint16_t *lineBuffer = nullptr;
886 };
887
888 CrowPanelRGBDisplay tft(255, 255, 255);
889 Preferences prefs;
890
891 DebouncedButton btnPlayerA;
892 DebouncedButton btnPlayerB;
893 DebouncedButton btnScoreAPlus;
894 DebouncedButton btnScoreAMinus;
895 DebouncedButton btnScoreBPlus;
896 DebouncedButton btnScoreBMinus;
897 DebouncedButton btnSelect;
898 DebouncedButton btnUp;
899 DebouncedButton btnDown;
900 ExpanderButton expanderBtnPlayerA;
901 ExpanderButton expanderBtnPlayerB;
902 ExpanderButton expanderBtnPlayPause;
903
904 int playerA_score = 0;
905 int playerB_score = 0;
906 int match_target = 9;
907 uint32_t playerA_time_remaining = DEFAULT_INITIAL_TIME_SECONDS * 1000UL;
908 uint32_t playerB_time_remaining = DEFAULT_INITIAL_TIME_SECONDS * 1000UL;
909 uint32_t match_elapsed_seconds = 0;
910 Player active_player = PLAYER_NONE;
911 bool clock_running = false;
912 bool crawford_used = false;
913 bool crawford_active = false;
914 bool post_crawford = false;
915 bool cube_enabled = true;
916 uint16_t game_number = 0;
917
918 uint32_t initial_time_seconds = DEFAULT_INITIAL_TIME_SECONDS;
919 uint8_t delay_seconds = DEFAULT_DELAY_SECONDS;
920 uint8_t increment_seconds = DEFAULT_INCREMENT_SECONDS;
921 uint8_t brightness = DEFAULT_BRIGHTNESS;
922 TimingMode timingMode = SIMPLE_DELAY;
923 TimerScope timerScope = TIMER_BY_MATCH;
924 DisplayTheme displayTheme = THEME_NIGHT;
925 ScoreDisplayMode scoreDisplayMode = SCORE_STANDARD;
926 String playerA_name = "PLAYER A";
927 String playerB_name = "PLAYER B";
928 uint16_t playerA_color = PLAYER_COLOR_BLUE;
929 uint16_t playerB_color = PLAYER_COLOR_RED;
930 String matchLog[MATCH_LOG_MAX_ENTRIES];
931 uint8_t matchLogCount = 0;
932
933 DeviceMode mode = MODE_MENU;
934 MenuItem currentMenuItem = MENU_MATCH_TARGET;
935 Player timeoutPlayer = PLAYER_NONE;
936 Player matchWinner = PLAYER_NONE;
937 Player editingNameFor = PLAYER_NONE;
938 String editingName;
939 uint16_t editingPlayerColor = PLAYER_COLOR_BLUE;

```

```

940
941 bool pending_crawford_next_game = false;
942 bool game_in_progress = false;
943 bool settingsDirty = false;
944 bool displayDirty = true;
945 bool displayModeKnown = false;
946 DeviceMode lastDrawMode = MODE_MENU;
947 uint32_t settingsDirtyMs = 0;
948 uint32_t lastTickMs = 0;
949 uint32_t turnStartMs = 0;
950 uint32_t pausedTurnElapsedMs = 0;
951 uint32_t lastDisplayMs = 0;
952 uint32_t lastActivityMs = 0;
953 uint32_t lastLowTimeBeepMs = 0;
954 uint8_t gt911Address = 0;
955 bool touchWasDown = false;
956 uint16_t touchStartX = 0;
957 uint16_t touchStartY = 0;
958 uint16_t touchLastX = 0;
959 uint16_t touchLastY = 0;
960 uint16_t touchMinY = 0;
961 uint16_t touchMaxY = 0;
962 uint32_t touchStartMs = 0;
963 uint32_t touchLastSeenMs = 0;
964 uint32_t touchLastHandledMs = 0;
965 uint8_t pca9557Address = 0;
966 uint8_t pcf8575Address = 0;
967 bool max17048Found = false;
968 bool ds3231Found = false;
969 bool lastTouchSeen = false;
970 float lastChargeCheckVoltage = 0.0f;
971 float lastChargeCheckPercent = 0.0f;
972 bool chargeTrendRising = false;
973 uint32_t lastChargeCheckMs = 0;
974 uint32_t chargeIndicatorUntilMs = 0;
975 String cachedRtcStatus = "DATE UNSET";
976 uint32_t lastRtcStatusMs = 0;
977 uint16_t editingRtcYear = 2026;
978 uint8_t editingRtcMonth = 1;
979 uint8_t editingRtcDay = 1;
980 uint8_t editingRtcHour = 12;
981 uint8_t editingRtcMinute = 0;
982
983 const int MATCH_TARGETS[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21};
984 const uint8_t MATCH_TARGET_COUNT = sizeof(MATCH_TARGETS) / sizeof(MATCH_TARGETS[0]);
985
986 const Rect REGION_STATUS = {0, 0, 800, 34};
987 const Rect REGION_HEADER_A = {12, 42, 228, 32};
988 const Rect REGION_TIMER_A = {12, 88, 228, 108};
989 const Rect REGION_SCORE_A = {12, 210, 228, 246};
990 const Rect REGION_PLAY_PAUSE = {255, 42, 290, 56};
991 const Rect REGION_END_GAME = {255, 113, 140, 56};
992 const Rect REGION_END_MATCH = {405, 113, 140, 56};
993 const Rect REGION_MATCH_TARGET = {255, 184, 290, 211};
994 const Rect REGION_LOG = {255, 410, 140, 46};
995 const Rect REGION_SETTINGS = {405, 410, 140, 46};
996 const Rect REGION_HEADER_B = {560, 42, 228, 32};
997 const Rect REGION_TIMER_B = {560, 88, 228, 108};
998 const Rect REGION_SCORE_B = {560, 210, 228, 246};
999 const Rect HIT_PLAY_PAUSE = {245, 36, 310, 70};
1000 const Rect HIT_END_GAME = {245, 106, 150, 70};
1001 const Rect HIT_END_MATCH = {405, 106, 150, 70};
1002 const Rect CONFIRM_NO = {172, 306, 190, 62};
1003 const Rect CONFIRM_YES = {438, 306, 190, 62};
1004
1005 const Rect SETTINGS_MATCH_MINUS = {372, 86, 96, 48};
1006 const Rect SETTINGS_MATCH_PLUS = {672, 86, 96, 48};
1007 const Rect SETTINGS_TIME_MINUS = {372, 166, 96, 48};
1008 const Rect SETTINGS_TIME_PLUS = {672, 166, 96, 48};
1009 const Rect SETTINGS_DELAY_MINUS = {372, 246, 96, 48};
1010 const Rect SETTINGS_DELAY_PLUS = {672, 246, 96, 48};
1011 const Rect SETTINGS_MATCH_ROW_MINUS = {52, 82, 348, 58};
1012 const Rect SETTINGS_MATCH_ROW_PLUS = {400, 82, 348, 58};
1013 const Rect SETTINGS_TIME_ROW_MINUS = {52, 162, 348, 58};
1014 const Rect SETTINGS_TIME_ROW_PLUS = {400, 162, 348, 58};
1015 const Rect SETTINGS_DELAY_ROW_MINUS = {52, 242, 348, 58};
1016 const Rect SETTINGS_DELAY_ROW_PLUS = {400, 242, 348, 58};
1017 const Rect SETTINGS_SCOPE_GAME = {255, 306, 120, 36};
1018 const Rect SETTINGS_SCOPE_MATCH = {392, 306, 120, 36};
1019 const Rect SETTINGS_MODE_SIMPLE = {52, 352, 210, 38};
1020 const Rect SETTINGS_MODE_BRONSTEIN = {295, 352, 210, 38};
1021 const Rect SETTINGS_MODE_FISCHER = {538, 352, 210, 38};
1022 const Rect SETTINGS_APPEARANCE = {52, 410, 170, 44};
1023 const Rect SETTINGS_CLOCK = {240, 410, 130, 44};
1024 const Rect SETTINGS_START = {388, 410, 170, 44};
1025 const Rect SETTINGS_END = {576, 410, 170, 44};

```

```

1026 const Rect SETTINGS_BACK = {24, 24, 108, 44};
1027 const Rect LOG_BACK = {24, 24, 108, 44};
1028 const Rect LOG_CLEAR = {650, 24, 108, 44};
1029 const Rect CLOCK_BACK = {24, 24, 108, 44};
1030 const Rect APPEARANCE_BACK = {24, 24, 108, 44};
1031 const Rect APPEARANCE_NIGHT = {170, 132, 220, 64};
1032 const Rect APPEARANCE_DAY = {410, 132, 220, 64};
1033 const Rect APPEARANCE_SCORE = {170, 286, 220, 64};
1034 const Rect APPEARANCE_POINTS = {410, 286, 220, 64};
1035 const Rect CLOCK_SAVE = {560, 400, 168, 48};
1036 const Rect CLOCK_YEAR_MINUS = {372, 84, 96, 44};
1037 const Rect CLOCK_YEAR_PLUS = {672, 84, 96, 44};
1038 const Rect CLOCK_MONTH_MINUS = {372, 142, 96, 44};
1039 const Rect CLOCK_MONTH_PLUS = {672, 142, 96, 44};
1040 const Rect CLOCK_DAY_MINUS = {372, 200, 96, 44};
1041 const Rect CLOCK_DAY_PLUS = {672, 200, 96, 44};
1042 const Rect CLOCK_HOUR_MINUS = {372, 258, 96, 44};
1043 const Rect CLOCK_HOUR_PLUS = {672, 258, 96, 44};
1044 const Rect CLOCK_MINUTE_MINUS = {372, 316, 96, 44};
1045 const Rect CLOCK_MINUTE_PLUS = {672, 316, 96, 44};
1046
1047 // -----
1048 // Forward declarations requested by the firmware structure.
1049 // -----
1050 void drawDisplay();
1051 void drawTimerPanels();
1052 void drawTimerPanel(Player player, const Rect &region);
1053 void drawSplashImage();
1054 void fillTriangleRows(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color);
1055 void fillCircleRows(int16_t cx, int16_t cy, int16_t radius, uint16_t color);
1056 void drawCenteredInRect(const String &text, const Rect &region, uint8_t textSize, uint16_t color, uint16_t background);
1057 void drawScoreInRect(int score, const Rect &region, uint16_t color, uint16_t background);
1058 void drawMatchTargetInRect(int target, const Rect &region, uint16_t color, uint16_t background);
1059 void drawGfxTextScaledInRect(const GFXfont *font, const String &text, const Rect &region, uint16_t color, uint8_t scaleNum, uint8_t
    scaleDen);
1060 void getGfxTextBoundsScaled(const GFXfont *font, const String &text, uint8_t scaleNum, uint8_t scaleDen, int16_t *x1, int16_t *y1,
    uint16_t *w, uint16_t *h);
1061 void drawGfxTextScaled(const GFXfont *font, const String &text, int16_t x, int16_t y, uint16_t color, uint8_t scaleNum, uint8_t scaleDen)
    ;
1062 void drawMatchWinnerBanner();
1063 void drawTimeoutBanner();
1064 void drawNameEditor();
1065 void drawMatchLog();
1066 void drawClockSettings();
1067 void drawAppearanceSettings();
1068 void drawEndMatchConfirm();
1069 void openClockSettings();
1070 void handleClockSettingsTap(uint16_t x, uint16_t y);
1071 void handleAppearanceSettingsTap(uint16_t x, uint16_t y);
1072 void handleEndMatchConfirmTap(uint16_t x, uint16_t y);
1073 void adjustRtcField(uint8_t field, int delta);
1074 void openNameEditor(Player player);
1075 void handleNameEditorTap(uint16_t x, uint16_t y);
1076 uint16_t colorForPlayer(Player player);
1077 uint16_t textColorForPlayerColor(uint16_t color);
1078 uint16_t playerColorOption(uint8_t index);
1079 uint8_t playerColorOptionCount();
1080 void savePlayerNames();
1081 void recordMatchLog();
1082 void clearMatchLog();
1083 String matchLogDate();
1084 String matchLogPart(const String &entry, uint8_t part);
1085 void handleButtons();
1086 void handleTouch();
1087 void handleTouchTap(uint16_t x, uint16_t y);
1088 void handleTouchSwipe(uint16_t startX, uint16_t startY, uint16_t endX, uint16_t endY);
1089 void updateClock();
1090 void adjustScore(Player player, int delta);
1091 void checkMatchState();
1092 void startNewGame();
1093 void endCurrentGame();
1094 void markCrawfordUsed();
1095 void forceEndMatch();
1096 void resetMatch();
1097 void enterMenu();
1098 void handleMenu();
1099
1100 // Helper declarations.
1101 void loadSettings();
1102 void saveSettingsIfNeeded();
1103 void markSettingsDirty();
1104 void markDisplayDirty();
1105 void setBrightness(uint8_t duty);
1106 void wakeActivity();
1107 void maybeSleep();
1108 void beep(uint16_t frequency, uint16_t durationMs);

```

```

1109 void playButtonBeep();
1110 void playTimeoutBeep();
1111 void playMatchOverBeep();
1112 void pauseClock();
1113 void resumeClock();
1114 void togglePlayPause();
1115 void switchClockTo(Player nextPlayer);
1116 void addIncrementTo(Player player);
1117 bool applyTurnElapsed(Player player, uint32_t nowMs);
1118 uint32_t visibleReserveMs(Player player, uint32_t nowMs);
1119 uint32_t *reserveFor(Player player);
1120 uint32_t matchClockSeconds();
1121 uint32_t startingClockMs();
1122 String formatTimeMs(uint32_t milliseconds);
1123 String timingModeLabel();
1124 String playPauseLabel();
1125 void drawCentered(const String &text, int16_t y, uint8_t textSize, uint16_t color);
1126 void drawPlayerPanel(Player player, int16_t x, int16_t y, int16_t w, int16_t h);
1127 void drawStatusBar();
1128 bool dayThemeEnabled();
1129 uint16_t uiBackgroundColor();
1130 uint16_t uiPanelColor();
1131 uint16_t uiTextColor();
1132 uint16_t uiMutedColor();
1133 void drawChargeBolt(int16_t x, int16_t y, uint16_t color, uint16_t background);
1134 void drawStartupTest();
1135 void initCrowPanelExpander();
1136 void initButtonExpander();
1137 void initFuelGauge();
1138 void initRtcClock();
1139 bool pca9557Write(uint8_t address, uint8_t reg, uint8_t value);
1140 uint8_t pca9557Read(uint8_t address, uint8_t reg);
1141 bool pcf8575Write(uint8_t address, uint16_t value);
1142 uint16_t pcf8575Read(uint8_t address);
1143 bool max17048Read16(uint8_t reg, uint16_t *value);
1144 bool max17048Write16(uint8_t reg, uint16_t value);
1145 bool max17048QuickStart();
1146 float max17048CellVoltage();
1147 float max17048StateOfCharge();
1148 bool ds3231ReadTime(uint8_t *second, uint8_t *minute, uint8_t *hour, uint8_t *day, uint8_t *month, uint16_t *year);
1149 bool ds3231SetTime(uint8_t second, uint8_t minute, uint8_t hour, uint8_t day, uint8_t month, uint16_t year);
1150 bool rtcDateLooksSet(uint8_t day, uint8_t month, uint16_t year);
1151 uint8_t daysInMonth(uint8_t month, uint16_t year);
1152 uint8_t bcdToDec(uint8_t value);
1153 uint8_t decToBcd(uint8_t value);
1154 String rtcStatusText();
1155 bool updateRtcStatusCache(bool force);
1156 void scanI2C(const char *label);
1157 void resetTouchController(uint8_t outputValue, uint8_t configValue);
1158 void initTouch();
1159 TouchPoint readTouch();
1160 bool gt911Read(uint16_t reg, uint8_t *data, uint8_t len);
1161 bool gt911Write(uint16_t reg, uint8_t value);
1162 String formatTime(uint32_t seconds);
1163 String matchStateLabel();
1164 int batteryPercent();
1165 bool chargingIndicatorOn();
1166 void changeMenuValue(int delta);
1167 void nextMenuItem();
1168 void previousMenuItem();
1169 uint8_t findCurrentTargetIndex();
1170
1171 // -----
1172 // Arduino entry points
1173 // -----
1174 void setup() {
1175     Serial.begin(115200);
1176     Serial0.begin(115200);
1177     delay(200);
1178     logLine("");
1179     logLine("Backgammon Match Clock booting");
1180     logPrintf("Firmware: %s\n", FIRMWARE_VERSION);
1181     logPrintf("PSRAM found: %s\n", psramFound() ? "yes" : "no");
1182     logPrintf("Free PSRAM: %u bytes\n", ESP.getFreePsram());
1183     logPrintf("Total PSRAM: %u bytes\n", ESP.getPsramSize());
1184     if (!psramFound() || ESP.getPsramSize() < 800000) {
1185         logLine("ERROR: PSRAM is not enabled. Set Tools > PSRAM to OPI PSRAM and upload again.");
1186     }
1187     logLine("Buttons are optional for startup; the menu should draw with no buttons connected.");
1188     logPrintf("DIS07050H RGB panel: %ux%u, BL=%u, PCLK=%u HSYNC=%u VSYNC=%u DE=%u\n",
1189         PANEL_WIDTH, PANEL_HEIGHT, PIN_TFT_BACKLIGHT, PIN_LCD_PCLK,
1190         PIN_LCD_HSYNC, PIN_LCD_VSYNC, PIN_LCD_DE);
1191
1192     btnPlayerA.begin(PIN_PLAYER_A_BUTTON, PLAYER_BUTTON_DEBOUNCE_MS);
1193     btnPlayerB.begin(PIN_PLAYER_B_BUTTON, PLAYER_BUTTON_DEBOUNCE_MS);
1194     btnScoreAPlus.begin(PIN_SCORE_A_PLUS, SMALL_BUTTON_DEBOUNCE_MS);

```

```

1195 btnScoreAMinus.begin(PIN_SCORE_A_MINUS, SMALL_BUTTON_DEBOUNCE_MS);
1196 btnScoreBPlus.begin(PIN_SCORE_B_PLUS, SMALL_BUTTON_DEBOUNCE_MS);
1197 btnScoreBMinus.begin(PIN_SCORE_B_MINUS, SMALL_BUTTON_DEBOUNCE_MS);
1198 btnSelect.begin(PIN_MENU_SELECT, SMALL_BUTTON_DEBOUNCE_MS);
1199 btnUp.begin(PIN_MENU_UP, SMALL_BUTTON_DEBOUNCE_MS);
1200 btnDown.begin(PIN_MENU_DOWN, SMALL_BUTTON_DEBOUNCE_MS);
1201 expanderBtnPlayerA.begin(PCF8575_PLAYER_A_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
1202 expanderBtnPlayerB.begin(PCF8575_PLAYER_B_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
1203 expanderBtnPlayPause.begin(PCF8575_PLAY_PAUSE_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
1204
1205 if (PIN_BUZZER != 255) {
1206     pinMode(PIN_BUZZER, OUTPUT);
1207     digitalWrite(PIN_BUZZER, LOW);
1208 }
1209 if (PIN_CHARGE_DETECT != 255) {
1210     pinMode(PIN_CHARGE_DETECT, INPUT_PULLUP);
1211 }
1212
1213 pinMode(PIN_TFT_BACKLIGHT, OUTPUT);
1214 setBrightness(255);
1215
1216 analogReadResolution(12);
1217
1218 initCrowPanelExpander();
1219 initButtonExpander();
1220 initFuelGauge();
1221 initRtcClock();
1222 initTouch();
1223 tft.begin();
1224 tft.setRotation(1);
1225 drawStartupTest();
1226
1227 loadSettings();
1228 setBrightness(brightness);
1229 resetMatch();
1230 mode = MODE_MATCH;
1231 markDisplayDirty();
1232
1233 lastTickMs = millis();
1234 lastActivityMs = lastTickMs;
1235 updateRtcStatusCache(true);
1236 drawDisplay();
1237 displayDirty = false;
1238 lastDisplayMs = lastTickMs;
1239 }
1240
1241 void loop() {
1242     handleTouch();
1243     handleButtons();
1244
1245     if (mode == MODE_MATCH) {
1246         updateClock();
1247     }
1248
1249     saveSettingsIfNeeded();
1250     maybeSleep();
1251
1252     uint32_t nowMs = millis();
1253     if (!displayDirty && nowMs - lastRtcStatusMs >= 1000UL) {
1254         bool rtcChanged = updateRtcStatusCache(false);
1255         if (rtcChanged && (mode == MODE_MATCH || mode == MODE_MATCH_OVER || mode == MODE_TIMEOUT)) {
1256             drawStatusBar();
1257         }
1258     }
1259
1260     bool timeDisplayDue = (clock_running && nowMs - lastDisplayMs >= DISPLAY_REFRESH_MS);
1261     if (displayDirty) {
1262         drawDisplay();
1263         lastDisplayMs = nowMs;
1264         displayDirty = false;
1265     } else if (timeDisplayDue) {
1266         drawTimerPanels();
1267         lastDisplayMs = nowMs;
1268     }
1269 }
1270
1271 // -----
1272 // Button handling
1273 // -----
1274 void handleButtons() {
1275     uint32_t nowMs = millis();
1276
1277     btnPlayerA.update(nowMs);
1278     btnPlayerB.update(nowMs);
1279     btnScoreAPlus.update(nowMs);
1280     btnScoreAMinus.update(nowMs);

```

```

1281 btnScoreBPlus.update(nowMs);
1282 btnScoreBMinus.update(nowMs);
1283 btnSelect.update(nowMs);
1284 btnUp.update(nowMs);
1285 btnDown.update(nowMs);
1286
1287 uint16_t expanderState = (pcf8575Address == 0) ? 0xFFFF : pcf8575Read(pcf8575Address);
1288 expanderBtnPlayerA.update(nowMs, expanderState);
1289 expanderBtnPlayerB.update(nowMs, expanderState);
1290 expanderBtnPlayPause.update(nowMs, expanderState);
1291
1292 bool gpioPlayerAPressed = btnPlayerA.wasPressed();
1293 bool gpioPlayerBPressed = btnPlayerB.wasPressed();
1294 bool expanderPlayerAPressed = expanderBtnPlayerA.wasPressed();
1295 bool expanderPlayerBPressed = expanderBtnPlayerB.wasPressed();
1296 bool expanderPlayPausePressed = expanderBtnPlayPause.wasPressed();
1297
1298 if (expanderPlayPausePressed) {
1299     logPrintf("PCF8575 P00 Play/Pause pressed: mode=%u running=%s game=%s\n",
1300             (unsigned int)mode,
1301             clock_running ? "yes" : "no",
1302             game_in_progress ? "yes" : "no");
1303 }
1304
1305 bool anyPress = gpioPlayerAPressed || gpioPlayerBPressed ||
1306                 expanderPlayerAPressed || expanderPlayerBPressed ||
1307                 expanderPlayPausePressed ||
1308                 btnScoreAPlus.wasPressed() || btnScoreAMinus.wasPressed() ||
1309                 btnScoreBPlus.wasPressed() || btnScoreBMinus.wasPressed() ||
1310                 btnSelect.wasPressed() || btnUp.wasPressed() || btnDown.wasPressed();
1311
1312 if (anyPress) {
1313     wakeActivity();
1314     playButtonBeep();
1315 }
1316
1317 if (mode == MODE_MENU) {
1318     handleMenu();
1319     return;
1320 }
1321
1322 if (mode == MODE_NAME_EDIT) {
1323     if (btnSelect.wasPressed()) {
1324         mode = MODE_MATCH;
1325         editingNameFor = PLAYER_NONE;
1326         markDisplayDirty();
1327     }
1328     return;
1329 }
1330
1331 if (mode == MODE_LOG) {
1332     if (btnSelect.wasPressed()) {
1333         mode = MODE_MATCH;
1334         markDisplayDirty();
1335     }
1336     return;
1337 }
1338
1339 if (mode == MODE_APPEARANCE) {
1340     if (btnSelect.wasPressed()) {
1341         mode = MODE_MENU;
1342         markDisplayDirty();
1343     }
1344     return;
1345 }
1346
1347 if (mode == MODE_CONFIRM_END_MATCH) {
1348     if (btnSelect.wasPressed()) {
1349         mode = MODE_MATCH;
1350         markDisplayDirty();
1351     }
1352     return;
1353 }
1354
1355 // The menu/select button pauses or resumes menu access during a match.
1356 if (btnSelect.wasPressed()) {
1357     enterMenu();
1358     return;
1359 }
1360
1361 if (mode == MODE_MATCH) {
1362     if (expanderPlayPausePressed) {
1363         togglePlayPause();
1364         return;
1365     }
1366 }

```

```

1367     if (gpioPlayerAPressed) {
1368         if (clock_running && active_player == PLAYER_A) {
1369             switchClockTo(PLAYER_B);
1370         } else {
1371             switchClockTo(PLAYER_A);
1372         }
1373     }
1374     if (gpioPlayerBPressed) {
1375         switchClockTo(PLAYER_A);
1376     }
1377     if (expanderPlayerAPressed) {
1378         switchClockTo(PLAYER_B);
1379     }
1380     if (expanderPlayerBPressed) {
1381         switchClockTo(PLAYER_A);
1382     }
1383
1384     if (btnScoreAPlus.wasPressed()) {
1385         adjustScore(PLAYER_A, +1);
1386     }
1387     if (btnScoreAMinus.wasPressed()) {
1388         adjustScore(PLAYER_A, -1);
1389     }
1390     if (btnScoreBPlus.wasPressed()) {
1391         adjustScore(PLAYER_B, +1);
1392     }
1393     if (btnScoreBMinus.wasPressed()) {
1394         adjustScore(PLAYER_B, -1);
1395     }
1396 }
1397
1398 // After timeout or match over, select returns to menu so the device can reset.
1399 if ((mode == MODE_TIMEOUT || mode == MODE_MATCH_OVER) && btnSelect.wasPressed()) {
1400     enterMenu();
1401 }
1402 }
1403
1404 void handleTouch() {
1405     const uint32_t TOUCH_RELEASE_GRACE_MS = 140;
1406     const uint32_t TOUCH_REPEAT_GUARD_MS = 220;
1407     const int16_t TOUCH_SWIPE_THRESHOLD = 32;
1408
1409     TouchPoint point = readTouch();
1410     uint32_t nowMs = millis();
1411
1412     if (point.touched) {
1413         touchLastX = point.x;
1414         touchLastY = point.y;
1415         if (point.y < touchMinY) {
1416             touchMinY = point.y;
1417         }
1418         if (point.y > touchMaxY) {
1419             touchMaxY = point.y;
1420         }
1421         touchLastSeenMs = nowMs;
1422         lastTouchSeen = true;
1423         if (!touchWasDown) {
1424             touchWasDown = true;
1425             touchStartX = point.x;
1426             touchStartY = point.y;
1427             touchMinY = point.y;
1428             touchMaxY = point.y;
1429             touchStartMs = nowMs;
1430         }
1431         return;
1432     }
1433
1434     if (!touchWasDown) {
1435         return;
1436     }
1437
1438     // The GT911 can briefly report no touch while a finger is still moving.
1439     // Waiting a moment here keeps swipes from being split into tiny taps.
1440     if (nowMs - touchLastSeenMs < TOUCH_RELEASE_GRACE_MS) {
1441         return;
1442     }
1443
1444     touchWasDown = false;
1445     wakeActivity();
1446
1447     if (nowMs - touchLastHandledMs < TOUCH_REPEAT_GUARD_MS) {
1448         return;
1449     }
1450     touchLastHandledMs = nowMs;
1451
1452     if (mode == MODE_MENU || mode == MODE_LOG || mode == MODE_NAME_EDIT ||

```

```

1453     mode == MODE_CLOCK_SET || mode == MODE_APPEARANCE) {
1454         handleTouchTap(touchLastX, touchLastY);
1455         return;
1456     }
1457
1458     int16_t dx = (int16_t)touchLastX - (int16_t)touchStartX;
1459     int16_t upDistance = (int16_t)touchStartY - (int16_t)touchMinY;
1460     int16_t downDistance = (int16_t)touchMaxY - (int16_t)touchStartY;
1461     int16_t dy = (downDistance > upDistance) ? downDistance : -upDistance;
1462     if (abs(dy) >= TOUCH_SWIPE_THRESHOLD && abs(dy) > abs(dx)) {
1463         uint16_t gestureEndY = touchStartY;
1464         if (dy > 0) {
1465             uint32_t candidate = (uint32_t)touchStartY + (uint32_t)abs(dy);
1466             gestureEndY = (candidate >= PANEL_HEIGHT) ? PANEL_HEIGHT - 1 : (uint16_t)candidate;
1467         } else {
1468             gestureEndY = (touchStartY > abs(dy)) ? touchStartY - abs(dy) : 0;
1469         }
1470         handleTouchSwipe(touchStartX, touchStartY, touchLastX, gestureEndY);
1471     } else {
1472         handleTouchTap(touchLastX, touchLastY);
1473     }
1474 }
1475
1476 void handleTouchTap(uint16_t x, uint16_t y) {
1477     if (mode == MODE_NAME_EDIT) {
1478         handleNameEditorTap(x, y);
1479         return;
1480     }
1481
1482     if (mode == MODE_CLOCK_SET) {
1483         handleClockSettingsTap(x, y);
1484         return;
1485     }
1486
1487     if (mode == MODE_APPEARANCE) {
1488         handleAppearanceSettingsTap(x, y);
1489         return;
1490     }
1491
1492     if (mode == MODE_CONFIRM_END_MATCH) {
1493         handleEndMatchConfirmTap(x, y);
1494         return;
1495     }
1496
1497     if (mode == MODE_LOG) {
1498         if (LOG_BACK.containsPadded(x, y, 18, 14)) {
1499             mode = MODE_MATCH;
1500             markDisplayDirty();
1501             return;
1502         }
1503         if (LOG_CLEAR.containsPadded(x, y, 18, 14)) {
1504             clearMatchLog();
1505             markDisplayDirty();
1506             return;
1507         }
1508         return;
1509     }
1510
1511     if (mode == MODE_MENU) {
1512         if (SETTINGS_BACK.containsPadded(x, y, 18, 14)) {
1513             mode = MODE_MATCH;
1514             markDisplayDirty();
1515             return;
1516         }
1517
1518         if (SETTINGS_MATCH_MINUS.containsPadded(x, y, 40, 18) ||
1519             SETTINGS_MATCH_ROW_MINUS.contains(x, y)) {
1520             currentMenuItem = MENU_MATCH_TARGET;
1521             changeMenuValue(-1);
1522             return;
1523         }
1524         if (SETTINGS_MATCH_PLUS.containsPadded(x, y, 40, 18) ||
1525             SETTINGS_MATCH_ROW_PLUS.contains(x, y)) {
1526             currentMenuItem = MENU_MATCH_TARGET;
1527             changeMenuValue(+1);
1528             return;
1529         }
1530         if (SETTINGS_TIME_MINUS.containsPadded(x, y, 40, 18) ||
1531             SETTINGS_TIME_ROW_MINUS.contains(x, y)) {
1532             currentMenuItem = MENU_INITIAL_TIME;
1533             changeMenuValue(-1);
1534             return;
1535         }
1536         if (SETTINGS_TIME_PLUS.containsPadded(x, y, 40, 18) ||
1537             SETTINGS_TIME_ROW_PLUS.contains(x, y)) {
1538             currentMenuItem = MENU_INITIAL_TIME;

```

```

1539     changeMenuValue(+1);
1540     return;
1541 }
1542 if (SETTINGS_DELAY_MINUS.containsPadded(x, y, 40, 18) ||
1543     SETTINGS_DELAY_ROW_MINUS.contains(x, y)) {
1544     currentMenuItem = MENU_DELAY;
1545     changeMenuValue(-1);
1546     return;
1547 }
1548 if (SETTINGS_DELAY_PLUS.containsPadded(x, y, 40, 18) ||
1549     SETTINGS_DELAY_ROW_PLUS.contains(x, y)) {
1550     currentMenuItem = MENU_DELAY;
1551     changeMenuValue(+1);
1552     return;
1553 }
1554 if (SETTINGS_SCOPE_GAME.containsPadded(x, y, 18, 10)) {
1555     timerScope = TIMER_BY_GAME;
1556     playerA_time_remaining = startingClockMs();
1557     playerB_time_remaining = startingClockMs();
1558     markSettingsDirty();
1559     markDisplayDirty();
1560     return;
1561 }
1562 if (SETTINGS_SCOPE_MATCH.containsPadded(x, y, 18, 10)) {
1563     timerScope = TIMER_BY_MATCH;
1564     playerA_time_remaining = startingClockMs();
1565     playerB_time_remaining = startingClockMs();
1566     markSettingsDirty();
1567     markDisplayDirty();
1568     return;
1569 }
1570 if (SETTINGS_MODE_SIMPLE.containsPadded(x, y, 12, 10)) {
1571     timingMode = SIMPLE_DELAY;
1572     markSettingsDirty();
1573     markDisplayDirty();
1574     return;
1575 }
1576 if (SETTINGS_MODE_BRONSTEIN.containsPadded(x, y, 12, 10)) {
1577     timingMode = BRONSTEIN_DELAY;
1578     markSettingsDirty();
1579     markDisplayDirty();
1580     return;
1581 }
1582 if (SETTINGS_MODE_FISCHER.containsPadded(x, y, 12, 10)) {
1583     timingMode = FISCHER_INCREMENT;
1584     markSettingsDirty();
1585     markDisplayDirty();
1586     return;
1587 }
1588 if (SETTINGS_APPEARANCE.containsPadded(x, y, 10, 10)) {
1589     mode = MODE_APPEARANCE;
1590     markDisplayDirty();
1591     return;
1592 }
1593 if (SETTINGS_CLOCK.containsPadded(x, y, 18, 14)) {
1594     openClockSettings();
1595     return;
1596 }
1597 if (SETTINGS_START.containsPadded(x, y, 18, 14)) {
1598     resetMatch();
1599     startNewGame();
1600     mode = MODE_MATCH;
1601     markDisplayDirty();
1602     return;
1603 }
1604 if (SETTINGS_END.containsPadded(x, y, 18, 14)) {
1605     resetMatch();
1606     mode = MODE_MATCH;
1607     markDisplayDirty();
1608     return;
1609 }
1610 return;
1611 }
1612
1613 bool timerAHit = REGION_TIMER_A.containsPadded(x, y, 12, 14) ||
1614     REGION_HEADER_A.containsPadded(x, y, 12, 10);
1615 bool timerBHit = REGION_TIMER_B.containsPadded(x, y, 12, 14) ||
1616     REGION_HEADER_B.containsPadded(x, y, 12, 10);
1617
1618 if (!clock_running && REGION_HEADER_A.containsPadded(x, y, 12, 10)) {
1619     openNameEditor(PPLAYER_A);
1620     return;
1621 }
1622
1623 if (!clock_running && REGION_HEADER_B.containsPadded(x, y, 12, 10)) {
1624     openNameEditor(PPLAYER_B);

```

```

1625     return;
1626 }
1627
1628 if (mode == MODE_MATCH && REGION_LOG.contains(x, y)) {
1629     mode = MODE_LOG;
1630     markDisplayDirty();
1631     return;
1632 }
1633
1634 if (mode == MODE_MATCH && (REGION_SETTINGS.contains(x, y) || REGION_MATCH_TARGET.contains(x, y))) {
1635     enterMenu();
1636     return;
1637 }
1638
1639 if (HIT_PLAY_PAUSE.contains(x, y)) {
1640     togglePlayPause();
1641     return;
1642 }
1643
1644 if (HIT_END_GAME.contains(x, y)) {
1645     if (mode == MODE_TIMEOUT) {
1646         mode = MODE_MATCH;
1647         startNewGame();
1648         return;
1649     }
1650     if (game_in_progress || clock_running) {
1651         endCurrentGame();
1652     }
1653     return;
1654 }
1655
1656 if (HIT_END_MATCH.contains(x, y)) {
1657     if (mode == MODE_MATCH_OVER) {
1658         resetMatch();
1659         startNewGame();
1660     } else {
1661         pauseClock();
1662         mode = MODE_CONFIRM_END_MATCH;
1663         markDisplayDirty();
1664     }
1665     return;
1666 }
1667
1668 if (timerAHit) {
1669     switchClockTo(PPLAYER_B);
1670     return;
1671 }
1672
1673 if (timerBHit) {
1674     switchClockTo(PPLAYER_A);
1675     return;
1676 }
1677
1678 if (!clock_running && REGION_SCORE_A.contains(x, y)) {
1679     int delta = (y < REGION_SCORE_A.y + (REGION_SCORE_A.h / 2)) ? +1 : -1;
1680     adjustScore(PPLAYER_A, delta);
1681     return;
1682 }
1683
1684 if (!clock_running && REGION_SCORE_B.contains(x, y)) {
1685     int delta = (y < REGION_SCORE_B.y + (REGION_SCORE_B.h / 2)) ? +1 : -1;
1686     adjustScore(PPLAYER_B, delta);
1687     return;
1688 }
1689 }
1690
1691 void handleTouchSwipe(uint16_t startX, uint16_t startY, uint16_t endX, uint16_t endY) {
1692     if ((mode != MODE_MATCH && mode != MODE_TIMEOUT) || clock_running) {
1693         return;
1694     }
1695
1696     int delta = (endY < startY) ? +1 : -1;
1697     bool scoreA = REGION_SCORE_A.contains(startX, startY);
1698     bool scoreB = REGION_SCORE_B.contains(startX, startY);
1699
1700     if (!scoreA && !scoreB) {
1701         return;
1702     }
1703
1704     adjustScore(scoreA ? PPLAYER_A : PPLAYER_B, delta);
1705     markDisplayDirty();
1706 }
1707
1708 void pauseClock() {
1709     if (!clock_running || active_player == PPLAYER_NONE) {
1710         return;

```

```

1711 }
1712
1713 uint32_t nowMs = millis();
1714 pausedTurnElapsedMs = nowMs - turnStartMs;
1715 clock_running = false;
1716 lastTickMs = nowMs;
1717 markDisplayDirty();
1718 }
1719
1720 void resumeClock() {
1721     if (clock_running || active_player == PLAYER_NONE || matchWinner != PLAYER_NONE || timeoutPlayer != PLAYER_NONE) {
1722         return;
1723     }
1724
1725     uint32_t nowMs = millis();
1726     turnStartMs = nowMs - pausedTurnElapsedMs;
1727     lastTickMs = nowMs;
1728     clock_running = true;
1729     markDisplayDirty();
1730 }
1731
1732 void togglePlayPause() {
1733     if (mode != MODE_MATCH) {
1734         return;
1735     }
1736
1737     if (clock_running) {
1738         pauseClock();
1739     } else if (!game_in_progress) {
1740         startNewGame();
1741     } else {
1742         resumeClock();
1743     }
1744 }
1745
1746 void switchClockTo(Player nextPlayer) {
1747     uint32_t nowMs = millis();
1748
1749     if (mode != MODE_MATCH || matchWinner != PLAYER_NONE || timeoutPlayer != PLAYER_NONE) {
1750         return;
1751     }
1752
1753     if (!game_in_progress) {
1754         startNewGame();
1755     }
1756
1757     // Once the clock is running, only the active player's own large button
1758     // should end the turn. Presses on the inactive side are ignored so an
1759     // accidental bump cannot reset the current player's delay window.
1760     if (clock_running && active_player == nextPlayer) {
1761         return;
1762     }
1763
1764     if (clock_running && active_player != PLAYER_NONE && active_player != nextPlayer) {
1765         if (!applyTurnElapsed(active_player, nowMs)) {
1766             return;
1767         }
1768     }
1769
1770     active_player = nextPlayer;
1771     clock_running = true;
1772     turnStartMs = nowMs;
1773     pausedTurnElapsedMs = 0;
1774     lastTickMs = nowMs;
1775     markDisplayDirty();
1776 }
1777
1778 // -----
1779 // Clock behavior
1780 // -----
1781 void updateClock() {
1782     if (!clock_running || active_player == PLAYER_NONE) {
1783         lastTickMs = millis();
1784         return;
1785     }
1786
1787     uint32_t nowMs = millis();
1788     uint32_t elapsedMs = nowMs - lastTickMs;
1789     if (elapsedMs < 1000) {
1790         return;
1791     }
1792
1793     uint32_t wholeSeconds = elapsedMs / 1000;
1794     lastTickMs += wholeSeconds * 1000;
1795     match_elapsed_seconds += wholeSeconds;
1796

```

```

1797 if (visibleReserveMs(active_player, nowMs) == 0) {
1798     uint32_t *remaining = reserveFor(active_player);
1799     if (remaining != nullptr) {
1800         *remaining = 0;
1801     }
1802     clock_running = false;
1803     timeoutPlayer = active_player;
1804     active_player = PLAYER_NONE;
1805     if (crawford_active) {
1806         markCrawfordUsed();
1807     }
1808     game_in_progress = false;
1809     mode = MODE_TIMEOUT;
1810     playTimeoutBeep();
1811     markDisplayDirty();
1812     return;
1813 }
1814
1815 // Low-time warning. The critical zone beeps more often.
1816 uint32_t visibleSeconds = visibleReserveMs(active_player, nowMs) / 1000UL;
1817 if (visibleSeconds <= LOW_TIME_WARNING_SECONDS) {
1818     uint32_t interval = (visibleSeconds <= CRITICAL_TIME_WARNING_SECONDS) ? 1000UL : 5000UL;
1819     if (nowMs - lastLowTimeBeepMs >= interval) {
1820         beep(2200, 45);
1821         lastLowTimeBeepMs = nowMs;
1822     }
1823 }
1824 }
1825
1826 void addIncrementTo(Player player) {
1827     if (increment_seconds == 0) {
1828         return;
1829     }
1830
1831     if (player == PLAYER_A) {
1832         playerA_time_remaining += (uint32_t)increment_seconds * 1000UL;
1833     } else if (player == PLAYER_B) {
1834         playerB_time_remaining += (uint32_t)increment_seconds * 1000UL;
1835     }
1836     markDisplayDirty();
1837 }
1838
1839 bool applyTurnElapsed(Player player, uint32_t nowMs) {
1840     uint32_t *reserve = reserveFor(player);
1841     if (reserve == nullptr) {
1842         return true;
1843     }
1844
1845     uint32_t elapsedMs = nowMs - turnStartMs;
1846     uint32_t delayOrIncrementMs = (uint32_t)delay_seconds * 1000UL;
1847     uint32_t deductionMs = 0;
1848
1849     if (timingMode == FISCHER_INCREMENT) {
1850         deductionMs = elapsedMs;
1851     } else {
1852         deductionMs = (elapsedMs > delayOrIncrementMs) ? (elapsedMs - delayOrIncrementMs) : 0;
1853     }
1854
1855     if (deductionMs >= *reserve) {
1856         *reserve = 0;
1857         clock_running = false;
1858         timeoutPlayer = player;
1859         active_player = PLAYER_NONE;
1860         if (crawford_active) {
1861             markCrawfordUsed();
1862         }
1863         game_in_progress = false;
1864         mode = MODE_TIMEOUT;
1865         playTimeoutBeep();
1866         markDisplayDirty();
1867         return false;
1868     }
1869
1870     *reserve -= deductionMs;
1871     if (timingMode == FISCHER_INCREMENT) {
1872         *reserve += delayOrIncrementMs;
1873     }
1874     return true;
1875 }
1876
1877 uint32_t visibleReserveMs(Player player, uint32_t nowMs) {
1878     uint32_t *reserve = reserveFor(player);
1879     if (reserve == nullptr) {
1880         return 0;
1881     }
1882     if (active_player != player) {

```

```

1883     return *reserve;
1884 }
1885
1886 uint32_t elapsedMs = 0;
1887 if (clock_running) {
1888     elapsedMs = nowMs - turnStartMs;
1889 } else if (pausedTurnElapsedMs > 0) {
1890     elapsedMs = pausedTurnElapsedMs;
1891 } else {
1892     return *reserve;
1893 }
1894
1895 uint32_t delayOrIncrementMs = (uint32_t)delay_seconds * 1000UL;
1896 uint32_t deductionMs = 0;
1897
1898 if (timingMode == SIMPLE_DELAY) {
1899     deductionMs = (elapsedMs > delayOrIncrementMs) ? (elapsedMs - delayOrIncrementMs) : 0;
1900 } else {
1901     deductionMs = elapsedMs;
1902 }
1903
1904 return (deductionMs >= *reserve) ? 0 : (*reserve - deductionMs);
1905 }
1906
1907 uint32_t *reserveFor(Player player) {
1908     if (player == PLAYER_A) {
1909         return &playerA_time_remaining;
1910     }
1911     if (player == PLAYER_B) {
1912         return &playerB_time_remaining;
1913     }
1914     return nullptr;
1915 }
1916
1917 uint32_t matchClockSeconds() {
1918     return initial_time_seconds * (uint32_t)match_target;
1919 }
1920
1921 uint32_t startingClockMs() {
1922     uint32_t seconds = (timerScope == TIMER_BY_MATCH) ? matchClockSeconds() : initial_time_seconds;
1923     return seconds * 1000UL;
1924 }
1925
1926 // -----
1927 // Score and match state
1928 // -----
1929 void adjustScore(Player player, int delta) {
1930     if ((mode != MODE_MATCH && mode != MODE_TIMEOUT) || clock_running) {
1931         return;
1932     }
1933
1934     bool scorerIsOpponentOfCrawfordPlayer =
1935         (match_target > 1) &&
1936         ((player == PLAYER_A && playerB_score == match_target - 1) ||
1937          (player == PLAYER_B && playerA_score == match_target - 1));
1938
1939     if (delta > 0 && (crawford_active || scorerIsOpponentOfCrawfordPlayer)) {
1940         // Any score recorded during the Crawford game ends Crawford immediately.
1941         // Also catch the real scoreboard pattern: the one-away player did not
1942         // score, so the opponent's score changed and the next game is post-Crawford.
1943         markCrawfordUsed();
1944     }
1945
1946     if (delta > 0 && game_in_progress) {
1947         // A positive score adjustment is treated as the end of the current game.
1948         // Manual corrections with minus buttons do not advance Crawford state.
1949         endCurrentGame();
1950     }
1951
1952     int *score = (player == PLAYER_A) ? &playerA_score : &playerB_score;
1953     *score += delta;
1954     if (*score < 0) {
1955         *score = 0;
1956     }
1957     if (*score > match_target) {
1958         *score = match_target;
1959     }
1960
1961     checkMatchState();
1962     markDisplayDirty();
1963
1964     if (matchWinner != PLAYER_NONE || mode == MODE_MATCH_OVER) {
1965         return;
1966     }
1967
1968     // New games are counted only when startNewGame() is explicitly called.

```

```

1969 // Scoring corrections should not advance the game counter.
1970 }
1971
1972 void checkMatchState() {
1973     if (playerA_score >= match_target || playerB_score >= match_target) {
1974         matchWinner = (playerA_score >= match_target) ? PLAYER_A : PLAYER_B;
1975         clock_running = false;
1976         active_player = PLAYER_NONE;
1977         game_in_progress = false;
1978         recordMatchLog();
1979         mode = MODE_MATCH_OVER;
1980         playMatchOverBeep();
1981         markDisplayDirty();
1982         return;
1983     }
1984
1985     if (crawford_used) {
1986         pending_crawford_next_game = false;
1987         if (!crawford_active) {
1988             post_crawford = true;
1989             cube_enabled = true;
1990         }
1991         return;
1992     }
1993
1994     // Crawford is only meaningful for match targets above 1.
1995     if (match_target > 1 && !crawford_used && !crawford_active) {
1996         bool someoneAtCrawfordScore =
1997             (playerA_score == match_target - 1) || (playerB_score == match_target - 1);
1998
1999         pending_crawford_next_game = someoneAtCrawfordScore;
2000     }
2001 }
2002
2003 void startNewGame() {
2004     if (mode == MODE_MATCH_OVER) {
2005         return;
2006     }
2007
2008     game_number++;
2009     game_in_progress = true;
2010     clock_running = false;
2011     active_player = PLAYER_NONE;
2012     timeoutPlayer = PLAYER_NONE;
2013     if (timerScope == TIMER_BY_GAME) {
2014         playerA_time_remaining = startingClockMs();
2015         playerB_time_remaining = startingClockMs();
2016     }
2017     pausedTurnElapsedMs = 0;
2018     lastLowTimeBeepMs = 0;
2019
2020     if (pending_crawford_next_game && !crawford_used) {
2021         crawford_active = true;
2022         post_crawford = false;
2023         cube_enabled = false;
2024         pending_crawford_next_game = false;
2025     } else if (crawford_used) {
2026         crawford_active = false;
2027         post_crawford = true;
2028         cube_enabled = true;
2029     } else {
2030         crawford_active = false;
2031         post_crawford = false;
2032         cube_enabled = true;
2033     }
2034
2035     lastTickMs = millis();
2036     turnStartMs = lastTickMs;
2037     markDisplayDirty();
2038 }
2039
2040 void endCurrentGame() {
2041     clock_running = false;
2042     active_player = PLAYER_NONE;
2043     pausedTurnElapsedMs = 0;
2044
2045     if (crawford_active) {
2046         markCrawfordUsed();
2047     }
2048
2049     game_in_progress = false;
2050     markDisplayDirty();
2051 }
2052
2053 void markCrawfordUsed() {
2054     crawford_active = false;

```

```

2055     Crawford_used = true;
2056     post_Crawford = true;
2057     pending_Crawford_next_game = false;
2058     cube_enabled = true;
2059 }
2060
2061 void forceEndMatch() {
2062     clock_running = false;
2063     active_player = PLAYER_NONE;
2064     pausedTurnElapsedMs = 0;
2065     game_in_progress = false;
2066
2067     matchWinner = (playerA_score == playerB_score) ? PLAYER_NONE :
2068                   ((playerA_score > playerB_score) ? PLAYER_A : PLAYER_B);
2069     recordMatchLog();
2070     mode = MODE_MATCH_OVER;
2071     playMatchOverBeep();
2072     markDisplayDirty();
2073 }
2074
2075 void resetMatch() {
2076     playerA_score = 0;
2077     playerB_score = 0;
2078     playerA_time_remaining = startingClockMs();
2079     playerB_time_remaining = startingClockMs();
2080     match_elapsed_seconds = 0;
2081     active_player = PLAYER_NONE;
2082     clock_running = false;
2083     pausedTurnElapsedMs = 0;
2084     Crawford_used = false;
2085     Crawford_active = false;
2086     post_Crawford = false;
2087     cube_enabled = true;
2088     game_number = 0;
2089     timeoutPlayer = PLAYER_NONE;
2090     matchWinner = PLAYER_NONE;
2091     pending_Crawford_next_game = false;
2092     game_in_progress = false;
2093     mode = MODE_MATCH;
2094     markDisplayDirty();
2095 }
2096
2097 // -----
2098 // Menu behavior
2099 // -----
2100 void enterMenu() {
2101     clock_running = false;
2102     active_player = PLAYER_NONE;
2103     mode = MODE_MENU;
2104     markDisplayDirty();
2105 }
2106
2107 void handleMenu() {
2108     if (btnPlayerA.wasPressed() || btnUp.wasPressed()) {
2109         changeMenuValue(+1);
2110     }
2111
2112     if (btnPlayerB.wasPressed() || btnDown.wasPressed()) {
2113         changeMenuValue(-1);
2114     }
2115
2116     if (btnScoreAPlus.wasPressed() || btnScoreBPlus.wasPressed()) {
2117         nextMenuItem();
2118     }
2119
2120     if (btnScoreAMinus.wasPressed() || btnScoreBMinus.wasPressed()) {
2121         previousMenuItem();
2122     }
2123
2124     if (btnSelect.wasPressed()) {
2125         if (currentMenuItem == MENU_START_MATCH) {
2126             resetMatch();
2127             startNewGame();
2128         } else if (currentMenuItem == MENU_RESET_MATCH) {
2129             resetMatch();
2130             enterMenu();
2131         } else {
2132             nextMenuItem();
2133         }
2134     }
2135 }
2136
2137 void changeMenuValue(int delta) {
2138     switch (currentMenuItem) {
2139         case MENU_MATCH_TARGET: {
2140             int index = findCurrentTargetIndex();

```

```

2141     index += delta;
2142     if (index < 0) {
2143         index = MATCH_TARGET_COUNT - 1;
2144     }
2145     if (index >= MATCH_TARGET_COUNT) {
2146         index = 0;
2147     }
2148     match_target = MATCH_TARGETS[index];
2149     playerA_time_remaining = startingClockMs();
2150     playerB_time_remaining = startingClockMs();
2151     markSettingsDirty();
2152     break;
2153 }
2154
2155 case MENU_INITIAL_TIME: {
2156     int32_t next = (int32_t)initial_time_seconds + (delta * 30);
2157     if (next < 60) {
2158         next = 60;
2159     }
2160     if (next > 99L * 60L) {
2161         next = 99L * 60L;
2162     }
2163     initial_time_seconds = (uint32_t)next;
2164     playerA_time_remaining = startingClockMs();
2165     playerB_time_remaining = startingClockMs();
2166     markSettingsDirty();
2167     break;
2168 }
2169
2170 case MENU_DELAY: {
2171     int next = (int)delay_seconds + delta;
2172     if (next < 0) {
2173         next = 0;
2174     }
2175     if (next > 60) {
2176         next = 60;
2177     }
2178     delay_seconds = (uint8_t)next;
2179     markSettingsDirty();
2180     break;
2181 }
2182
2183 case MENU_INCREMENT: {
2184     int next = (int)increment_seconds + delta;
2185     if (next < 0) {
2186         next = 0;
2187     }
2188     if (next > 60) {
2189         next = 60;
2190     }
2191     increment_seconds = (uint8_t)next;
2192     markSettingsDirty();
2193     break;
2194 }
2195
2196 case MENU_BRIGHTNESS: {
2197     int next = (int)brightness + (delta * 15);
2198     if (next < 15) {
2199         next = 15;
2200     }
2201     if (next > 255) {
2202         next = 255;
2203     }
2204     brightness = (uint8_t)next;
2205     setBrightness(brightness);
2206     markSettingsDirty();
2207     break;
2208 }
2209
2210 default:
2211     break;
2212 }
2213 markDisplayDirty();
2214 }
2215
2216 void nextMenuItem() {
2217     currentMenuItem = (MenuItem)(((uint8_t)currentMenuItem + 1) % MENU_COUNT);
2218     markDisplayDirty();
2219 }
2220
2221 void previousMenuItem() {
2222     uint8_t item = (uint8_t)currentMenuItem;
2223     currentMenuItem = (MenuItem)((item == 0) ? (MENU_COUNT - 1) : (item - 1));
2224     markDisplayDirty();
2225 }
2226

```

```

2227 uint8_t findCurrentTargetIndex() {
2228     for (uint8_t i = 0; i < MATCH_TARGET_COUNT; i++) {
2229         if (MATCH_TARGETS[i] == match_target) {
2230             return i;
2231         }
2232     }
2233     return 4; // Default to match to 9.
2234 }
2235
2236 // -----
2237 // Display
2238 // -----
2239 void drawDisplay() {
2240     bool modeChanged = !displayModeKnown || mode != lastDrawMode;
2241     if (modeChanged) {
2242         tft.fillScreen(uiBackgroundColor());
2243     }
2244     displayModeKnown = true;
2245     lastDrawMode = mode;
2246
2247     if (mode == MODE_NAME_EDIT) {
2248         drawNameEditor();
2249         return;
2250     }
2251
2252     if (mode == MODE_LOG) {
2253         drawMatchLog();
2254         return;
2255     }
2256
2257     if (mode == MODE_CLOCK_SET) {
2258         drawClockSettings();
2259         return;
2260     }
2261
2262     if (mode == MODE_APPEARANCE) {
2263         drawAppearanceSettings();
2264         return;
2265     }
2266
2267     if (mode == MODE_MENU) {
2268         tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
2269         tft.setTextSize(3);
2270         tft.setCursor(248, 28);
2271         tft.print("SETTINGS");
2272
2273         tft.fillRect(SETTINGS_BACK.x, SETTINGS_BACK.y, SETTINGS_BACK.w, SETTINGS_BACK.h, TFT_PANEL);
2274         tft.drawRoundRect(SETTINGS_BACK.x, SETTINGS_BACK.y, SETTINGS_BACK.w, SETTINGS_BACK.h, 6, TFT_BORDER);
2275         tft.setTextSize(2);
2276         tft.setCursor(44, 38);
2277         tft.print("BACK");
2278
2279         tft.fillRect(52, 82, 696, 58, TFT_PANEL);
2280         tft.fillRect(52, 162, 696, 58, TFT_PANEL);
2281         tft.fillRect(52, 242, 696, 58, TFT_PANEL);
2282         tft.drawRoundRect(52, 82, 696, 58, 6, TFT_BORDER);
2283         tft.drawRoundRect(52, 162, 696, 58, 6, TFT_BORDER);
2284         tft.drawRoundRect(52, 242, 696, 58, 6, TFT_BORDER);
2285
2286         tft.setTextColor(TFT_TEXT, TFT_PANEL);
2287         tft.setTextSize(2);
2288         tft.setCursor(76, 102);
2289         tft.print("MATCH TARGET");
2290         tft.setCursor(536, 102);
2291         tft.printf("%d", match_target);
2292
2293         tft.setCursor(76, 182);
2294         tft.print("TIME / GAME");
2295         tft.setCursor(520, 182);
2296         tft.print(formatTime(initial_time_seconds));
2297         tft.setTextColor(TFT_MUTED, TFT_PANEL);
2298         tft.setTextSize(1);
2299         tft.setCursor(76, 207);
2300         tft.print("TOTAL ");
2301         tft.print(formatTime(matchClockSeconds()));
2302         tft.setTextColor(TFT_TEXT, TFT_PANEL);
2303         tft.setTextSize(2);
2304
2305         tft.setCursor(76, 262);
2306         tft.print("BONUS / DELAY");
2307         tft.setCursor(536, 262);
2308         tft.printf("%u", delay_seconds);
2309
2310         const Rect controls[] = {
2311             SETTINGS_MATCH_MINUS, SETTINGS_MATCH_PLUS,
2312             SETTINGS_TIME_MINUS, SETTINGS_TIME_PLUS,

```

```

2313     SETTINGS_DELAY_MINUS, SETTINGS_DELAY_PLUS
2314 };
2315 const char *symbols[] = {"-", "+", "-", "+", "-", "+"};
2316 for (uint8_t i = 0; i < 6; i++) {
2317     tft.fillRect(controls[i].x, controls[i].y, controls[i].w, controls[i].h, TFT_PANEL_ALT);
2318     tft.drawRoundRect(controls[i].x, controls[i].y, controls[i].w, controls[i].h, 6, TFT_BORDER);
2319     tft.setTextColor(TFT_TEXT, TFT_PANEL_ALT);
2320     tft.setTextSize(3);
2321     tft.setCursor(controls[i].x + 40, controls[i].y + 14);
2322     tft.print(symbols[i]);
2323 }
2324
2325 tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
2326 tft.setTextSize(2);
2327 tft.setCursor(76, 315);
2328 tft.print("TIMER");
2329 tft.fillRect(SETTINGS_SCOPE_GAME.x, SETTINGS_SCOPE_GAME.y, SETTINGS_SCOPE_GAME.w, SETTINGS_SCOPE_GAME.h,
2330     timerScope == TIMER_BY_GAME ? TFT_OK : TFT_PANEL_ALT);
2331 tft.drawRoundRect(SETTINGS_SCOPE_GAME.x, SETTINGS_SCOPE_GAME.y, SETTINGS_SCOPE_GAME.w, SETTINGS_SCOPE_GAME.h, 6, TFT_BORDER);
2332 drawCenteredInRect("GAME", SETTINGS_SCOPE_GAME, 2, timerScope == TIMER_BY_GAME ? ILI9341_BLACK : TFT_TEXT,
2333     timerScope == TIMER_BY_GAME ? TFT_OK : TFT_PANEL_ALT);
2334 tft.fillRect(SETTINGS_SCOPE_MATCH.x, SETTINGS_SCOPE_MATCH.y, SETTINGS_SCOPE_MATCH.w, SETTINGS_SCOPE_MATCH.h,
2335     timerScope == TIMER_BY_MATCH ? TFT_OK : TFT_PANEL_ALT);
2336 tft.drawRoundRect(SETTINGS_SCOPE_MATCH.x, SETTINGS_SCOPE_MATCH.y, SETTINGS_SCOPE_MATCH.w, SETTINGS_SCOPE_MATCH.h, 6, TFT_BORDER);
2337 drawCenteredInRect("MATCH", SETTINGS_SCOPE_MATCH, 2, timerScope == TIMER_BY_MATCH ? ILI9341_BLACK : TFT_TEXT,
2338     timerScope == TIMER_BY_MATCH ? TFT_OK : TFT_PANEL_ALT);
2339
2340 const Rect modeButtons[] = {SETTINGS_MODE_SIMPLE, SETTINGS_MODE_BRONSTEIN, SETTINGS_MODE_FISCHER};
2341 const char *modeLabels[] = {"SIMPLE", "BRONSTEIN", "FISCHER"};
2342 for (uint8_t i = 0; i < 3; i++) {
2343     bool selected = (uint8_t)timingMode == i;
2344     tft.fillRect(modeButtons[i].x, modeButtons[i].y, modeButtons[i].w, modeButtons[i].h,
2345         selected ? TFT_OK : TFT_PANEL_ALT);
2346     tft.drawRoundRect(modeButtons[i].x, modeButtons[i].y, modeButtons[i].w, modeButtons[i].h, 6, TFT_BORDER);
2347     drawCenteredInRect(modeLabels[i], modeButtons[i], 2, selected ? ILI9341_BLACK : TFT_TEXT,
2348         selected ? TFT_OK : TFT_PANEL_ALT);
2349 }
2350
2351 tft.fillRect(SETTINGS_APPEARANCE.x, SETTINGS_APPEARANCE.y, SETTINGS_APPEARANCE.w, SETTINGS_APPEARANCE.h, TFT_PANEL_ALT);
2352 tft.drawRoundRect(SETTINGS_APPEARANCE.x, SETTINGS_APPEARANCE.y, SETTINGS_APPEARANCE.w, SETTINGS_APPEARANCE.h, 6, TFT_BORDER);
2353 drawCenteredInRect("APPEARANCE", SETTINGS_APPEARANCE, 2, TFT_TEXT, TFT_PANEL_ALT);
2354
2355 tft.fillRect(SETTINGS_CLOCK.x, SETTINGS_CLOCK.y, SETTINGS_CLOCK.w, SETTINGS_CLOCK.h, TFT_PANEL_ALT);
2356 tft.drawRoundRect(SETTINGS_CLOCK.x, SETTINGS_CLOCK.y, SETTINGS_CLOCK.w, SETTINGS_CLOCK.h, 6, TFT_BORDER);
2357 drawCenteredInRect("CLOCK", SETTINGS_CLOCK, 2, TFT_TEXT, TFT_PANEL_ALT);
2358
2359 tft.fillRect(SETTINGS_START.x, SETTINGS_START.y, SETTINGS_START.w, SETTINGS_START.h, TFT_OK);
2360 tft.fillRect(SETTINGS_END.x, SETTINGS_END.y, SETTINGS_END.w, SETTINGS_END.h, TFT_DANGER);
2361 tft.drawRoundRect(SETTINGS_START.x, SETTINGS_START.y, SETTINGS_START.w, SETTINGS_START.h, 6, TFT_BORDER);
2362 tft.drawRoundRect(SETTINGS_END.x, SETTINGS_END.y, SETTINGS_END.w, SETTINGS_END.h, 6, TFT_BORDER);
2363 drawCenteredInRect("START MATCH", SETTINGS_START, 2, ILI9341_BLACK, TFT_OK);
2364 drawCenteredInRect("END MATCH", SETTINGS_END, 2, TFT_TEXT, TFT_DANGER);
2365
2366 return;
2367 }
2368
2369 drawStatusBar();
2370
2371 uint16_t background = uiBackgroundColor();
2372 uint16_t panel = uiPanelColor();
2373 uint16_t text = uiTextColor();
2374 bool showCrawford = mode != MODE_MATCH_OVER && (crawford_active || pending_crawford_next_game);
2375 bool showPostCrawford = mode != MODE_MATCH_OVER && !showCrawford && (post_crawford || crawford_used);
2376 uint16_t matchPanelBg = showCrawford ? TFT_CRAWFORD_BG :
2377     (showPostCrawford ? TFT_POST_CRAWFORD_BG : panel);
2378 uint16_t matchPanelText = showCrawford ? ILI9341_BLACK :
2379     (showPostCrawford ? ILI9341_BLACK : text);
2380
2381 uint16_t headerABg = playerA_color;
2382 uint16_t headerBBg = playerB_color;
2383 uint16_t headerAText = ILI9341_WHITE;
2384 uint16_t headerBText = ILI9341_WHITE;
2385
2386 tft.fillRect(0, REGION_STATUS.h, PANEL_WIDTH, PANEL_HEIGHT - REGION_STATUS.h, background);
2387
2388 tft.fillRect(REGION_HEADER_A.x, REGION_HEADER_A.y, REGION_HEADER_A.w, REGION_HEADER_A.h, headerABg);
2389 tft.fillRect(REGION_HEADER_B.x, REGION_HEADER_B.y, REGION_HEADER_B.w, REGION_HEADER_B.h, headerBBg);
2390 drawTimerPanel(PLAYER_A, REGION_TIMER_A);
2391 tft.fillRect(REGION_PLAY_PAUSE.x, REGION_PLAY_PAUSE.y, REGION_PLAY_PAUSE.w, REGION_PLAY_PAUSE.h, panel);
2392 tft.fillRect(REGION_END_GAME.x, REGION_END_GAME.y, REGION_END_GAME.w, REGION_END_GAME.h, panel);
2393 tft.fillRect(REGION_END_MATCH.x, REGION_END_MATCH.y, REGION_END_MATCH.w, REGION_END_MATCH.h,
2394     mode == MODE_MATCH_OVER ? TFT_DANGER : panel);
2395 tft.fillRect(REGION_SCORE_A.x, REGION_SCORE_A.y, REGION_SCORE_A.w, REGION_SCORE_A.h, playerA_color);
2396 tft.fillRect(REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y, REGION_MATCH_TARGET.w, REGION_MATCH_TARGET.h, matchPanelBg);
2397 tft.fillRect(REGION_LOG.x, REGION_LOG.y, REGION_LOG.w, REGION_LOG.h, panel);
2398 tft.fillRect(REGION_SETTINGS.x, REGION_SETTINGS.y, REGION_SETTINGS.w, REGION_SETTINGS.h, panel);

```

```

2399 tft.fillRect(REGION_SCORE_B.x, REGION_SCORE_B.y, REGION_SCORE_B.w, REGION_SCORE_B.h, playerB_color);
2400
2401 tft.drawRoundRect(REGION_HEADER_A.x, REGION_HEADER_A.y, REGION_HEADER_A.w, REGION_HEADER_A.h, 4, TFT_ACCENT_A);
2402 tft.drawRoundRect(REGION_HEADER_B.x, REGION_HEADER_B.y, REGION_HEADER_B.w, REGION_HEADER_B.h, 4, TFT_ACCENT_B);
2403 tft.drawRoundRect(REGION_PLAY_PAUSE.x, REGION_PLAY_PAUSE.y, REGION_PLAY_PAUSE.w, REGION_PLAY_PAUSE.h, 6, TFT_BUTTON_BORDER);
2404 tft.drawRoundRect(REGION_END_GAME.x, REGION_END_GAME.y, REGION_END_GAME.w, REGION_END_GAME.h, 6, TFT_BUTTON_BORDER);
2405 tft.drawRoundRect(REGION_END_MATCH.x, REGION_END_MATCH.y, REGION_END_MATCH.w, REGION_END_MATCH.h, 6, TFT_BUTTON_BORDER);
2406 drawTimerPanel(PYER_B, REGION_TIMER_B);
2407 tft.drawRoundRect(REGION_SCORE_A.x, REGION_SCORE_A.y, REGION_SCORE_A.w, REGION_SCORE_A.h, 6, TFT_ACCENT_A);
2408 tft.drawRoundRect(REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y, REGION_MATCH_TARGET.w, REGION_MATCH_TARGET.h, 6, TFT_BUTTON_BORDER);
2409 tft.drawRoundRect(REGION_LOG.x, REGION_LOG.y, REGION_LOG.w, REGION_LOG.h, 6, TFT_BUTTON_BORDER);
2410 tft.drawRoundRect(REGION_SETTINGS.x, REGION_SETTINGS.y, REGION_SETTINGS.w, REGION_SETTINGS.h, 6, TFT_BUTTON_BORDER);
2411 tft.drawRoundRect(REGION_SCORE_B.x, REGION_SCORE_B.y, REGION_SCORE_B.w, REGION_SCORE_B.h, 6, TFT_ACCENT_B);
2412
2413 tft.setTextSize(2);
2414 tft.setTextColor(text, panel);
2415 drawCenteredInRect(playerA_name, REGION_HEADER_A, playerA_name.length() > 9 ? 1 : 2, headerAText, headerABg);
2416 drawCenteredInRect(playerB_name, REGION_HEADER_B, playerB_name.length() > 9 ? 1 : 2, headerBText, headerBBg);
2417
2418 drawCenteredInRect(playPauseLabel(), REGION_PLAY_PAUSE, 2, text, panel);
2419
2420 drawCenteredInRect(mode == MODE_TIMEOUT ? "New Game" : "End Game", REGION_END_GAME, 2, text, panel);
2421 drawCenteredInRect(mode == MODE_MATCH_OVER ? "Reset" : "End Match", REGION_END_MATCH, 2,
2422     mode == MODE_MATCH_OVER ? TFT_TEXT : text, mode == MODE_MATCH_OVER ? TFT_DANGER : panel);
2423 tft.setTextColor(text, panel);
2424
2425 int displayScoreA = scoreDisplayMode == SCORE_POINTS_TO_WIN ? max(0, match_target - playerA_score) : playerA_score;
2426 int displayScoreB = scoreDisplayMode == SCORE_POINTS_TO_WIN ? max(0, match_target - playerB_score) : playerB_score;
2427 Rect scoreNumberA = REGION_SCORE_A;
2428 Rect scoreNumberB = REGION_SCORE_B;
2429 if (scoreDisplayMode == SCORE_POINTS_TO_WIN) {
2430     scoreNumberA = {REGION_SCORE_A.x, REGION_SCORE_A.y + 4, REGION_SCORE_A.w, REGION_SCORE_A.h - 48};
2431     scoreNumberB = {REGION_SCORE_B.x, REGION_SCORE_B.y + 4, REGION_SCORE_B.w, REGION_SCORE_B.h - 48};
2432 }
2433 drawScoreInRect(displayScoreA, scoreNumberA, textColorForPlayerColor(playerA_color), playerA_color);
2434 drawScoreInRect(displayScoreB, scoreNumberB, textColorForPlayerColor(playerB_color), playerB_color);
2435 if (scoreDisplayMode == SCORE_POINTS_TO_WIN) {
2436     drawCenteredInRect("Points to Win", {REGION_SCORE_A.x, REGION_SCORE_A.y + REGION_SCORE_A.h - 46, REGION_SCORE_A.w, 34},
2437         2, textColorForPlayerColor(playerA_color), playerA_color);
2438     drawCenteredInRect("Points to Win", {REGION_SCORE_B.x, REGION_SCORE_B.y + REGION_SCORE_B.h - 46, REGION_SCORE_B.w, 34},
2439         2, textColorForPlayerColor(playerB_color), playerB_color);
2440 }
2441
2442 drawCenteredInRect("Match to:", {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 22, REGION_MATCH_TARGET.w, 26}, 2,
2443     matchPanelText, matchPanelBg);
2444 drawMatchTargetInRect(match_target, {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 56, REGION_MATCH_TARGET.w, 74},
2445     matchPanelText, matchPanelBg);
2446 drawCenteredInRect(String("Game ") + String(game_number),
2447     {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 136, REGION_MATCH_TARGET.w, 28}, 2,
2448     matchPanelText, matchPanelBg);
2449 String stateLabel = "";
2450 uint16_t stateColor = TFT_MAGENTA;
2451 if (mode == MODE_MATCH_OVER) {
2452     stateLabel = "Match Ended";
2453     stateColor = TFT_WARN;
2454 } else if (showCrawford) {
2455     stateLabel = "CRAWFORD";
2456     stateColor = TFT_CRAWFORD_LABEL;
2457 } else if (showPostCrawford) {
2458     stateLabel = "POST CRAWFORD";
2459     stateColor = TFT_POST_CRAWFORD_TEXT;
2460 }
2461 drawCenteredInRect(stateLabel, {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 166, REGION_MATCH_TARGET.w, 30}, 2,
2462     stateColor, matchPanelBg);
2463
2464 tft.setTextColor(text, panel);
2465 drawCenteredInRect("LOG", REGION_LOG, 2, text, panel);
2466 drawCenteredInRect("SETTINGS", REGION_SETTINGS, 2, text, panel);
2467
2468 if (mode == MODE_TIMEOUT) {
2469     drawTimeoutBanner();
2470 }
2471
2472 if (mode == MODE_CONFIRM_END_MATCH) {
2473     drawEndMatchConfirm();
2474 }
2475
2476 }
2477
2478 void drawMatchWinnerBanner() {
2479     const Rect banner = {6, 204, 788, 260};
2480     const Rect nameLine = {24, 270, 752, 76};
2481     const Rect winnerLine = {24, 348, 752, 76};
2482     String winnerName = (matchWinner == PLAYER_A) ? playerA_name : playerB_name;
2483
2484     tft.fillRect(banner.x, banner.y, banner.w, banner.h, TFT_PANEL);

```

```

2485 tft.drawRoundRect(banner.x, banner.y, banner.w, banner.h, 6, TFT_OK);
2486 drawCenteredInRect(winnerName, nameLine, winnerName.length() > 8 ? 6 : 7, TFT_TEXT, TFT_PANEL);
2487 drawCenteredInRect("IS THE WINNER", winnerLine, 5, TFT_OK, TFT_PANEL);
2488 }
2489
2490 void drawTimeoutBanner() {
2491     const Rect banner = {42, 224, 716, 174};
2492     Player loser = timeoutPlayer;
2493     String loserName = (loser == PLAYER_A) ? playerA_name : playerB_name;
2494
2495     tft.fillRect(banner.x, banner.y, banner.w, banner.h, TFT_PANEL);
2496     tft.drawRoundRect(banner.x, banner.y, banner.w, banner.h, 6, TFT_DANGER);
2497     drawCenteredInRect(loserName, {banner.x + 16, banner.y + 24, banner.w - 32, 48}, loserName.length() > 8 ? 4 : 5, TFT_TEXT, TFT_PANEL);
2498     drawCenteredInRect("RAN OUT OF TIME", {banner.x + 16, banner.y + 80, banner.w - 32, 36}, 3, TFT_DANGER, TFT_PANEL);
2499     drawCenteredInRect("LOSES THE GAME", {banner.x + 16, banner.y + 122, banner.w - 32, 34}, 3, TFT_WARN, TFT_PANEL);
2500 }
2501
2502 void drawEndMatchConfirm() {
2503     const Rect dialog = {118, 132, 564, 236};
2504
2505     tft.fillRect(dialog.x, dialog.y, dialog.w, dialog.h, TFT_PANEL);
2506     tft.drawRoundRect(dialog.x, dialog.y, dialog.w, dialog.h, 8, TFT_BUTTON_BORDER);
2507
2508     drawCenteredInRect("END MATCH?", {dialog.x, dialog.y + 28, dialog.w, 34}, 3, TFT_WARN, TFT_PANEL);
2509     drawCenteredInRect("This will finish the match", {dialog.x, dialog.y + 84, dialog.w, 24}, 2, TFT_TEXT, TFT_PANEL);
2510     drawCenteredInRect("and record the current leader.", {dialog.x, dialog.y + 112, dialog.w, 24}, 2, TFT_TEXT, TFT_PANEL);
2511
2512     tft.fillRect(CONFIRM_NO.x, CONFIRM_NO.y, CONFIRM_NO.w, CONFIRM_NO.h, TFT_PANEL_ALT);
2513     tft.drawRoundRect(CONFIRM_NO.x, CONFIRM_NO.y, CONFIRM_NO.w, CONFIRM_NO.h, 6, TFT_BUTTON_BORDER);
2514     drawCenteredInRect("NO", CONFIRM_NO, 3, TFT_TEXT, TFT_PANEL_ALT);
2515
2516     tft.fillRect(CONFIRM_YES.x, CONFIRM_YES.y, CONFIRM_YES.w, CONFIRM_YES.h, TFT_DANGER);
2517     tft.drawRoundRect(CONFIRM_YES.x, CONFIRM_YES.y, CONFIRM_YES.w, CONFIRM_YES.h, 6, TFT_BUTTON_BORDER);
2518     drawCenteredInRect("YES", CONFIRM_YES, 3, TFT_TEXT, TFT_DANGER);
2519 }
2520
2521 void drawMatchLog() {
2522     tft.fillScreen(TFT_BACKGROUND);
2523
2524     tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
2525     tft.setTextSize(3);
2526     tft.setCursor(286, 28);
2527     tft.print("MATCH LOG");
2528
2529     tft.fillRect(LOG_BACK.x, LOG_BACK.y, LOG_BACK.w, LOG_BACK.h, TFT_PANEL);
2530     tft.drawRoundRect(LOG_BACK.x, LOG_BACK.y, LOG_BACK.w, LOG_BACK.h, 6, TFT_BORDER);
2531     drawCenteredInRect("BACK", LOG_BACK, 2, TFT_TEXT, TFT_PANEL);
2532
2533     tft.fillRect(LOG_CLEAR.x, LOG_CLEAR.y, LOG_CLEAR.w, LOG_CLEAR.h, TFT_DANGER);
2534     tft.drawRoundRect(LOG_CLEAR.x, LOG_CLEAR.y, LOG_CLEAR.w, LOG_CLEAR.h, 6, TFT_BORDER);
2535     drawCenteredInRect("CLEAR", LOG_CLEAR, 2, TFT_TEXT, TFT_DANGER);
2536
2537     if (matchLogCount == 0) {
2538         drawCenteredInRect("NO MATCHES YET", {120, 210, 560, 70}, 3, TFT_MUTED, TFT_BACKGROUND);
2539         return;
2540     }
2541
2542     for (uint8_t i = 0; i < matchLogCount && i < MATCH_LOG_MAX_ENTRIES; i++) {
2543         const int16_t y = 88 + (i * 46);
2544         const Rect row = {34, y, 732, 40};
2545         String date = matchLogPart(matchLog[i], 0);
2546         String nameA = matchLogPart(matchLog[i], 1);
2547         String nameB = matchLogPart(matchLog[i], 2);
2548         String score = matchLogPart(matchLog[i], 3);
2549         String names = nameA + " VS " + nameB;
2550         String scoreLine = "SCORE " + score;
2551
2552         tft.fillRect(row.x, row.y, row.w, row.h, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
2553         tft.drawRoundRect(row.x, row.y, row.w, row.h, 4, TFT_BORDER);
2554         tft.setTextColor(TFT_MUTED, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
2555         tft.setTextSize(1);
2556         tft.setCursor(row.x + 12, row.y + 8);
2557         tft.print(date);
2558         tft.setCursor(row.x + 152, row.y + 8);
2559         tft.print(names);
2560         tft.setTextColor(TFT_TEXT, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
2561         tft.setTextSize(2);
2562         tft.setCursor(row.x + 560, row.y + 12);
2563         tft.print(scoreLine);
2564     }
2565 }
2566
2567 void drawClockSettings() {
2568     tft.fillScreen(TFT_BACKGROUND);
2569
2570     tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);

```

```

2571 tft.setTextSize(3);
2572 tft.setCursor(248, 28);
2573 tft.print("CLOCK SET");
2574
2575 tft.fillRect(CLOCK_BACK.x, CLOCK_BACK.y, CLOCK_BACK.w, CLOCK_BACK.h, TFT_PANEL);
2576 tft.drawRoundRect(CLOCK_BACK.x, CLOCK_BACK.y, CLOCK_BACK.w, CLOCK_BACK.h, 6, TFT_BORDER);
2577 drawCenteredInRect("BACK", CLOCK_BACK, 2, TFT_TEXT, TFT_PANEL);
2578
2579 const Rect rows[] = {
2580     {52, 82, 696, 48},
2581     {52, 140, 696, 48},
2582     {52, 198, 696, 48},
2583     {52, 256, 696, 48},
2584     {52, 314, 696, 48}
2585 };
2586 const Rect minusButtons[] = {
2587     CLOCK_YEAR_MINUS,
2588     CLOCK_MONTH_MINUS,
2589     CLOCK_DAY_MINUS,
2590     CLOCK_HOUR_MINUS,
2591     CLOCK_MINUTE_MINUS
2592 };
2593 const Rect plusButtons[] = {
2594     CLOCK_YEAR_PLUS,
2595     CLOCK_MONTH_PLUS,
2596     CLOCK_DAY_PLUS,
2597     CLOCK_HOUR_PLUS,
2598     CLOCK_MINUTE_PLUS
2599 };
2600 const char *labels[] = {"YEAR", "MONTH", "DAY", "HOUR", "MINUTE"};
2601 uint16_t values[] = {editingRtcYear, editingRtcMonth, editingRtcDay, editingRtcHour, editingRtcMinute};
2602
2603 for (uint8_t i = 0; i < 5; i++) {
2604     tft.fillRect(rows[i].x, rows[i].y, rows[i].w, rows[i].h, TFT_PANEL);
2605     tft.drawRoundRect(rows[i].x, rows[i].y, rows[i].w, rows[i].h, 6, TFT_BORDER);
2606     tft.setTextColor(TFT_TEXT, TFT_PANEL);
2607     tft.setTextSize(2);
2608     tft.setCursor(76, rows[i].y + 16);
2609     tft.print(labels[i]);
2610     tft.setCursor(i == 0 ? 514 : 536, rows[i].y + 16);
2611     if (i == 0) {
2612         tft.printf("%u", values[i]);
2613     } else {
2614         tft.printf("%02u", values[i]);
2615     }
2616
2617     tft.fillRect(minusButtons[i].x, minusButtons[i].y, minusButtons[i].w, minusButtons[i].h, TFT_PANEL_ALT);
2618     tft.drawRoundRect(minusButtons[i].x, minusButtons[i].y, minusButtons[i].w, minusButtons[i].h, 6, TFT_BORDER);
2619     drawCenteredInRect("-", minusButtons[i], 3, TFT_TEXT, TFT_PANEL_ALT);
2620     tft.fillRect(plusButtons[i].x, plusButtons[i].y, plusButtons[i].w, plusButtons[i].h, TFT_PANEL_ALT);
2621     tft.drawRoundRect(plusButtons[i].x, plusButtons[i].y, plusButtons[i].w, plusButtons[i].h, 6, TFT_BORDER);
2622     drawCenteredInRect("+", plusButtons[i], 3, TFT_TEXT, TFT_PANEL_ALT);
2623 }
2624
2625 tft.setTextColor(ds3231Found ? TFT_MUTED : TFT_DANGER, TFT_BACKGROUND);
2626 tft.setTextSize(1);
2627 tft.setCursor(72, 380);
2628 tft.print(ds3231Found ? "SAVE writes this date/time to the DS3231 RTC." : "DS3231 RTC not detected on I2C address 0x68.");
2629
2630 tft.fillRect(CLOCK_SAVE.x, CLOCK_SAVE.y, CLOCK_SAVE.w, CLOCK_SAVE.h, ds3231Found ? TFT_OK : TFT_PANEL_ALT);
2631 tft.drawRoundRect(CLOCK_SAVE.x, CLOCK_SAVE.y, CLOCK_SAVE.w, CLOCK_SAVE.h, 6, TFT_BORDER);
2632 drawCenteredInRect("SAVE", CLOCK_SAVE, 2, ds3231Found ? ILI9341_BLACK : TFT_MUTED, ds3231Found ? TFT_OK : TFT_PANEL_ALT);
2633 }
2634
2635 void drawAppearanceSettings() {
2636     uint16_t background = uiBackgroundColor();
2637     uint16_t panel = uiPanelColor();
2638     uint16_t text = uiTextColor();
2639
2640     tft.fillScreen(background);
2641     drawStatusBar();
2642     drawCenteredInRect("APPEARANCE", {180, 42, 440, 46}, 3, text, background);
2643
2644     tft.fillRect(APPEARANCE_BACK.x, APPEARANCE_BACK.y, APPEARANCE_BACK.w, APPEARANCE_BACK.h, panel);
2645     tft.drawRoundRect(APPEARANCE_BACK.x, APPEARANCE_BACK.y, APPEARANCE_BACK.w, APPEARANCE_BACK.h, 6, TFT_BORDER);
2646     drawCenteredInRect("BACK", APPEARANCE_BACK, 2, text, panel);
2647
2648     drawCenteredInRect("COLOR THEME", {170, 92, 460, 30}, 2, text, background);
2649     drawCenteredInRect("SCORE DISPLAY", {170, 246, 460, 30}, 2, text, background);
2650
2651     const Rect buttons[] = {APPEARANCE_NIGHT, APPEARANCE_DAY, APPEARANCE_SCORE, APPEARANCE_POINTS};
2652     const char *labels[] = {"NIGHT", "DAY", "SCORE", "POINTS TO WIN"};
2653     const bool selected[] = {
2654         displayTheme == THEME_NIGHT,
2655         displayTheme == THEME_DAY,
2656         scoreDisplayMode == SCORE_STANDARD,

```

```

2657     scoreDisplayMode == SCORE_POINTS_TO_WIN
2658 };
2659
2660 for (uint8_t i = 0; i < 4; i++) {
2661     uint16_t buttonBg = selected[i] ? TFT_OK : panel;
2662     uint16_t buttonText = selected[i] ? ILI9341_BLACK : text;
2663     tft.fillRect(buttons[i].x, buttons[i].y, buttons[i].w, buttons[i].h, buttonBg);
2664     tft.drawRoundRect(buttons[i].x, buttons[i].y, buttons[i].w, buttons[i].h, 6, TFT_BORDER);
2665     drawCenteredInRect(labels[i], buttons[i], 2, buttonText, buttonBg);
2666 }
2667 }
2668
2669 void drawNameEditor() {
2670     const Rect title = {70, 24, 660, 42};
2671     const Rect input = {92, 78, 616, 64};
2672     const Rect cancel = {72, 420, 168, 44};
2673     const Rect clearKey = {258, 420, 168, 44};
2674     const Rect ok = {560, 420, 168, 44};
2675     const char *rows[] = {"ABCDEFGHIJ", "KLMNOPQRST", "UVWXYZ"};
2676     const int16_t rowY[] = {148, 208, 268};
2677     const int16_t keyW = 66;
2678     const int16_t keyH = 44;
2679     const int16_t gap = 8;
2680     const int16_t swatchY = 326;
2681     const int16_t swatchSize = 36;
2682     const int16_t swatchGap = 14;
2683
2684     tft.fillRect(TFT_BACKGROUND);
2685     tft.fillRect(48, 14, 704, 452, TFT_PANEL);
2686     tft.drawRoundRect(48, 14, 704, 452, 8, TFT_BORDER);
2687     drawCenteredInRect(editingNameFor == PLAYER_A ? "PLAYER A NAME" : "PLAYER B NAME", title, 3, TFT_TEXT, TFT_PANEL);
2688
2689     tft.fillRect(input.x, input.y, input.w, input.h, TFT_BACKGROUND);
2690     tft.drawRoundRect(input.x, input.y, input.w, input.h, 6, TFT_OK);
2691     drawCenteredInRect(editingName.length() == 0 ? " " : editingName, input, 4, TFT_TEXT, TFT_BACKGROUND);
2692
2693     for (uint8_t r = 0; r < 3; r++) {
2694         uint8_t count = strlen(rows[r]);
2695         int16_t startX = (PANEL_WIDTH - ((count * keyW) + ((count - 1) * gap))) / 2;
2696         for (uint8_t i = 0; i < count; i++) {
2697             Rect key = {startX + (int16_t)i * (keyW + gap), rowY[r], keyW, keyH};
2698             tft.fillRect(key.x, key.y, key.w, key.h, TFT_PANEL_ALT);
2699             tft.drawRoundRect(key.x, key.y, key.w, key.h, 6, TFT_BORDER);
2700             String label;
2701             label += rows[r][i];
2702             drawCenteredInRect(label, key, 3, TFT_TEXT, TFT_PANEL_ALT);
2703         }
2704     }
2705
2706     for (uint8_t i = 0; i < playerColorOptionCount(); i++) {
2707         int16_t startX = (PANEL_WIDTH - (playerColorOptionCount() * swatchSize + (playerColorOptionCount() - 1) * swatchGap)) / 2;
2708         Rect swatch = {startX + (int16_t)i * (swatchSize + swatchGap), swatchY, swatchSize, swatchSize};
2709         uint16_t color = playerColorOption(i);
2710         tft.fillRect(swatch.x, swatch.y, swatch.w, swatch.h, color);
2711         tft.drawRoundRect(swatch.x, swatch.y, swatch.w, swatch.h, 5, color == editingPlayerColor ? TFT_OK : TFT_BORDER);
2712         if (color == PLAYER_COLOR_WHITE) {
2713             tft.drawRoundRect(swatch.x + 3, swatch.y + 3, swatch.w - 6, swatch.h - 6, 4, TFT_MUTED);
2714         }
2715     }
2716
2717     const Rect spaceKey = {166, 372, 244, 38};
2718     const Rect delKey = {430, 372, 204, 38};
2719     tft.fillRect(spaceKey.x, spaceKey.y, spaceKey.w, spaceKey.h, TFT_PANEL_ALT);
2720     tft.drawRoundRect(spaceKey.x, spaceKey.y, spaceKey.w, spaceKey.h, 6, TFT_BORDER);
2721     drawCenteredInRect("SPACE", spaceKey, 2, TFT_TEXT, TFT_PANEL_ALT);
2722     tft.fillRect(delKey.x, delKey.y, delKey.w, delKey.h, TFT_PANEL_ALT);
2723     tft.drawRoundRect(delKey.x, delKey.y, delKey.w, delKey.h, 6, TFT_BORDER);
2724     drawCenteredInRect("DEL", delKey, 2, TFT_TEXT, TFT_PANEL_ALT);
2725
2726     tft.fillRect(cancel.x, cancel.y, cancel.w, cancel.h, TFT_DANGER);
2727     tft.drawRoundRect(cancel.x, cancel.y, cancel.w, cancel.h, 6, TFT_BORDER);
2728     drawCenteredInRect("CANCEL", cancel, 2, TFT_TEXT, TFT_DANGER);
2729     tft.fillRect(clearKey.x, clearKey.y, clearKey.w, clearKey.h, TFT_PANEL_ALT);
2730     tft.drawRoundRect(clearKey.x, clearKey.y, clearKey.w, clearKey.h, 6, TFT_BORDER);
2731     drawCenteredInRect("CLEAR", clearKey, 2, TFT_TEXT, TFT_PANEL_ALT);
2732     tft.fillRect(ok.x, ok.y, ok.w, ok.h, TFT_OK);
2733     tft.drawRoundRect(ok.x, ok.y, ok.w, ok.h, 6, TFT_BORDER);
2734     drawCenteredInRect("OK", ok, 2, ILI9341_BLACK, TFT_OK);
2735 }
2736
2737 void openNameEditor(Player player) {
2738     if (player != PLAYER_A && player != PLAYER_B) {
2739         return;
2740     }
2741
2742     clock_running = false;

```

```

2743     active_player = PLAYER_NONE;
2744     editingNameFor = player;
2745     editingName = (player == PLAYER_A) ? playerA_name : playerB_name;
2746     editingPlayerColor = colorForPlayer(player);
2747     mode = MODE_NAME_EDIT;
2748     markDisplayDirty();
2749 }
2750
2751 void openClockSettings() {
2752     uint8_t second = 0;
2753     uint8_t minute = 0;
2754     uint8_t hour = 0;
2755     uint8_t day = 0;
2756     uint8_t month = 0;
2757     uint16_t year = 0;
2758     if (ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) && rtcDateLooksSet(day, month, year)) {
2759         editingRtcYear = year;
2760         editingRtcMonth = month;
2761         editingRtcDay = day;
2762         editingRtcHour = hour;
2763         editingRtcMinute = minute;
2764     } else {
2765         editingRtcYear = 2026;
2766         editingRtcMonth = 1;
2767         editingRtcDay = 1;
2768         editingRtcHour = 12;
2769         editingRtcMinute = 0;
2770     }
2771     mode = MODE_CLOCK_SET;
2772     markDisplayDirty();
2773 }
2774
2775 void handleClockSettingsTap(uint16_t x, uint16_t y) {
2776     if (CLOCK_BACK.containsPadded(x, y, 18, 14)) {
2777         mode = MODE_MENU;
2778         markDisplayDirty();
2779         return;
2780     }
2781
2782     const Rect minusButtons[] = {
2783         CLOCK_YEAR_MINUS,
2784         CLOCK_MONTH_MINUS,
2785         CLOCK_DAY_MINUS,
2786         CLOCK_HOUR_MINUS,
2787         CLOCK_MINUTE_MINUS
2788     };
2789     const Rect plusButtons[] = {
2790         CLOCK_YEAR_PLUS,
2791         CLOCK_MONTH_PLUS,
2792         CLOCK_DAY_PLUS,
2793         CLOCK_HOUR_PLUS,
2794         CLOCK_MINUTE_PLUS
2795     };
2796
2797     for (uint8_t i = 0; i < 5; i++) {
2798         if (minusButtons[i].containsPadded(x, y, 36, 12)) {
2799             adjustRtcField(i, -1);
2800             return;
2801         }
2802         if (plusButtons[i].containsPadded(x, y, 36, 12)) {
2803             adjustRtcField(i, +1);
2804             return;
2805         }
2806     }
2807
2808     if (CLOCK_SAVE.containsPadded(x, y, 18, 14) && ds3231Found) {
2809         if (ds3231SetTime(0, editingRtcMinute, editingRtcHour, editingRtcDay, editingRtcMonth, editingRtcYear)) {
2810             logPrintf("DS3231 RTC set to %04u-%02u-%02u %02u:%02u:00\n",
2811                 editingRtcYear, editingRtcMonth, editingRtcDay, editingRtcHour, editingRtcMinute);
2812             updateRtcStatusCache(true);
2813             mode = MODE_MENU;
2814         } else {
2815             logLine("DS3231 RTC save failed.");
2816         }
2817         markDisplayDirty();
2818         return;
2819     }
2820 }
2821
2822 void handleAppearanceSettingsTap(uint16_t x, uint16_t y) {
2823     if (APPEARANCE_BACK.containsPadded(x, y, 18, 14)) {
2824         mode = MODE_MENU;
2825         markDisplayDirty();
2826         return;
2827     }
2828 }

```

```

2829     if (APPEARANCE_NIGHT.containsPadded(x, y, 12, 10)) {
2830         displayTheme = THEME_NIGHT;
2831     } else if (APPEARANCE_DAY.containsPadded(x, y, 12, 10)) {
2832         displayTheme = THEME_DAY;
2833     } else if (APPEARANCE_SCORE.containsPadded(x, y, 12, 10)) {
2834         scoreDisplayMode = SCORE_STANDARD;
2835     } else if (APPEARANCE_POINTS.containsPadded(x, y, 12, 10)) {
2836         scoreDisplayMode = SCORE_POINTS_TO_WIN;
2837     } else {
2838         return;
2839     }
2840
2841     markSettingsDirty();
2842     markDisplayDirty();
2843 }
2844
2845 void handleEndMatchConfirmTap(uint16_t x, uint16_t y) {
2846     if (CONFIRM_NO.containsPadded(x, y, 18, 14)) {
2847         mode = MODE_MATCH;
2848         markDisplayDirty();
2849         return;
2850     }
2851
2852     if (CONFIRM_YES.containsPadded(x, y, 18, 14)) {
2853         forceEndMatch();
2854         return;
2855     }
2856 }
2857
2858 void adjustRtcField(uint8_t field, int delta) {
2859     switch (field) {
2860     case 0:
2861         editingRtcYear = constrain((int)editingRtcYear + delta, 2024, 2099);
2862         break;
2863     case 1:
2864         editingRtcMonth = constrain((int)editingRtcMonth + delta, 1, 12);
2865         break;
2866     case 2:
2867         editingRtcDay = constrain((int)editingRtcDay + delta, 1, (int)daysInMonth(editingRtcMonth, editingRtcYear));
2868         break;
2869     case 3:
2870         editingRtcHour = (uint8_t)(((int)editingRtcHour + delta + 24) % 24);
2871         break;
2872     case 4:
2873         editingRtcMinute = (uint8_t)(((int)editingRtcMinute + delta + 60) % 60);
2874         break;
2875     default:
2876         break;
2877     }
2878
2879     uint8_t maxDay = daysInMonth(editingRtcMonth, editingRtcYear);
2880     if (editingRtcDay > maxDay) {
2881         editingRtcDay = maxDay;
2882     }
2883     markDisplayDirty();
2884 }
2885
2886 void handleNameEditorTap(uint16_t x, uint16_t y) {
2887     const Rect cancel = {72, 420, 168, 44};
2888     const Rect clearKey = {258, 420, 168, 44};
2889     const Rect ok = {560, 420, 168, 44};
2890     const Rect spaceKey = {166, 372, 244, 38};
2891     const Rect delKey = {430, 372, 204, 38};
2892     const char *rows[] = {"ABCDEFGH I J", "KLMNOPQRST", "UVWXYZ"};
2893     const int16_t rowY[] = {148, 208, 268};
2894     const int16_t keyW = 66;
2895     const int16_t keyH = 44;
2896     const int16_t gap = 8;
2897     const int16_t swatchY = 326;
2898     const int16_t swatchSize = 36;
2899     const int16_t swatchGap = 14;
2900
2901     if (cancel.contains(x, y)) {
2902         editingNameFor = PLAYER_NONE;
2903         mode = MODE_MATCH;
2904         markDisplayDirty();
2905         return;
2906     }
2907
2908     if (ok.contains(x, y)) {
2909         if (editingName.length() == 0) {
2910             editingName = (editingNameFor == PLAYER_A) ? "PLAYER A" : "PLAYER B";
2911         }
2912         if (editingNameFor == PLAYER_A) {
2913             playerA_name = editingName;
2914             playerA_color = editingPlayerColor;

```

```

2915     } else if (editingNameFor == PLAYER_B) {
2916         playerB_name = editingName;
2917         playerB_color = editingPlayerColor;
2918     }
2919     savePlayerNames();
2920     editingNameFor = PLAYER_NONE;
2921     mode = MODE_MATCH;
2922     markDisplayDirty();
2923     return;
2924 }
2925
2926 if (clearKey.contains(x, y)) {
2927     editingName = "";
2928     markDisplayDirty();
2929     return;
2930 }
2931
2932 if (delKey.contains(x, y)) {
2933     if (editingName.length() > 0) {
2934         editingName.remove(editingName.length() - 1);
2935     }
2936     markDisplayDirty();
2937     return;
2938 }
2939
2940 if (spaceKey.contains(x, y)) {
2941     if (editingName.length() < PLAYER_NAME_MAX_LEN && editingName.length() > 0) {
2942         editingName += ' ';
2943     }
2944     markDisplayDirty();
2945     return;
2946 }
2947
2948 int16_t swatchStartX = (PANEL_WIDTH - (playerColorOptionCount() * swatchSize + (playerColorOptionCount() - 1) * swatchGap)) / 2;
2949 for (uint8_t i = 0; i < playerColorOptionCount(); i++) {
2950     Rect swatch = {swatchStartX + (int16_t)i * (swatchSize + swatchGap), swatchY, swatchSize, swatchSize};
2951     if (swatch.contains(x, y)) {
2952         editingPlayerColor = playerColorOption(i);
2953         markDisplayDirty();
2954         return;
2955     }
2956 }
2957
2958 for (uint8_t r = 0; r < 3; r++) {
2959     uint8_t count = strlen(rows[r]);
2960     int16_t startX = (PANEL_WIDTH - ((count * keyW) + ((count - 1) * gap))) / 2;
2961     for (uint8_t i = 0; i < count; i++) {
2962         Rect key = {startX + (int16_t)i * (keyW + gap), rowY[r], keyW, keyH};
2963         if (key.contains(x, y) && editingName.length() < PLAYER_NAME_MAX_LEN) {
2964             editingName += rows[r][i];
2965             markDisplayDirty();
2966             return;
2967         }
2968     }
2969 }
2970 }
2971
2972 uint16_t colorForPlayer(Player player) {
2973     if (player == PLAYER_A) {
2974         return playerA_color;
2975     }
2976     if (player == PLAYER_B) {
2977         return playerB_color;
2978     }
2979     return TFT_PANEL;
2980 }
2981
2982 uint16_t textColorForPlayerColor(uint16_t color) {
2983     return (color == PLAYER_COLOR_WHITE || color == PLAYER_COLOR_YELLOW) ? TFT_TEXT_DARK : TFT_TEXT;
2984 }
2985
2986 uint16_t playerColorOption(uint8_t index) {
2987     switch (index) {
2988     case 0:
2989         return PLAYER_COLOR_RED;
2990     case 1:
2991         return PLAYER_COLOR_BLUE;
2992     case 2:
2993         return PLAYER_COLOR_GREEN;
2994     case 3:
2995         return PLAYER_COLOR_YELLOW;
2996     case 4:
2997         return PLAYER_COLOR_GRAY;
2998     case 5:
2999         return PLAYER_COLOR_WHITE;
3000     default:

```

```

3001     return PLAYER_COLOR_BLUE;
3002 }
3003 }
3004
3005 uint8_t playerColorOptionCount() {
3006     return 6;
3007 }
3008
3009 bool dayThemeEnabled() {
3010     return displayTheme == THEME_DAY;
3011 }
3012
3013 uint16_t uiBackgroundColor() {
3014     return dayThemeEnabled() ? 0x8410 : TFT_BACKGROUND;
3015 }
3016
3017 uint16_t uiPanelColor() {
3018     return dayThemeEnabled() ? 0xEF5D : TFT_PANEL;
3019 }
3020
3021 uint16_t uiTextColor() {
3022     return dayThemeEnabled() ? ILI9341_BLACK : TFT_TEXT;
3023 }
3024
3025 uint16_t uiMutedColor() {
3026     return dayThemeEnabled() ? 0x630C : TFT_MUTED;
3027 }
3028
3029 void drawTimerPanels() {
3030     if (mode != MODE_MATCH) {
3031         return;
3032     }
3033
3034     drawStatusBar();
3035     drawTimerPanel(PLAYER_A, REGION_TIMER_A);
3036     drawTimerPanel(PLAYER_B, REGION_TIMER_B);
3037 }
3038
3039 void drawTimerPanel(Player player, const Rect &region) {
3040     bool isA = (player == PLAYER_A);
3041     bool active = (active_player == player && clock_running);
3042     uint16_t accent = isA ? TFT_ACCENT_A : TFT_ACCENT_B;
3043     uint16_t panelColor = dayThemeEnabled() ? uiPanelColor() : (active ? ILI9341_WHITE : TFT_PANEL);
3044     uint32_t remaining = visibleReserveMs(player, millis());
3045     String value = formatTimeMs(remaining);
3046     uint16_t valueColor = dayThemeEnabled() ? ILI9341_BLACK : (active ? TFT_ACTIVE_TEXT : TFT_TEXT);
3047     bool showingDelay = false;
3048
3049     if (remaining <= 90UL * 1000UL) {
3050         panelColor = TFT_TIMER_WARNING_BG;
3051         valueColor = ILI9341_BLACK;
3052     }
3053
3054     if (active && timingMode == SIMPLE_DELAY && delay_seconds > 0) {
3055         uint32_t elapsedMs = millis() - turnStartMs;
3056         uint32_t delayMs = (uint32_t)delay_seconds * 1000UL;
3057         if (elapsedMs < delayMs) {
3058             uint32_t delayRemaining = (delayMs - elapsedMs + 999UL) / 1000UL;
3059             value = String(delayRemaining);
3060             valueColor = dayThemeEnabled() ? ILI9341_BLACK : (active ? TFT_ACTIVE_TEXT : TFT_WARN);
3061             showingDelay = true;
3062         }
3063     }
3064
3065     tft.fillRect(region.x, region.y, region.w, region.h, panelColor);
3066     tft.drawRoundRect(region.x, region.y, region.w, region.h, 6, accent);
3067
3068     #if BGMC_HAS_CLOCK_FONT
3069     tft.setFont(&BGMC_CLOCK_FONT);
3070     tft.setTextSize(1);
3071     tft.setTextColor(valueColor, panelColor);
3072     int16_t x1;
3073     int16_t y1;
3074     uint16_t w;
3075     uint16_t h;
3076     tft.getTextBounds(value, 0, 0, &x1, &y1, &w, &h);
3077     int16_t valueX = region.x + ((region.w - (int16_t)w) / 2) - x1;
3078     int16_t valueY = region.y + ((region.h - (int16_t)h) / 2) - y1;
3079     tft.setCursor(valueX, valueY);
3080     tft.print(value);
3081     tft.setFont();
3082     #else
3083     tft.setTextSize(showingDelay ? 6 : 4);
3084     tft.setTextColor(valueColor, panelColor);
3085     int16_t valueX = showingDelay ? (region.x + 104) : (region.x + 48);
3086     if (!showingDelay && remaining >= 10UL * 60UL * 1000UL) {

```

```

3087     valueX = region.x + 36;
3088 }
3089 tft.setCursor(valueX, showingDelay ? (region.y + 30) : (region.y + 38));
3090 tft.print(value);
3091 #endif
3092 }
3093
3094 void drawPlayerPanel(Player player, int16_t x, int16_t y, int16_t w, int16_t h) {
3095     bool isA = (player == PLAYER_A);
3096     bool active = (active_player == player && clock_running);
3097     uint16_t accent = isA ? TFT_ACCENT_A : TFT_ACCENT_B;
3098     uint16_t border = active ? TFT_OK : accent;
3099     uint32_t remaining = isA ? playerA_time_remaining : playerB_time_remaining;
3100     int score = isA ? playerA_score : playerB_score;
3101
3102     tft.fillRect(x, y, w, h, TFT_PANEL);
3103     tft.drawRoundRect(x, y, w, h, 6, border);
3104     tft.drawRoundRect(x + 3, y + 3, w - 6, h - 6, 6, TFT_PANEL_ALT);
3105     if (active) {
3106         tft.fillRect(x + 8, y + 8, w - 16, 10, TFT_OK);
3107     }
3108
3109     tft.setTextColor(accent, TFT_PANEL);
3110     tft.setTextSize(3);
3111     tft.setCursor(x + 24, y + 34);
3112     tft.print(isA ? "PLAYER A" : "PLAYER B");
3113
3114     tft.setTextColor((remaining <= LOW_TIME_WARNING_SECONDS) ? TFT_DANGER : TFT_TEXT, TFT_PANEL);
3115     tft.setTextSize(6);
3116     tft.setCursor(x + 24, y + 98);
3117     tft.print(formatTimeMs(remaining));
3118
3119     tft.fillRect(x + 24, y + 188, 150, 34, active ? TFT_OK : TFT_PANEL_ALT);
3120     tft.setTextColor(active ? ILI9341_BLACK : TFT_TEXT, active ? TFT_OK : TFT_PANEL_ALT);
3121     tft.setTextSize(2);
3122     tft.setCursor(x + 40, y + 198);
3123     tft.printf("SCORE %d", score);
3124
3125     tft.setTextColor(TFT_MUTED, TFT_PANEL);
3126     tft.setTextSize(1);
3127     tft.setCursor(x + 212, y + 204);
3128     tft.print(active ? "RUNNING" : "WAITING");
3129 }
3130
3131 void drawStatusBar() {
3132     tft.fillRect(REGION_STATUS.x, REGION_STATUS.y, REGION_STATUS.w, REGION_STATUS.h, TFT_PANEL);
3133     tft.fillRect(0, REGION_STATUS.h - 1, tft.width(), 1, TFT_BORDER);
3134     tft.setTextColor(TFT_TEXT, TFT_PANEL);
3135     tft.setTextSize(2);
3136     tft.setCursor(18, 10);
3137     tft.print("BATTERY");
3138     tft.setCursor(114, 10);
3139     int pct = batteryPercent();
3140     if (pct < 0) {
3141         tft.print("--%");
3142     } else {
3143         tft.printf("%d%%", pct);
3144     }
3145     if (chargingIndicatorOn()) {
3146         drawChargeBolt(168, 6, TFT_WARN, TFT_PANEL);
3147     }
3148
3149     tft.setTextColor(TFT_MUTED, TFT_PANEL);
3150     tft.setCursor(300, 10);
3151     tft.print("ELAPSED");
3152     tft.setTextColor(TFT_TEXT, TFT_PANEL);
3153     tft.setCursor(410, 10);
3154     tft.print(formatTime(match_elapsed_seconds));
3155
3156     tft.setTextColor(TFT_TEXT, TFT_PANEL);
3157     tft.setTextSize(2);
3158     tft.setCursor(tft.width() - 168, 10);
3159     tft.print(cachedRtcStatus);
3160 }
3161
3162 void drawChargeBolt(int16_t x, int16_t y, uint16_t color, uint16_t background) {
3163     tft.fillRect(x, y, 20, 24, background);
3164     fillTriangleRows(x + 11, y + 3, x + 5, y + 12, x + 11, y + 12, color);
3165     fillTriangleRows(x + 9, y + 10, x + 16, y + 10, x + 6, y + 21, color);
3166 }
3167
3168 void drawStartupTest() {
3169     logLine("Drawing startup splash image");
3170
3171     setBrightness(255);
3172     drawSplashImage();

```

```

3173     delay(4000);
3174 }
3175
3176 void drawSplashImage() {
3177     if (SPLASH_IMAGE_SCALE == 2) {
3178         tft.drawRGB565ImageScaled2x(0, 0, SPLASH_IMAGE_WIDTH, SPLASH_IMAGE_HEIGHT, SPLASH_IMAGE_RGB565);
3179     } else {
3180         tft.drawRGB565Image(0, 0, SPLASH_IMAGE_WIDTH, SPLASH_IMAGE_HEIGHT, SPLASH_IMAGE_RGB565);
3181     }
3182 }
3183
3184 void drawGeneratedSplashImage() {
3185     const uint16_t board = 0x0841;
3186     const uint16_t wood = 0x3922;
3187     const uint16_t woodLite = 0x7A84;
3188     const uint16_t gold = 0xDDA6;
3189     const uint16_t amber = 0xB3A0;
3190     const uint16_t cream = 0xDEB2;
3191     const uint16_t orange = 0x8A80;
3192     const uint16_t blackChecker = 0x10A2;
3193     const uint16_t whiteChecker = 0xF6D8;
3194     const uint16_t clockBg = 0x0000;
3195     const uint16_t silver = 0xD6BA;
3196
3197     tft.fillScreen(board);
3198     tft.fillRect(0, 0, 58, PANEL_HEIGHT, wood);
3199     tft.fillRect(742, 0, 58, PANEL_HEIGHT, wood);
3200     for (int16_t x = 8; x < 58; x += 12) {
3201         tft.fillRect(x, 0, 2, PANEL_HEIGHT, woodLite);
3202         tft.fillRect(742 + x, 0, 2, PANEL_HEIGHT, woodLite);
3203     }
3204
3205     const int16_t pointTop = 0;
3206     const int16_t pointBottom = PANEL_HEIGHT - 1;
3207     const int16_t pointH = 220;
3208     const int16_t leftPoints[] = {88, 140, 190};
3209     const int16_t rightPoints[] = {610, 662, 714};
3210     for (uint8_t i = 0; i < 3; i++) {
3211         uint16_t color = (i == 1) ? orange : cream;
3212         fillTriangleRows(leftPoints[i], pointTop, leftPoints[i] + 44, pointTop, leftPoints[i] + 22, pointTop + pointH, color);
3213         fillTriangleRows(rightPoints[i], pointTop, rightPoints[i] + 44, pointTop, rightPoints[i] + 22, pointTop + pointH, color);
3214         fillTriangleRows(leftPoints[i], pointBottom, leftPoints[i] + 44, pointBottom, leftPoints[i] + 22, pointBottom - pointH, color);
3215         fillTriangleRows(rightPoints[i], pointBottom, rightPoints[i] + 44, pointBottom, rightPoints[i] + 22, pointBottom - pointH, color);
3216     }
3217
3218     fillCircleRows(372, 120, 32, whiteChecker);
3219     fillCircleRows(418, 128, 28, blackChecker);
3220     fillCircleRows(372, 120, 22, 0xE5B1);
3221     fillCircleRows(418, 128, 19, 0x2945);
3222     tft.fillRect(296, 138, 78, 2, gold);
3223     tft.fillRect(450, 138, 106, 2, gold);
3224     tft.fillRect(314, 122, 36, 2, gold);
3225     tft.fillRect(318, 110, 24, 2, gold);
3226
3227     drawCenteredInRect("BACKGAMMON", {110, 166, 580, 60}, 6, TFT_TEXT, board);
3228     tft.fillRect(250, 236, 58, 2, gold);
3229     tft.fillRect(492, 236, 58, 2, gold);
3230     drawCenteredInRect("CLOCK", {298, 220, 204, 48}, 4, amber, board);
3231
3232     tft.fillRect(224, 288, 352, 78, clockBg);
3233     tft.drawRoundRect(224, 288, 352, 78, 8, gold);
3234     drawCenteredInRect("15:00 : 15:00", {242, 300, 316, 52}, 4, silver, clockBg);
3235     tft.setTextColor(amber, board);
3236     tft.setTextSize(2);
3237     tft.setCursor(254, 390);
3238     tft.print("FOCUS. STRATEGY. EVERY MOVE COUNTS.");
3239     tft.setCursor(286, 434);
3240     tft.print("PLAY FAIR. PLAY STRONG.");
3241 }
3242
3243 void fillTriangleRows(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color) {
3244     if (y1 < y0) {
3245         int16_t tx = x0; x0 = x1; x1 = tx;
3246         int16_t ty = y0; y0 = y1; y1 = ty;
3247     }
3248     if (y2 < y0) {
3249         int16_t tx = x0; x0 = x2; x2 = tx;
3250         int16_t ty = y0; y0 = y2; y2 = ty;
3251     }
3252     if (y2 < y1) {
3253         int16_t tx = x1; x1 = x2; x2 = tx;
3254         int16_t ty = y1; y1 = y2; y2 = ty;
3255     }
3256
3257     int16_t totalHeight = y2 - y0;
3258     if (totalHeight == 0) {

```

```

3259     return;
3260 }
3261
3262 for (int16_t y = y0; y <= y2; y++) {
3263     bool secondHalf = y > y1 || y1 == y0;
3264     int16_t segmentHeight = secondHalf ? (y2 - y1) : (y1 - y0);
3265     if (segmentHeight == 0) {
3266         continue;
3267     }
3268     float alpha = (float)(y - y0) / (float)totalHeight;
3269     float beta = secondHalf ? (float)(y - y1) / (float)segmentHeight : (float)(y - y0) / (float)segmentHeight;
3270     int16_t ax = x0 + (int16_t)((x2 - x0) * alpha);
3271     int16_t bx = secondHalf ? x1 + (int16_t)((x2 - x1) * beta) : x0 + (int16_t)((x1 - x0) * beta);
3272     if (ax > bx) {
3273         int16_t tmp = ax;
3274         ax = bx;
3275         bx = tmp;
3276     }
3277     tft.fillRect(ax, y, bx - ax + 1, 1, color);
3278 }
3279 }
3280
3281 void fillCircleRows(int16_t cx, int16_t cy, int16_t radius, uint16_t color) {
3282     int16_t r2 = radius * radius;
3283     for (int16_t y = -radius; y <= radius; y++) {
3284         int16_t x = 0;
3285         while ((x + 1) * (x + 1) + y * y <= r2) {
3286             x++;
3287         }
3288         tft.fillRect(cx - x, cy + y, (x * 2) + 1, 1, color);
3289     }
3290     tft.drawRoundRect(cx - radius, cy - radius, radius * 2, radius * 2, radius, TFT_BORDER);
3291 }
3292
3293 void initCrowPanelExpander() {
3294     Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
3295     Wire.setClock(400000);
3296     scanI2C("before PCA9557 init");
3297
3298     uint8_t pcaAddress = 0;
3299     for (uint8_t address = 0x18; address <= 0x1F; address++) {
3300         Wire.beginTransmission(address);
3301         if (Wire.endTransmission() == 0) {
3302             pcaAddress = address;
3303             break;
3304         }
3305     }
3306
3307     if (pcaAddress == 0) {
3308         logLine("PCA9557 not found on I2C; continuing without expander init");
3309         return;
3310     }
3311
3312     logPrintf("PCA9557 found at 0x%02X\n", pcaAddress);
3313     pca9557Address = pcaAddress;
3314
3315     // Elecrow V3 sequence: I00/I01 low, short delay, I00 high, I01 input.
3316     // Registers: 1=output port, 2=polarity, 3=configuration (1=input, 0=output).
3317     bool ok = true;
3318     ok &= pca9557Write(pcaAddress, 0x02, 0x00);
3319     ok &= pca9557Write(pcaAddress, 0x03, 0x00);
3320     ok &= pca9557Write(pcaAddress, 0x01, 0x00);
3321     delay(20);
3322     ok &= pca9557Write(pcaAddress, 0x01, 0x01);
3323     delay(100);
3324     ok &= pca9557Write(pcaAddress, 0x03, 0x02);
3325
3326     logPrintf("PCA9557 init: %s\n", ok ? "OK" : "FAILED");
3327     scanI2C("after PCA9557 init");
3328 }
3329
3330 bool pca9557Write(uint8_t address, uint8_t reg, uint8_t value) {
3331     Wire.beginTransmission(address);
3332     Wire.write(reg);
3333     Wire.write(value);
3334     return Wire.endTransmission() == 0;
3335 }
3336
3337 uint8_t pca9557Read(uint8_t address, uint8_t reg) {
3338     Wire.beginTransmission(address);
3339     Wire.write(reg);
3340     if (Wire.endTransmission(false) != 0) {
3341         return 0xFF;
3342     }
3343     if (Wire.requestFrom((int)address, 1) != 1) {
3344         return 0xFF;

```

```

3345     }
3346     return Wire.read();
3347 }
3348
3349 void initButtonExpander() {
3350     uint8_t foundAddress = 0;
3351     for (uint8_t address = 0x20; address <= 0x27; address++) {
3352         Wire.beginTransmission(address);
3353         if (Wire.endTransmission() == 0) {
3354             foundAddress = address;
3355             break;
3356         }
3357     }
3358
3359     if (foundAddress == 0) {
3360         logLine("PCF8575 button expander not found; physical I2C buttons disabled.");
3361         return;
3362     }
3363
3364     pcf8575Address = foundAddress;
3365     bool ok = pcf8575Write(pcf8575Address, 0xFFFF);
3366     logPrintf("PCF8575 button expander found at 0x%02X: %s\n", pcf8575Address, ok ? "OK" : "FAILED");
3367     logPrintf("PCF8575 buttons: Play/Pause=P%02u Player A=P%02u Player B=P%02u\n",
3368         PCF8575_PLAY_PAUSE_PIN, PCF8575_PLAYER_A_PIN, PCF8575_PLAYER_B_PIN);
3369     scanI2C("after PCF8575 init");
3370 }
3371
3372 bool pcf8575Write(uint8_t address, uint16_t value) {
3373     Wire.beginTransmission(address);
3374     Wire.write(value & 0xFF);
3375     Wire.write((value >> 8) & 0xFF);
3376     return Wire.endTransmission() == 0;
3377 }
3378
3379 uint16_t pcf8575Read(uint8_t address) {
3380     if (Wire.requestFrom((int)address, 2) != 2) {
3381         return 0xFFFF;
3382     }
3383     uint8_t low = Wire.read();
3384     uint8_t high = Wire.read();
3385     return (uint16_t)low | ((uint16_t)high << 8);
3386 }
3387
3388 void initFuelGauge() {
3389     Wire.beginTransmission(MAX17048_I2C_ADDRESS);
3390     if (Wire.endTransmission() != 0) {
3391         max17048Found = false;
3392         logLine("MAX17048 fuel gauge not found; battery display will use fallback.");
3393         return;
3394     }
3395
3396     uint16_t version = 0;
3397     max17048Found = true;
3398     max17048Read16(0x08, &version);
3399     logPrintf("MAX17048 fuel gauge found at 0x%02X version=0x%04X\n", MAX17048_I2C_ADDRESS, version);
3400     logPrintf("MAX17048 quick-start reset: %s\n", max17048QuickStart() ? "OK" : "FAILED");
3401     delay(250);
3402     logPrintf("MAX17048 battery: %.2fV %.1f%%\n", max17048CellVoltage(), max17048StateOfCharge());
3403     scanI2C("after MAX17048 init");
3404 }
3405
3406 bool max17048Read16(uint8_t reg, uint16_t *value) {
3407     if (value == nullptr) {
3408         return false;
3409     }
3410
3411     Wire.beginTransmission(MAX17048_I2C_ADDRESS);
3412     Wire.write(reg);
3413     if (Wire.endTransmission(false) != 0) {
3414         return false;
3415     }
3416
3417     if (Wire.requestFrom((int)MAX17048_I2C_ADDRESS, 2) != 2) {
3418         return false;
3419     }
3420
3421     uint8_t msb = Wire.read();
3422     uint8_t lsb = Wire.read();
3423     *value = ((uint16_t)msb << 8) | lsb;
3424     return true;
3425 }
3426
3427 bool max17048Write16(uint8_t reg, uint16_t value) {
3428     if (!max17048Found) {
3429         return false;
3430     }

```

```

3431 Wire.beginTransmission(MAX17048_I2C_ADDRESS);
3432 Wire.write(reg);
3433 Wire.write((value >> 8) & 0xFF);
3434 Wire.write(value & 0xFF);
3435 return Wire.endTransmission() == 0;
3436 }
3437
3438
3439 bool max17048QuickStart() {
3440     // MAX17048 MODE register quick-start command. This resets/restarts the
3441     // fuel-gauge algorithm without clearing the whole chip.
3442     return max17048Write16(0x06, 0x4000);
3443 }
3444
3445 float max17048CellVoltage() {
3446     uint16_t raw = 0;
3447     if (!max17048Found || !max17048Read16(0x02, &raw)) {
3448         return 0.0f;
3449     }
3450     return (float)(raw >> 4) * 0.00125f;
3451 }
3452
3453 float max17048StateOfCharge() {
3454     uint16_t raw = 0;
3455     if (!max17048Found || !max17048Read16(0x04, &raw)) {
3456         return 0.0f;
3457     }
3458     float pct = (float)(raw >> 8) + ((float)(raw & 0xFF) / 256.0f);
3459     if (pct < 0.0f) {
3460         pct = 0.0f;
3461     }
3462     if (pct > 100.0f) {
3463         pct = 100.0f;
3464     }
3465     return pct;
3466 }
3467
3468 void initRtcClock() {
3469     Wire.beginTransmission(DS3231_I2C_ADDRESS);
3470     if (Wire.endTransmission() != 0) {
3471         ds3231Found = false;
3472         logLine("DS3231 RTC not found; match log dates will be DATE UNSET.");
3473         return;
3474     }
3475
3476     ds3231Found = true;
3477     uint8_t second = 0;
3478     uint8_t minute = 0;
3479     uint8_t hour = 0;
3480     uint8_t day = 0;
3481     uint8_t month = 0;
3482     uint16_t year = 0;
3483     if (ds3231ReadTime(&second, &minute, &hour, &day, &month, &year)) {
3484         if (rtcDateLooksSet(day, month, year)) {
3485             logPrintf("DS3231 RTC found at 0x%02X date=%04u-%02u-%02u time=%02u:%02u:%02u\n",
3486                     DS3231_I2C_ADDRESS, year, month, day, hour, minute, second);
3487         } else {
3488             logPrintf("DS3231 RTC found at 0x%02X but date looks unset: %04u-%02u-%02u\n",
3489                     DS3231_I2C_ADDRESS, year, month, day);
3490         }
3491     } else {
3492         logPrintf("DS3231 RTC found at 0x%02X but time read failed.\n", DS3231_I2C_ADDRESS);
3493     }
3494     scanI2C("after DS3231 init");
3495 }
3496
3497 bool ds3231ReadTime(uint8_t *second, uint8_t *minute, uint8_t *hour, uint8_t *day, uint8_t *month, uint16_t *year) {
3498     if (second == nullptr || minute == nullptr || hour == nullptr || day == nullptr || month == nullptr || year == nullptr) {
3499         return false;
3500     }
3501
3502     Wire.beginTransmission(DS3231_I2C_ADDRESS);
3503     Wire.write(0x00);
3504     if (Wire.endTransmission(false) != 0) {
3505         return false;
3506     }
3507
3508     if (Wire.requestFrom((int)DS3231_I2C_ADDRESS, 7) != 7) {
3509         return false;
3510     }
3511
3512     uint8_t rawSecond = Wire.read();
3513     uint8_t rawMinute = Wire.read();
3514     uint8_t rawHour = Wire.read();
3515     Wire.read(); // Day of week, unused for match logs.
3516     uint8_t rawDay = Wire.read();

```

```

3517 uint8_t rawMonth = Wire.read();
3518 uint8_t rawYear = Wire.read();
3519
3520 *second = bcdToDec(rawSecond & 0x7F);
3521 *minute = bcdToDec(rawMinute & 0x7F);
3522 if ((rawHour & 0x40) != 0) {
3523     uint8_t hour12 = bcdToDec(rawHour & 0x1F);
3524     bool pm = (rawHour & 0x20) != 0;
3525     *hour = (hour12 % 12) + (pm ? 12 : 0);
3526 } else {
3527     *hour = bcdToDec(rawHour & 0x3F);
3528 }
3529 *day = bcdToDec(rawDay & 0x3F);
3530 *month = bcdToDec(rawMonth & 0x1F);
3531 *year = 2000 + bcdToDec(rawYear);
3532 return true;
3533 }
3534
3535 bool ds3231SetTime(uint8_t second, uint8_t minute, uint8_t hour, uint8_t day, uint8_t month, uint16_t year) {
3536     if (!ds3231Found || !rtcDateLooksSet(day, month, year) || hour > 23 || minute > 59 || second > 59) {
3537         return false;
3538     }
3539
3540     Wire.beginTransmission(DS3231_I2C_ADDRESS);
3541     Wire.write(0x00);
3542     Wire.write(decToBcd(second));
3543     Wire.write(decToBcd(minute));
3544     Wire.write(decToBcd(hour)); // 24-hour mode.
3545     Wire.write(decToBcd(1)); // Day of week is not used by this firmware.
3546     Wire.write(decToBcd(day));
3547     Wire.write(decToBcd(month));
3548     Wire.write(decToBcd((uint8_t)(year - 2000)));
3549     return Wire.endTransmission() == 0;
3550 }
3551
3552 bool rtcDateLooksSet(uint8_t day, uint8_t month, uint16_t year) {
3553     return year >= 2024 && year <= 2099 && month >= 1 && month <= 12 && day >= 1 && day <= 31;
3554 }
3555
3556 uint8_t daysInMonth(uint8_t month, uint16_t year) {
3557     switch (month) {
3558         case 4:
3559         case 6:
3560         case 9:
3561         case 11:
3562             return 30;
3563         case 2: {
3564             bool leap = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
3565             return leap ? 29 : 28;
3566         }
3567         default:
3568             return 31;
3569     }
3570 }
3571
3572 uint8_t bcdToDec(uint8_t value) {
3573     return ((value >> 4) * 10) + (value & 0x0F);
3574 }
3575
3576 uint8_t decToBcd(uint8_t value) {
3577     return ((value / 10) << 4) | (value % 10);
3578 }
3579
3580 String rtcStatusText() {
3581     uint8_t second = 0;
3582     uint8_t minute = 0;
3583     uint8_t hour = 0;
3584     uint8_t day = 0;
3585     uint8_t month = 0;
3586     uint16_t year = 0;
3587     if (!ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) || !rtcDateLooksSet(day, month, year)) {
3588         return "DATE UNSET";
3589     }
3590
3591     char buffer[18];
3592     snprintf(buffer, sizeof(buffer), "%02u/%02u %02u:%02u", month, day, hour, minute);
3593     return String(buffer);
3594 }
3595
3596 bool updateRtcStatusCache(bool force) {
3597     uint32_t nowMs = millis();
3598     if (!force && nowMs - lastRtcStatusMs < 1000UL) {
3599         return false;
3600     }
3601
3602     String next = rtcStatusText();

```

```

3603     bool changed = force || next != cachedRtcStatus;
3604     cachedRtcStatus = next;
3605     lastRtcStatusMs = nowMs;
3606     return changed;
3607 }
3608
3609 void scanI2C(const char *label) {
3610     logPrintf("I2C scan %s:", label);
3611     bool any = false;
3612     for (uint8_t address = 1; address < 127; address++) {
3613         Wire.beginTransmission(address);
3614         if (Wire.endTransmission() == 0) {
3615             logPrintf(" 0x%02X", address);
3616             any = true;
3617         }
3618     }
3619     logLine(any ? "" : " none");
3620 }
3621
3622 void resetTouchController(uint8_t outputValue, uint8_t configValue) {
3623     if (pca9557Address == 0) {
3624         return;
3625     }
3626     pca9557Write(pca9557Address, 0x03, 0x00);
3627     pca9557Write(pca9557Address, 0x01, outputValue);
3628     delay(20);
3629     pca9557Write(pca9557Address, 0x03, configValue);
3630     delay(80);
3631 }
3632
3633 void initTouch() {
3634     scanI2C("before GT911 init");
3635
3636     const uint8_t addresses[] = {0x5D, 0x14};
3637     for (uint8_t i = 0; i < sizeof(addresses); i++) {
3638         Wire.beginTransmission(addresses[i]);
3639         if (Wire.endTransmission() == 0) {
3640             gt911Address = addresses[i];
3641             break;
3642         }
3643     }
3644
3645     if (gt911Address == 0) {
3646         const uint8_t resetOutputs[] = {0x01, 0x00, 0x03, 0x02};
3647         const uint8_t resetConfigs[] = {0x02, 0x02, 0x00, 0x00};
3648         for (uint8_t attempt = 0; attempt < 4 && gt911Address == 0; attempt++) {
3649             resetTouchController(resetOutputs[attempt], resetConfigs[attempt]);
3650             scanI2C("after touch reset attempt");
3651             for (uint8_t i = 0; i < sizeof(addresses); i++) {
3652                 Wire.beginTransmission(addresses[i]);
3653                 if (Wire.endTransmission() == 0) {
3654                     gt911Address = addresses[i];
3655                     break;
3656                 }
3657             }
3658         }
3659     }
3660
3661     if (gt911Address == 0) {
3662         logLine("GT911 touch controller not found. Touch disabled.");
3663         return;
3664     }
3665
3666     uint8_t productId[4] = {};
3667     gt911Read(0x8140, productId, sizeof(productId));
3668     logPrintf("GT911 found at 0x%02X id=%c%c%c%c\n", gt911Address,
3669             productId[0], productId[1], productId[2], productId[3]);
3670     gt911Write(0x814E, 0x00);
3671 }
3672
3673 TouchPoint readTouch() {
3674     TouchPoint point = {false, 0, 0};
3675     if (gt911Address == 0) {
3676         return point;
3677     }
3678
3679     uint8_t status = 0;
3680     if (!gt911Read(0x814E, &status, 1)) {
3681         return point;
3682     }
3683
3684     uint8_t count = status & 0x0F;
3685     if (count == 0) {
3686         if (status & 0x80) {
3687             gt911Write(0x814E, 0x00);
3688         }

```

```

3689     return point;
3690 }
3691 if (count > 5) {
3692     gt911Write(0x814E, 0x00);
3693     return point;
3694 }
3695
3696 uint8_t data[7] = {};
3697 if (gt911Read(0x8150, data, sizeof(data))) {
3698     uint16_t xLowHigh = ((uint16_t)data[1] << 8) | data[0];
3699     uint16_t yLowHigh = ((uint16_t)data[3] << 8) | data[2];
3700     uint16_t xHighLow = ((uint16_t)data[0] << 8) | data[1];
3701     uint16_t yHighLow = ((uint16_t)data[2] << 8) | data[3];
3702     bool lowHighValid = xLowHigh < PANEL_WIDTH && yLowHigh < PANEL_HEIGHT;
3703     bool highLowValid = xHighLow < PANEL_WIDTH && yHighLow < PANEL_HEIGHT;
3704
3705     if (lowHighValid || highLowValid) {
3706         point.touched = true;
3707         point.x = lowHighValid ? xLowHigh : xHighLow;
3708         point.y = lowHighValid ? yLowHigh : yHighLow;
3709     } else {
3710         uint16_t shiftedXLowHigh = ((uint16_t)data[2] << 8) | data[1];
3711         uint16_t shiftedYLowHigh = ((uint16_t)data[4] << 8) | data[3];
3712         uint16_t shiftedXHighLow = ((uint16_t)data[1] << 8) | data[2];
3713         uint16_t shiftedYHighLow = ((uint16_t)data[3] << 8) | data[4];
3714         bool shiftedLowHighValid = shiftedXLowHigh < PANEL_WIDTH && shiftedYLowHigh < PANEL_HEIGHT;
3715         bool shiftedHighLowValid = shiftedXHighLow < PANEL_WIDTH && shiftedYHighLow < PANEL_HEIGHT;
3716
3717         if (shiftedLowHighValid || shiftedHighLowValid) {
3718             point.touched = true;
3719             point.x = shiftedLowHighValid ? shiftedXLowHigh : shiftedXHighLow;
3720             point.y = shiftedLowHighValid ? shiftedYLowHigh : shiftedYHighLow;
3721         }
3722     }
3723 }
3724
3725 gt911Write(0x814E, 0x00);
3726 return point;
3727 }
3728
3729 bool gt911Read(uint16_t reg, uint8_t *data, uint8_t len) {
3730     Wire.beginTransmission(gt911Address);
3731     Wire.write(reg >> 8);
3732     Wire.write(reg & 0xFF);
3733     if (Wire.endTransmission(false) != 0) {
3734         return false;
3735     }
3736     uint8_t received = Wire.requestFrom((int)gt911Address, (int)len);
3737     if (received != len) {
3738         return false;
3739     }
3740
3741     for (uint8_t i = 0; i < len; i++) {
3742         data[i] = Wire.read();
3743     }
3744     return true;
3745 }
3746
3747 bool gt911Write(uint16_t reg, uint8_t value) {
3748     Wire.beginTransmission(gt911Address);
3749     Wire.write(reg >> 8);
3750     Wire.write(reg & 0xFF);
3751     Wire.write(value);
3752     return Wire.endTransmission() == 0;
3753 }
3754
3755 void logLine(const char *message) {
3756     Serial.println(message);
3757     Serial0.println(message);
3758 }
3759
3760 void logPrintf(const char *format, ...) {
3761     char buffer[160];
3762     va_list args;
3763     va_start(args, format);
3764     vsnprintf(buffer, sizeof(buffer), format, args);
3765     va_end(args);
3766
3767     Serial.print(buffer);
3768     Serial0.print(buffer);
3769 }
3770
3771 void drawCentered(const String &text, int16_t y, uint8_t textSize, uint16_t color) {
3772     tft.setTextSize(textSize);
3773     tft.setTextColor(color, TFT_BACKGROUND);
3774     int16_t x1;

```

```

3775     int16_t y1;
3776     uint16_t w;
3777     uint16_t h;
3778     tft.getTextBounds(text, 0, y, &x1, &y1, &w, &h);
3779     tft.setCursor((tft.width() - w) / 2, y);
3780     tft.print(text);
3781 }
3782
3783 void drawCenteredInRect(const String &text, const Rect &region, uint8_t textSize, uint16_t color, uint16_t background) {
3784     #if BGMC_HAS_SMALL_UI_FONT
3785     bool useUIFont = (textSize >= 2 && textSize <= 3);
3786     if (useUIFont) {
3787         tft.setFont(&FreeSans12pt7b);
3788         tft.setTextSize(1);
3789     } else {
3790         tft.setFont();
3791         tft.setTextSize(textSize);
3792     }
3793     #elif BGMC_HAS_UI_FONT
3794     bool useUIFont = (textSize >= 2 && textSize <= 3);
3795     if (useUIFont) {
3796         tft.setFont(&FreeSans18pt7b);
3797         tft.setTextSize(1);
3798     } else {
3799         tft.setFont();
3800         tft.setTextSize(textSize);
3801     }
3802     #else
3803     tft.setFont();
3804     tft.setTextSize(textSize);
3805     #endif
3806     tft.setTextColor(color, background);
3807     int16_t x1;
3808     int16_t y1;
3809     uint16_t w;
3810     uint16_t h;
3811     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3812     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3813     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3814     tft.setCursor(x, y);
3815     tft.print(text);
3816     tft.setFont();
3817 }
3818
3819 void drawScoreInRect(int score, const Rect &region, uint16_t color, uint16_t background) {
3820     String text = String(score);
3821     #if BGMC_HAS_COLLEGE_SCORE_FONT
3822     tft.setFont(&BGMC_COLLEGE_SCORE_FONT);
3823     tft.setTextSize(1);
3824     #elif BGMC_HAS_SCORE_96_FONT
3825     tft.setFont(&FreeSansBold96pt7b);
3826     tft.setTextSize(1);
3827     #elif BGMC_HAS_SCORE_FONT
3828     tft.setFont(&FreeSansBold24pt7b);
3829     tft.setTextSize(4);
3830     #elif BGMC_HAS_UI_FONT
3831     tft.setFont(&FreeSans18pt7b);
3832     tft.setTextSize(3);
3833     #else
3834     tft.setFont();
3835     tft.setTextSize(10);
3836     #endif
3837     tft.setTextColor(color, background);
3838     int16_t x1;
3839     int16_t y1;
3840     uint16_t w;
3841     uint16_t h;
3842     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3843     #if BGMC_HAS_COLLEGE_SCORE_FONT || BGMC_HAS_SCORE_96_FONT
3844     if (w > region.w - 28 || h > region.h - 28) {
3845         #if BGMC_HAS_SCORE_FONT
3846         tft.setFont(&FreeSansBold24pt7b);
3847         tft.setTextSize(4);
3848         #else
3849         tft.setFont();
3850         tft.setTextSize(10);
3851         #endif
3852         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3853     }
3854     #endif
3855     #if BGMC_HAS_SCORE_FONT
3856     uint8_t scoreSize = 4;
3857     while (scoreSize > 1 && (w > region.w - 28 || h > region.h - 28)) {
3858         scoreSize--;
3859         tft.setTextSize(scoreSize);
3860         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);

```

```

3861     }
3862 #endif
3863     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3864     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3865     tft.setCursor(x, y);
3866     tft.print(text);
3867     tft.setFont();
3868 }
3869
3870 void drawMatchTargetInRect(int target, const Rect &region, uint16_t color, uint16_t background) {
3871     String text = String(target);
3872     #if BGMC_HAS_MATCH_TARGET_48_FONT
3873     {
3874         tft.setFont(&BGMC_MATCH_TARGET_48_FONT);
3875         tft.setTextSize(1);
3876         tft.setTextColor(color, background);
3877         int16_t x1;
3878         int16_t y1;
3879         uint16_t w;
3880         uint16_t h;
3881         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3882         int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3883         int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3884         tft.setCursor(x, y);
3885         tft.print(text);
3886         tft.setFont();
3887     }
3888     return;
3889 #elif BGMC_HAS_SCORE_FONT
3890     (void)background;
3891     drawGfxTextScaledInRect(&FreeSansBold24pt7b, text, region, color, 3, 2);
3892     return;
3893 #elif BGMC_HAS_SMALL_UI_FONT
3894     tft.setFont(&FreeSans12pt7b);
3895     tft.setTextSize(3);
3896 #elif BGMC_HAS_UI_FONT
3897     tft.setFont(&FreeSans18pt7b);
3898     tft.setTextSize(2);
3899 #else
3900     tft.setFont();
3901     tft.setTextSize(7);
3902 #endif
3903     tft.setTextColor(color, background);
3904     int16_t x1;
3905     int16_t y1;
3906     uint16_t w;
3907     uint16_t h;
3908     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3909     #if BGMC_HAS_SMALL_UI_FONT
3910     if (w > region.w - 24 || h > region.h - 8) {
3911         tft.setTextSize(2);
3912         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3913     }
3914 #endif
3915     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3916     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3917     tft.setCursor(x, y);
3918     tft.print(text);
3919     tft.setFont();
3920 }
3921
3922 void drawGfxTextScaledInRect(const GFXfont *font, const String &text, const Rect &region, uint16_t color,
3923                             uint8_t scaleNum, uint8_t scaleDen) {
3924     int16_t x1;
3925     int16_t y1;
3926     uint16_t w;
3927     uint16_t h;
3928     getGfxTextBoundsScaled(font, text, scaleNum, scaleDen, &x1, &y1, &w, &h);
3929
3930     if (w > region.w - 24 || h > region.h - 8) {
3931         scaleNum = 1;
3932         scaleDen = 1;
3933         getGfxTextBoundsScaled(font, text, scaleNum, scaleDen, &x1, &y1, &w, &h);
3934     }
3935
3936     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3937     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3938     drawGfxTextScaled(font, text, x, y, color, scaleNum, scaleDen);
3939 }
3940
3941 void getGfxTextBoundsScaled(const GFXfont *font, const String &text, uint8_t scaleNum, uint8_t scaleDen,
3942                             int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
3943     int16_t minX = 32767;
3944     int16_t minY = 32767;
3945     int16_t maxX = -32768;
3946     int16_t maxY = -32768;

```

```

3947 int16_t cursor = 0;
3948 uint16_t first = pgm_read_word(&font->first);
3949 uint16_t last = pgm_read_word(&font->last);
3950 const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&font->glyph);
3951
3952 for (uint16_t i = 0; i < text.length(); i++) {
3953     char c = text.charAt(i);
3954     if (c < first || c > last) {
3955         continue;
3956     }
3957     GFXglyph glyph;
3958     memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
3959     if (glyph.width > 0 && glyph.height > 0) {
3960         int16_t gx1 = cursor + ((int32_t)glyph.xOffset * scaleNum) / scaleDen;
3961         int16_t gy1 = ((int32_t)glyph.yOffset * scaleNum) / scaleDen;
3962         int16_t gx2 = cursor + ((int32_t)(glyph.xOffset + glyph.width) * scaleNum + scaleDen - 1) / scaleDen - 1;
3963         int16_t gy2 = ((int32_t)(glyph.yOffset + glyph.height) * scaleNum + scaleDen - 1) / scaleDen - 1;
3964         if (gx1 < minX) minX = gx1;
3965         if (gy1 < minY) minY = gy1;
3966         if (gx2 > maxX) maxX = gx2;
3967         if (gy2 > maxY) maxY = gy2;
3968     }
3969     cursor += ((int32_t)glyph.xAdvance * scaleNum + scaleDen - 1) / scaleDen;
3970 }
3971
3972 if (maxX < minX || maxY < minY) {
3973     *x1 = 0;
3974     *y1 = 0;
3975     *w = 0;
3976     *h = 0;
3977     return;
3978 }
3979 *x1 = minX;
3980 *y1 = minY;
3981 *w = maxX - minX + 1;
3982 *h = maxY - minY + 1;
3983 }
3984
3985 void drawGfxTextScaled(const GFXfont *font, const String &text, int16_t x, int16_t y, uint16_t color,
3986                       uint8_t scaleNum, uint8_t scaleDen) {
3987     int16_t cursor = x;
3988     uint16_t first = pgm_read_word(&font->first);
3989     uint16_t last = pgm_read_word(&font->last);
3990     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&font->glyph);
3991     const uint8_t *bitmap = (const uint8_t *)pgm_read_ptr(&font->bitmap);
3992
3993     for (uint16_t i = 0; i < text.length(); i++) {
3994         char c = text.charAt(i);
3995         if (c < first || c > last) {
3996             continue;
3997         }
3998
3999         GFXglyph glyph;
4000         memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
4001         uint16_t bitIndex = 0;
4002         uint16_t bitmapOffset = glyph.bitmapOffset;
4003         uint8_t bits = 0;
4004
4005         for (uint8_t yy = 0; yy < glyph.height; yy++) {
4006             int16_t rowTop = y + ((int32_t)(glyph.yOffset + yy) * scaleNum) / scaleDen;
4007             int16_t rowBottom = y + ((int32_t)(glyph.yOffset + yy + 1) * scaleNum + scaleDen - 1) / scaleDen;
4008             int16_t rowH = max((int16_t)1, (int16_t)(rowBottom - rowTop));
4009
4010             for (uint8_t xx = 0; xx < glyph.width; xx++) {
4011                 if ((bitIndex & 7) == 0) {
4012                     bits = pgm_read_byte(&bitmap[bitmapOffset++]);
4013                 }
4014                 if (bits & 0x80) {
4015                     int16_t colLeft = cursor + ((int32_t)(glyph.xOffset + xx) * scaleNum) / scaleDen;
4016                     int16_t colRight = cursor + ((int32_t)(glyph.xOffset + xx + 1) * scaleNum + scaleDen - 1) / scaleDen;
4017                     int16_t colW = max((int16_t)1, (int16_t)(colRight - colLeft));
4018                     tft.fillRect(colLeft, rowTop, colW, rowH, color);
4019                 }
4020                 bits <<= 1;
4021                 bitIndex++;
4022             }
4023         }
4024         cursor += ((int32_t)glyph.xAdvance * scaleNum + scaleDen - 1) / scaleDen;
4025     }
4026 }
4027
4028 String formatTime(uint32_t seconds) {
4029     uint32_t minutes = seconds / 60;
4030     uint32_t secs = seconds % 60;
4031     char buffer[12];
4032     sprintf(buffer, sizeof(buffer), "%02lu:%02lu", (unsigned long)minutes, (unsigned long)secs);

```

```

4033     return String(buffer);
4034 }
4035
4036 String formatTimeMs(uint32_t milliseconds) {
4037     uint32_t seconds = (milliseconds + 999UL) / 1000UL;
4038     return formatTime(seconds);
4039 }
4040
4041 String timingModeLabel() {
4042     switch (timingMode) {
4043         case BRONSTEIN_DELAY:
4044             return "Bronstein";
4045         case FISCHER_INCREMENT:
4046             return "Fischer";
4047         case SIMPLE_DELAY:
4048             default:
4049                 return "Simple";
4050     }
4051 }
4052
4053 String playPauseLabel() {
4054     if (clock_running) {
4055         return "Pause Game";
4056     }
4057     if (game_number == 0 && !game_in_progress) {
4058         return "Start Match";
4059     }
4060     return "Play Game";
4061 }
4062
4063 String matchStateLabel() {
4064     if (crawford_active) {
4065         return "CRAWFORD GAME - CUBE OFF";
4066     }
4067     if (post_crawford) {
4068         return "POST-CRAWFORD - CUBE ON";
4069     }
4070     return "NORMAL";
4071 }
4072
4073 int batteryPercent() {
4074     if (max17048Found) {
4075         return (int)(max17048StateOfCharge() + 0.5f);
4076     }
4077
4078     if (PIN_BATTERY_ADC == 255) {
4079         return -1;
4080     }
4081
4082     int raw = analogRead(PIN_BATTERY_ADC);
4083     float adcVolts = ((float)raw / 4095.0f) * BATTERY_ADC_REF_VOLTS;
4084     float batteryVolts = adcVolts * BATTERY_DIVIDER_RATIO;
4085     float pct = (batteryVolts - BATTERY_EMPTY_VOLTS) * 100.0f /
4086         (BATTERY_FULL_VOLTS - BATTERY_EMPTY_VOLTS);
4087     if (pct < 0.0f) {
4088         pct = 0.0f;
4089     }
4090     if (pct > 100.0f) {
4091         pct = 100.0f;
4092     }
4093     return (int)(pct + 0.5f);
4094 }
4095
4096 bool chargingIndicatorOn() {
4097     if (PIN_CHARGE_DETECT == 255) {
4098         if (!max17048Found) {
4099             return false;
4100         }
4101
4102         uint32_t nowMs = millis();
4103         if (lastChargeCheckMs == 0 || nowMs - lastChargeCheckMs >= 30000UL) {
4104             float voltage = max17048CellVoltage();
4105             float percent = max17048StateOfCharge();
4106             if (lastChargeCheckMs != 0) {
4107                 chargeTrendRising = (voltage > lastChargeCheckVoltage + 0.006f) ||
4108                     (percent > lastChargeCheckPercent + 0.08f);
4109                 if (chargeTrendRising) {
4110                     chargeIndicatorUntilMs = nowMs + 120000UL;
4111                 }
4112             }
4113             lastChargeCheckVoltage = voltage;
4114             lastChargeCheckPercent = percent;
4115             lastChargeCheckMs = nowMs;
4116         }
4117
4118         return chargeTrendRising || nowMs < chargeIndicatorUntilMs;

```

```

4119     }
4120     return digitalRead(PIN_CHARGE_DETECT) == LOW;
4121 }
4122
4123 void recordMatchLog() {
4124     String entry = matchLogDate();
4125     entry += "|";
4126     entry += playerA_name;
4127     entry += "|";
4128     entry += playerB_name;
4129     entry += "|";
4130     entry += String(playerA_score);
4131     entry += "-";
4132     entry += String(playerB_score);
4133
4134     for (int8_t i = MATCH_LOG_MAX_ENTRIES - 1; i > 0; i--) {
4135         matchLog[i] = matchLog[i - 1];
4136     }
4137     matchLog[0] = entry;
4138     if (matchLogCount < MATCH_LOG_MAX_ENTRIES) {
4139         matchLogCount++;
4140     }
4141
4142     prefs.putUChar("logCount", matchLogCount);
4143     for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
4144         char key[8];
4145         snprintf(key, sizeof(key), "log%u", i);
4146         prefs.putString(key, matchLog[i]);
4147     }
4148 }
4149
4150 void clearMatchLog() {
4151     matchLogCount = 0;
4152     prefs.putUChar("logCount", matchLogCount);
4153     for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
4154         matchLog[i] = "";
4155         char key[8];
4156         snprintf(key, sizeof(key), "log%u", i);
4157         prefs.putString(key, "");
4158     }
4159 }
4160
4161 String matchLogDate() {
4162     uint8_t second = 0;
4163     uint8_t minute = 0;
4164     uint8_t hour = 0;
4165     uint8_t day = 0;
4166     uint8_t month = 0;
4167     uint16_t year = 0;
4168     if (!ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) || !rtcDateLooksSet(day, month, year)) {
4169         return "DATE UNSET";
4170     }
4171
4172     char buffer[12];
4173     snprintf(buffer, sizeof(buffer), "%04u-%02u-%02u", year, month, day);
4174     return String(buffer);
4175 }
4176
4177 String matchLogPart(const String &entry, uint8_t part) {
4178     int start = 0;
4179     for (uint8_t i = 0; i < part; i++) {
4180         int separator = entry.indexOf('|', start);
4181         if (separator < 0) {
4182             return "";
4183         }
4184         start = separator + 1;
4185     }
4186
4187     int end = entry.indexOf('|', start);
4188     if (end < 0) {
4189         return entry.substring(start);
4190     }
4191     return entry.substring(start, end);
4192 }
4193
4194 // -----
4195 // Settings and power management
4196 // -----
4197 void loadSettings() {
4198     prefs.begin("bgclock", false);
4199     match_target = prefs.getInt("target", 9);
4200     initial_time_seconds = prefs.getULong("time", DEFAULT_INITIAL_TIME_SECONDS);
4201     delay_seconds = prefs.getUChar("delay", DEFAULT_DELAY_SECONDS);
4202     if (delay_seconds == 7) {
4203         delay_seconds = DEFAULT_DELAY_SECONDS;
4204         prefs.putUChar("delay", delay_seconds);

```

```

4205 }
4206 increment_seconds = prefs.getUChar("inc", DEFAULT_INCREMENT_SECONDS);
4207 brightness = prefs.getUChar("bright", DEFAULT_BRIGHTNESS);
4208 timingMode = (TimingMode)prefs.getUChar("timing", (uint8_t)SIMPLE_DELAY);
4209 if (timingMode > FISCHER_INCREMENT) {
4210     timingMode = SIMPLE_DELAY;
4211 }
4212 timerScope = (TimerScope)prefs.getUChar("scope", (uint8_t)TIMER_BY_MATCH);
4213 if (timerScope > TIMER_BY_MATCH) {
4214     timerScope = TIMER_BY_MATCH;
4215 }
4216 displayTheme = (DisplayTheme)prefs.getUChar("theme", (uint8_t)THEME_NIGHT);
4217 if (displayTheme > THEME_DAY) {
4218     displayTheme = THEME_NIGHT;
4219 }
4220 scoreDisplayMode = (ScoreDisplayMode)prefs.getUChar("scoreView", (uint8_t)SCORE_STANDARD);
4221 if (scoreDisplayMode > SCORE_POINTS_TO_WIN) {
4222     scoreDisplayMode = SCORE_STANDARD;
4223 }
4224 playerA_name = prefs.getString("nameA", "PLAYER A");
4225 playerB_name = prefs.getString("nameB", "PLAYER B");
4226 playerA_color = prefs.getUShort("colorA", PLAYER_COLOR_BLUE);
4227 playerB_color = prefs.getUShort("colorB", PLAYER_COLOR_RED);
4228 if (playerA_name.length() == 0 || playerA_name.length() > PLAYER_NAME_MAX_LEN) {
4229     playerA_name = "PLAYER A";
4230 }
4231 if (playerB_name.length() == 0 || playerB_name.length() > PLAYER_NAME_MAX_LEN) {
4232     playerB_name = "PLAYER B";
4233 }
4234
4235 matchLogCount = prefs.getUChar("logCount", 0);
4236 if (matchLogCount > MATCH_LOG_MAX_ENTRIES) {
4237     matchLogCount = MATCH_LOG_MAX_ENTRIES;
4238 }
4239 for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
4240     char key[8];
4241     snprintf(key, sizeof(key), "log%u", i);
4242     matchLog[i] = prefs.getString(key, "");
4243 }
4244
4245 bool validTarget = false;
4246 for (uint8_t i = 0; i < MATCH_TARGET_COUNT; i++) {
4247     if (MATCH_TARGETS[i] == match_target) {
4248         validTarget = true;
4249     }
4250 }
4251 if (!validTarget) {
4252     match_target = 9;
4253 }
4254 }
4255
4256 void saveSettingsIfNeeded() {
4257     if (!settingsDirty) {
4258         return;
4259     }
4260
4261     uint32_t nowMs = millis();
4262     if (nowMs - settingsDirtyMs < SETTINGS_SAVE_DELAY_MS) {
4263         return;
4264     }
4265
4266     prefs.putInt("target", match_target);
4267     prefs.putULong("time", initial_time_seconds);
4268     prefs.putUChar("delay", delay_seconds);
4269     prefs.putUChar("inc", increment_seconds);
4270     prefs.putUChar("bright", brightness);
4271     prefs.putUChar("timing", (uint8_t)timingMode);
4272     prefs.putUChar("scope", (uint8_t)timerScope);
4273     prefs.putUChar("theme", (uint8_t)displayTheme);
4274     prefs.putUChar("scoreView", (uint8_t)scoreDisplayMode);
4275     settingsDirty = false;
4276 }
4277
4278 void savePlayerNames() {
4279     prefs.putString("nameA", playerA_name);
4280     prefs.putString("nameB", playerB_name);
4281     prefs.putUShort("colorA", playerA_color);
4282     prefs.putUShort("colorB", playerB_color);
4283 }
4284
4285 void markSettingsDirty() {
4286     settingsDirty = true;
4287     settingsDirtyMs = millis();
4288     markDisplayDirty();
4289 }
4290

```

```

4291 void markDisplayDirty() {
4292     displayDirty = true;
4293 }
4294
4295 void setBrightness(uint8_t duty) {
4296     if (duty == 0) {
4297         digitalWrite(PIN_TFT_BACKLIGHT, TFT_BACKLIGHT_ACTIVE_HIGH ? LOW : HIGH);
4298         return;
4299     }
4300     digitalWrite(PIN_TFT_BACKLIGHT, TFT_BACKLIGHT_ACTIVE_HIGH ? HIGH : LOW);
4301 }
4302
4303 void wakeActivity() {
4304     lastActivityMs = millis();
4305 }
4306
4307 void maybeSleep() {
4308     if (clock_running) {
4309         return;
4310     }
4311
4312     if (PIN_PLAYER_A_BUTTON == 255 || PIN_PLAYER_B_BUTTON == 255 || PIN_MENU_SELECT == 255) {
4313         return;
4314     }
4315
4316     uint32_t nowMs = millis();
4317     if (nowMs - lastActivityMs < SLEEP_AFTER_IDLE_MS) {
4318         return;
4319     }
4320
4321     tft.fillScreen(ILI9341_BLACK);
4322     setBrightness(0);
4323     delay(20); // Tiny hardware settle before deep sleep.
4324
4325     // Wake on the two large player buttons and menu button. Add more pins if
4326     // your board supports the required EXT1 wake bitmask for those GPIOs.
4327     uint64_t wakeMask = (1ULL << PIN_PLAYER_A_BUTTON) |
4328         (1ULL << PIN_PLAYER_B_BUTTON) |
4329         (1ULL << PIN_MENU_SELECT);
4330     esp_sleep_enable_ext1_wakeup(wakeMask, ESP_EXT1_WAKEUP_ANY_LOW);
4331     esp_deep_sleep_start();
4332 }
4333
4334 // -----
4335 // Buzzer feedback
4336 // -----
4337 void beep(uint16_t frequency, uint16_t durationMs) {
4338     if (PIN_BUZZER == 255) {
4339         return;
4340     }
4341
4342     tone(PIN_BUZZER, frequency, durationMs);
4343 }
4344
4345 void playButtonBeep() {
4346     beep(1800, 20);
4347 }
4348
4349 void playTimeoutBeep() {
4350     beep(900, 300);
4351 }
4352
4353 void playMatchOverBeep() {
4354     beep(1600, 120);
4355     // tone() is non-blocking on most Arduino cores, so keep this simple.
4356 }
4357

```